

GATE CSE

Operating Systems

The Complete Beginner-to-GATE-Ready Guide

Concepts • Shortcuts • Solved examples • Practice sets

Processes • Scheduling • Synchronization • Deadlocks • Memory • Storage

Contents

How to Use This Book 1

0.1 How to use this book . . . 1
0.2 What GATE actually asks in OS (from your PYQ analysis) . . . 1
0.3 Suggested study plan . . . 4
0.4 Notation used everywhere . . . 4

Part I • Foundations . . . 5

1 OS Basics: Modes, System Calls, Interrupts, Boot 6

1.1 What is an Operating System? (60-second version) . . . 6
1.2 Dual-mode operation (user mode vs kernel mode) . . . 7
1.3 System calls . . . 7
1.4 Interrupts, traps and exceptions . . . 8
1.5 System boot (how the OS gets started) — low yield, 5 lines . . . 9
1.6 Quick-revision capsule . . . 9
1.7 MSQ drill (with reasons) . . . 9

2 Processes, PCB, Context Switch, fork / exec / wait 10

2.1 Program vs Process . . . 10
2.2 A process in memory . . . 11
2.3 Process Control Block (PCB) . . . 11
2.4 Process states and the schedulers . . . 12
2.5 Context switch . . . 12
2.6 fork(), exec(), wait() — the heart of this chapter . . . 13
2.7 How to count processes (the mechanical method) . . . 14
2.8 Quick-revision capsule . . . 17
2.9 Practice set (answers verified by running the programs) . . . 17

3 Threads 19

3.1 The problem threads solve . . . 19
3.2 Parallelism vs concurrency . . . 20
3.3 Why a process is “heavy” and a thread is “light” . . . 20
3.4 How are threads implemented? — User-level vs Kernel-level . . . 20
3.5 Thread vs Process — the comparison table to memorise . . . 21
3.6 Threads + shared variables = race conditions (preview of Chapter 5) . . . 22
3.7 Quick-revision capsule . . . 23
3.8 Practice set . . . 23

Part II • CPU Scheduling . . . 24

4 CPU Scheduling 25

4.1 Why do we need a scheduler? . . . 25
4.2 The terms (learn these 7 and the formulas — everything else follows) . . . 26
4.3 The universal recipe to solve any scheduling numerical . . . 26
4.4 FCFS — First Come First Served . . . 27
4.5 SJF — Shortest Job First (non-preemptive) . . . 28

4.6 SRTF — Shortest Remaining Time First (preemptive SJF) . . . 29
4.7 Round Robin (RR) . . . 30
4.8 LJF and LRTF . . . 32
4.9 Priority scheduling . . . 33
4.10 Three more algorithms GATE may mention (brief) . . . 33
4.11 Scheduling with I/O bursts . . . 34
4.12 Multi-CPU (multiprocessor) scheduling . . . 35
4.13 Master cheat-sheet: which algorithm is best / worst at what? . . . 35
4.14 MSQ / statement bank (with reasons) . . . 36
4.15 Conventions and traps in scheduling questions . . . 36
4.16 Quick-revision capsule . . . 36
4.17 Practice set (GATE style) — try on paper first . . . 37

Part III • Concurrency . . . 39

5 Process Synchronization 40
5.1 The problem: shared data + interleaving = wrong answers . . . 40
5.2 The Critical Section (CS) problem . . . 41
5.3 How to check a CS solution (the 5-minute checklist) . . . 42
5.4 Software solutions for two contexts (T0 and T1) . . . 42
5.5 Hardware support — atomic instructions . . . 43
5.6 Blocking locks (mutex) — don't waste the CPU . . . 44
5.7 Semaphores (the most important tool) . . . 45
5.8 The three classic problems . . . 45
5.9 Semaphore puzzles — how to solve them . . . 47
5.10 Fork/threads + shared variables (link back to Chapter 3) . . . 48
5.11 MSQ / statement bank (with reasons) . . . 48
5.12 Quick-revision capsule . . . 49
5.13 Practice set (GATE style) . . . 49

6 Deadlocks 51
6.1 What is a deadlock? . . . 51
6.2 The four necessary conditions (Coffman) — all four must hold together . . . 51
6.3 Strategies to handle deadlock . . . 52
6.4 Deadlock prevention — break one condition . . . 52
6.5 Resource-Allocation Graph (RAG) . . . 52
6.6 Deadlock avoidance — the min-resources formulas . . . 53
6.7 Banker's algorithm (avoidance with multiple instances) . . . 53
6.8 Deadlock detection and recovery . . . 56
6.9 Comparison: prevention / avoidance / detection . . . 56
6.10 MSQ / statement bank . . . 57
6.11 Quick-revision capsule . . . 57
6.12 Practice set . . . 57

Part IV • Memory . . . 59

7 Memory Management Basics: Address Spaces, Protection, Contiguous Allocation 60
7.1 What is an address space? . . . 60
7.2 The MMU and the base–limit scheme . . . 61
7.3 Contiguous memory allocation . . . 61
7.4 MSQ / statement bank . . . 63
7.5 Quick-revision capsule . . . 63

7.6 Practice . . . 63

8 Paging, TLB, Multilevel Paging, Segmentation 64

8.1 Why paging? (continuing Chapter 7) . . . 64
8.2 How an address is translated . . . 64
8.3 The formula sheet (memorise) . . . 65
8.4 What is inside a page-table entry? . . . 66
8.5 TLB — Translation Lookaside Buffer . . . 66
8.6 Sharing pages and dynamic linking . . . 67
8.7 Multilevel (hierarchical) paging — the highest-yield part of the chapter . . . 67
8.8 Inverted page table . . . 69
8.9 Segmentation . . . 69
8.10 Optimal page size . . . 70
8.11 MSQ / statement bank . . . 70
8.12 Quick-revision capsule . . . 70
8.13 Practice set (GATE style) . . . 70

9 Virtual Memory, Page Faults, Page Replacement, Thrashing 72

9.1 The idea of virtual memory . . . 72
9.2 Page fault . . . 72
9.3 Performance of demand paging — EAT with page faults . . . 73
9.4 Page replacement — the common set-up . . . 74
9.5 The algorithms with worked tables . . . 74
9.6 Allocation of frames . . . 77
9.7 Thrashing . . . 78
9.8 MSQ / statement bank . . . 78
9.9 Quick-revision capsule . . . 79
9.10 Practice set (GATE style) . . . 79

Part V • Storage . . . 81

10 File Systems: Files, Directories, Inodes, Allocation, Free Space 82

10.1 What is a file system? . . . 82
10.2 Inode . . . 82
10.3 Directory structures . . . 83
10.4 Disk layout (old Unix) . . . 83
10.5 File allocation (implementation) methods . . . 83
10.6 The three arithmetic templates . . . 85
10.7 Free-space management . . . 86
10.8 MSQ / statement bank . . . 86
10.9 Quick-revision capsule . . . 86
10.10 Practice set (GATE style) . . . 86

11 Disk Structure, Disk Scheduling, and I/O 88

11.1 Disk structure . . . 88
11.2 Disk access time — the formula you must know . . . 88
11.3 Disk scheduling . . . 89
11.4 Input/Output (I/O) methods . . . 90
11.5 MSQ / statement bank . . . 92
11.6 Quick-revision capsule . . . 92
11.7 Practice set . . . 92

Part VI • Exam Toolkit **94**

12 Formula Sheet, Trap List, Mixed Practice **95**

12.1 Master formula sheet 95

12.2 The trap list (every item has cost someone marks) 96

12.3 Mixed practice — 12 problems (30 minutes) 97

12.4 Final-week checklist 98

A PYQ-style playbook (what to do, in what order, with time budgets) **99**

B Conventions used in this book (and what to do when the question disagrees) **100**

C Corrections to the lecture notes (small slips, fixed in this book) **101**

START HERE

How to Use This Book

Who this book is for: someone who has never studied Operating Systems (OS) and wants to be able to solve GATE questions on it — without sitting through 40+ hours of lectures.
What this book is built from: 1. A student's handwritten GATE lecture notes (118 pages) — *every topic, every example, every shortcut in those notes is covered here.* 2. Your **GATE PYQ Analysis** (1987–2026, 3,823 questions) — used to decide **how much space each topic gets**. High-frequency topics get extra depth and extra solved problems; low-yield introductory material is trimmed. 3. Additions by me, clearly labelled, for topics that GATE asks but the notes do not contain (disk scheduling, I/O, effective-access-time formulas, semaphore puzzles, etc.).

0.1 How to use this book

Symbol	Meaning
NOTES	**From the lecture notes** — example/idea taken from the handwritten notes (re-solved and re-checked)
ADDED	**Added by me** — not in the notes, but needed for GATE (comes from the PYQ analysis styles)
⚡	**Shortcut / trick** — time-saver for the exam
⚠	**Trap** — a mistake GATE loves to bait
◎	**GATE pattern** — what examiners actually ask on this topic (from your PYQ analysis)
✔	Quick self-check
★★★ / ★★ / ★	Importance of the chapter for GATE

Reading strategy (for a complete beginner):

1. Read each chapter top to bottom **once**, doing the NOTES examples *with a pen* (cover my solution, try, then compare).
2. Re-read only the ⚡ and ⚠ boxes the next day.
3. Do the "Practice set" at the end of the chapter. Every answer is worked out.
4. Only after that, go and solve the real PYQs yourself.

Everything numerical in this book was re-computed by program (Gantt charts, page-replacement tables, Banker's safety checks, disk-head movement, fork output orders, semaphore interleavings). Where a hand-written note had a small slip, I fixed it and listed the fix in **Appendix C**.

0.2 What GATE actually asks in OS (from your PYQ analysis)

Weight: OS gives ≈ **8.7 marks per paper** (range 6.5–10; last-5-papers average 8.6; GATE 2026 had 6.5). That's ≈ 9% of the paper. The **top 3 clusters carry 61%** of OS marks.

Topic clusters, ranked

Rank	Topic cluster	Tier	Avg marks / paper	Years hit ('18–'26)	Years hit ('14–'26)	Last 3 papers ('24 '25 '26)	1M · 2M Qs	NAT · MCQ · MSQ	Book chapter
1	**Virtual memory & paging** (TLB, page replacement)	**T1**	**2.39** ↑	9/9	13/13	✔✔✔	4 · 13	10 · 4 · 3	Ch 8, 9
2	**CPU / process scheduling**	**T1**	**1.50** →	8/9	12/13	✔✔✔	7 · 6	4 · 6 · 3	Ch 4
3	**Disk scheduling, file systems & I/O**	T2	1.44 ↓	7/9	11/13	✔✔·	3 · 7	7 · 2 · 1	Ch 10, 11
4	**Process synchronization** (semaphores, mutex, race)	**T1**	**1.22** ↓	8/9	12/13	✔·✔	2 · 6	1 · 4 · 3	Ch 5
5	**Deadlock** (Banker's, RAG)	T2	1.00 ↓	6/9	9/13	·✔✔	4 · 4	2 · 2 · 4	Ch 6
6	**Processes, threads, fork & system calls**	T3	0.94 ↑	5/9	7/13	✔·✔	6 · 4	3 · 1 · 6	Ch 1, 2, 3
7	Memory management (allocation, fragmentation)	T4	0.22 →	2/9	4/13	··✔	1 · 1	1 · 1 · 0	Ch 7

(*Tier 1 = asked in ≥ 8 of last 9 GATEs; Tier 2 = 6–7 of 9; Tier 3 = 4–5 of 9; Tier 4 = ≤ 3 of 9. Tiers are odds, not guarantees.*)

Marks per paper from each cluster (heat-map)

Cluster	'18	'19	'20	'21	'22	'23	'24	'25	'26	Avg
Virtual memory & paging	1	2	2	1.5	**4**	**4**	2.5	3.5	1	2.39
CPU / process scheduling	–	2	**3**	1	2	1	1.5	1.5	1.5	1.50
Disk scheduling, FS & I/O	**3**	2	2	0.5	2	–	2	1.5	–	1.44
Process synchronization	2	1	2	1	1	2	1	–	1	1.22
Deadlock	**3**	2	–	1	1	–	–	1	1	1.00
Processes, threads, fork, syscalls	–	1	–	1.5	–	2	**3**	–	1	0.94
Memory management	–	–	1	–	–	–	–	–	1	0.22
OS total	9	10	10	6.5	10	9	10	7.5	6.5	**8.7**

Exactly which *question styles* appear (and the years they appeared)

◎ Virtual memory & paging (T1, 2.39 marks)

- **Page-table geometry** — number of levels, entries/size of page table, address-bit split, page size — *7 Q* (5 NAT) — 2019 · 2021 S2 · 2023 · 2024 S1 · 2024 S2 · 2025 S1 · 2025 S2 → **Chapter 8**
- **Page-replacement trace** (FIFO/LRU/Optimal), count faults — *4 Q* (3 NAT) — 2022 · 2023 · 2025 S1 · 2025 S2 → **Chapter 9**
- **Paging concepts** — TLB, MMU duties, demand paging, thrashing — *3 Q* (2 MSQ) — 2021 S1 · 2022 · 2024 S2
- **Effective memory-access time** with TLB / page-fault service time — *2 Q* — 2018 · 2020
- **TLB reach / TLB calculation** — 1 Q — 2026 S2

◎ CPU scheduling (T1, 1.50 marks)

- **Waiting/turnaround/response time or Gantt chart** (FCFS, SJF, SRTF, RR, priority, **multi-CPU**) — *8 Q* — 2019 · 2020 · 2021 S1 · 2022 · 2024 S2 · 2025 S1 · 2025 S2 · 2026 S1
- **Scheduling concepts** — process states, starvation, preemptive vs non-preemptive (MCQ/MSQ) — *5 Q* (3 MSQ) — 2020 · 2021 S2 · 2023 · 2024 S1 · 2026 S2

◎ Disk scheduling, file systems & I/O (T2, 1.44 marks)

- **Disk capacity / access time** (seek + rotation + transfer) and **scheduling order** (SSTF/SCAN/C-SCAN) — *4 Q* (3 NAT) — 2018 · 2020 · 2024 S1 · 2024 S2

- **File-system structure** — inode direct/indirect pointers & max file size, linked vs contiguous allocation, directories — *4 Q* — 2019 · 2021 S1 · 2022 · 2025 S1
- **I/O handling** — device-controller / interrupt event sequence — *2 Q* — 2018 · 2025 S1

> ◎ **Process synchronization (T1, 1.22 marks)**

- **Semaphore pseudocode** — possible outputs, final value of a shared variable, deadlock / mutual-exclusion check — *7 Q* (3 MSQ, 3 MCQ, 1 NAT) — 2019 · 2020 · 2021 S1 · 2022 · 2023 · 2024 S2 · 2026 S2
- **Classical problems** (producer–consumer…) — correctness of a proposed solution — 1 Q — 2018

> ◎ **Deadlock (T2, 1.0 marks)**

- **Banker's / safe state / minimum resources to avoid deadlock / RAG detection** — *6 Q* — 2018 · 2019 · 2021 S2 · 2025 S2 · 2026 S1
- **Statements about deadlock conditions, prevention & avoidance** (MSQ) — 2 Q — 2022 · 2026 S1

> ◎ **Processes, threads, fork & system calls (T3, 0.94 marks)**

- **System calls, context-switch contents, mode transitions, thread properties** — *5 Q, all 1-mark MSQ* — 2021 S1 · 2023 · 2024 S1 · 2024 S2
- **fork() / wait()** — count processes or printed lines (NAT) — 3 Q — 2019 · 2024 S1 · 2026 S1
- **Threads & shared variables** — race conditions / possible outcomes — 2 Q — 2021 S2 · 2024 S1

> ◎ **Memory management (T4)** — Best-fit / allocation — 2020 (MCQ), 2026 S1 (2M NAT)

What this means for how you study

Priority	Topics	Rule of thumb
Must-do	Virtual memory & paging; CPU scheduling; Process synchronization	Do *every* example. Be able to do these without looking.
Next	Disk scheduling / file systems / I/O; Deadlock	Learn the 3–4 formulas + solve the standard examples
If time permits	Processes, threads, fork, system calls	Cheap marks — fork counting and MSQ statements are quick to learn
Last / skim	Memory management (allocation)	One page of notes is enough (Chapter 7 is deliberately short)

Where I trimmed: historical OS generations, batch/time-sharing definitions, long OS-structure essays, microkernel vs monolithic debates. They gave ≈ 0 marks in 9 recent GATEs. **Where I added weight:** paging geometry (7 Qs!), page-replacement tables, scheduling Gantt charts (incl. multi-CPU), semaphore puzzles, disk scheduling + access time, file-system pointer arithmetic.

0.3 Suggested study plan

Day	Chapters	Goal
1	Ch 1 + 2 (OS basics, processes, fork)	Fork counting becomes mechanical
2	Ch 3 + Ch 4 first half (threads; FCFS/SJF/SRTF)	Gantt-chart method
3	Ch 4 second half (RR, priority, multi-CPU, I/O) + practice	Zero errors on scheduling tables
4	Ch 5 (synchronization)	Semaphore puzzles + CS-solution checking
5	Ch 6 (deadlock)	Banker's + min-resources formula
6	Ch 7 + Ch 8 (memory, paging)	Page-table geometry
7	Ch 8 practice + Ch 9 first half	TLB/EAT, multilevel paging
8	Ch 9 second half	Page replacement, thrashing
9	Ch 10 + 11 (files, disk, I/O)	inode formula, disk scheduling
10	Ch 12 (revision sheet) + PYQs	Solve past papers by cluster

If you only have 3 days: Ch 8 $\rightarrow$ 9 $\rightarrow$ 4 $\rightarrow$ 5 ($\approx$ 60% of OS marks), then Ch 6, 10, 11 formulas only.

0.4 Notation used everywhere

Scheduling

- **AT** = arrival time **BT** = burst time (CPU time needed) **CT** = completion time
- **TAT** = turnaround time = CT $-$ AT
- **WT** = waiting time = TAT $-$ BT (time spent in the ready queue)
- **RT** = response time = (time of first CPU allocation) $-$ AT

Memory sizes (memorise this table — paging questions run on it)

Power	Value	Name
2^{10}	1,024	1 K
2^{20}	1,048,576	1 M
2^{30}	$\approx 10^9$	1 G
2^{40}	$\approx 10^{12}$	1 T

- 1 KB = 2^{10} B, 4 KB = 2^{12}, 64 KB = 2^{16}, 1 MB = 2^{20}, 4 MB = 2^{22}, 1 GB = 2^{30}, 4 GB = 2^{32}, 1 TB = 2^{40}.
- **Bits needed for N distinct things =** $\log_2$ **N.** (e.g., 512 entries $\rightarrow$ 9 bits.)
- **Address bits of a memory =** $\log_2$**(number of addressable units).** If memory is *word-addressable* with 2-byte words, a 4 GB memory has $2^{32}/2 = 2^{31}$ words $\rightarrow$ **31-bit** addresses (NOTES notes use exactly this).

> ϟ **Conversion habit:** always write sizes as powers of 2 first, then add/subtract exponents. Never multiply big decimals in paging questions.

PART I

Foundations

CHAPTER 1

OS Basics: Modes, System Calls, Interrupts, Boot

GATE IMPORTANCE ★☆☆☆☆

◎ **GATE pattern (from your analysis):** 1-mark **MSQ** statements on *system calls, mode transitions, context-switch contents* — asked in 2021 S1, 2023, 2024 S1, 2024 S2. They are cheap marks *if you know ~10 facts*. This chapter gives exactly those facts and nothing more.

1.1 What is an Operating System? (60-second version)

A computer, at the most basic level, is just three things:



the three things every OS manages

Figure 1.1. The three resources an operating system manages: CPU, memory and I/O.

With these you can run any program. But someone must decide:

1. **Who gets the CPU next?**
2. **Which part of memory should my program live in?**
3. **Which program may touch which memory location, and when?**
4. **How are files stored and found on the disk?**
5. ...and a lot more.

An **Operating System (OS)** is the software that answers those questions so that each programmer doesn't have to. It *manages the resources* — **CPU, memory, I/O devices, files**.

NOTES *The notes add a nice point:* the OS is not designed only with hardware in mind — **hardware is also designed with the OS in mind** (MMU, timer, interrupts, mode bit exist *because* OSes need them).

Mapping to this book: CPU → Ch 2–4 · Concurrency → Ch 5–6 · Memory → Ch 7–9 · Storage/I/O → Ch 10–11.

1.2 Dual-mode operation (user mode vs kernel mode)

The problem. The OS does *critical* work (scheduling, memory protection, talking to devices). A normal user program must **not** be able to disturb it — a buggy program must not be able to wipe the memory of the OS or another program.

The solution — a mode bit in the CPU.

Mode	Who runs	What it can do
User mode (mode bit = 1 in the notes)	ordinary programs	Read/write *its own* memory; run **non-privileged** instructions only
Kernel mode (a.k.a. supervisor / privileged / monitor; mode bit = 0 in the notes)	the OS kernel	Run **all** instructions — privileged + non-privileged; scheduling, I/O, memory management

NOTES The check "am I in user or kernel mode?" is done **in hardware on every instruction**, so it is very fast. The CPU may support two, three, or more modes depending on the OS it wants to support.

Privileged instructions (only legal in kernel mode). If a user program tries one, the CPU raises an exception and the OS typically kills the program:

- All **I/O instructions** (read/write device ports)
- **Setting the timer**
- **Disabling / enabling interrupts**
- **Changing memory-management registers** (base/limit, page-table base register) and **clearing memory**
- **Changing the mode bit** back to user ("return from interrupt")
- **Halt**

Non-privileged (allowed in user mode): arithmetic/logic, load/store *within your own address space*, branches, **reading the clock**, and — importantly — the **trap/system-call instruction** itself (a user program *must* be able to ask the OS for help).

> ⚠ **Trap 1:** *"Switching from user mode to kernel mode is done by a privileged instruction."* → **False.** It happens *automatically* (hardware) when a **trap / interrupt / exception** occurs. Only the switch **kernel → user** is done by a privileged instruction (return-from-interrupt / `iret`-style).

Three protections built on this: *I/O protection* (I/O instructions privileged), *memory protection* (base-limit / MMU), *CPU protection* (a timer interrupt guarantees the OS gets the CPU back).

1.3 System calls

> **System call = the set of APIs the kernel offers to user programs to use kernel services** (create a process, read a file, send data…). NOTES

How a system call actually executes (step by step):

1. User program calls a library wrapper (e.g., `read()`).
2. The wrapper puts the **system-call number** (and arguments) in registers/stack and executes a **software-interrupt / trap instruction** (e.g., `TRAP 80`).
3. The CPU **automatically switches to kernel mode** (mode bit → 0) and jumps to the OS's **trap handler / ISR** through the interrupt-vector table.
4. Kernel looks up the system-call number in the **system-call table**, runs the service routine.
5. A **privileged "return-from-trap" instruction** switches the mode bit back to user mode and the user program continues.

In kernel mode, *all* instructions (privileged and non-privileged) can run. NOTES

Categories of system calls NOTES:

Category	Examples
Process control	load, execute, **fork**, exit, wait
File management	open, read, write, seek, close
Communication	pipes, FIFOs, sockets, shared memory
Device management	read, write (device), ioctl
Information maintenance	get time, get PID, set attributes

⚠ **Trap 2 — library function ≠ system call.** `printf`, `malloc`, `strlen`, `sqrt` are **library functions** (run in user mode; `printf` internally ends up calling the `write` system call; `malloc` internally uses `brk/mmap`). `fork`, `exec`, `read`, `write`, `open`, `wait`, `exit` are system calls.

⚠ **Trap 3 — mode switch ≠ context switch.** A system call causes a **mode switch** (user → kernel → user). It does **not** by itself change which process owns the CPU. A **context switch** changes the running *process*. A system call may *lead to* a context switch (e.g., a blocking `read`), but it doesn't have to.

1.4 Interrupts, traps and exceptions

Kind	Caused by	Synchronous?	Examples
Interrupt (hardware)	an external device	**Asynchronous** (can come between any two instructions)	timer tick, disk I/O completed, keyboard
Trap / software interrupt	the program *on purpose*	Synchronous	system call (`TRAP 80`), breakpoint
Exception / fault	the program's instruction misbehaves	Synchronous	divide-by-zero, illegal/privileged instruction in user mode, **page fault**, segmentation fault

What the CPU does on **any** of these: save the PC (and flags), switch to kernel mode, jump to the handler (ISR) via the **interrupt-vector table**. When the ISR finishes it executes a return-from-interrupt, restoring PC and mode.

- A **page fault** is a *fault*: after the OS fixes it the **same instruction is restarted**.
- **Interrupts can be masked** (disabled) – except non-maskable ones. Disabling interrupts is privileged.
- **Timer interrupt** = how the OS pre-empts a process (basis of Round Robin, Ch 4).

NOTES **Cases when the PC value changes** (a favourite one-liner in the notes):

#	Case	Critical?
1	**Context switch** (start a new program, stop the previous)	**Critical**
2	**Function call**	Non-critical
3	**ISR runs** (interrupt/trap)	**Critical**
4	**Normal execution** (PC + 1)	–

(Branches/jumps also change the PC, of course.)

1.5 System boot (how the OS gets started) — low yield, 5 lines

NOTES When power comes on the **RAM contents are unreliable**, so the CPU starts executing a fixed program stored in **ROM** — the **bootstrap program**.

1. Bootstrap (ROM) diagnoses the machine.
2. It reads a fixed location on disk — the **boot block** (e.g., block 0).
3. The boot-block code (or a more sophisticated bootstrap such as **GRUB**) **loads the whole OS kernel into RAM**.
4. It sets the **PC to the kernel's start** — the OS is now running.

(Very small OSes may live entirely in ROM; large ones live on disk and are loaded into RAM.)

1.6 Quick-revision capsule

- Kernel mode can run **all** instructions; user mode only non-privileged.
- user → kernel: **automatic** (trap/interrupt/exception). kernel → user: **privileged instruction**.
- System call = software interrupt (trap) → mode bit changes → ISR → return.
- Library function ≠ system call. Mode switch ≠ context switch.
- Interrupt = async/hardware; trap/exception = sync/program.
- Boot: ROM bootstrap → boot block → (GRUB) → kernel in RAM, PC = kernel.

1.7 MSQ drill (with reasons)

Mark **True/False**:

1. *A system call always causes a context switch.* — **False.** Only if the process blocks/yields or the scheduler decides so.
2. *Reading the system clock must be a privileged instruction.* — **False.** Reading is harmless; *setting* the timer is privileged.
3. *A trap instruction can be executed in user mode.* — **True.** That is the only way user code enters the kernel.
4. *When an interrupt occurs, the hardware saves at least the PC before the ISR runs.* — **True.**
5. *Divide-by-zero in a user program is handled by the OS through an exception.* — **True.**
6. *The process page table base register may be changed by a user process.* — **False** (privileged).
7. *`malloc()` is a system call.* — **False** (library function; may use `brk`/`mmap`).
8. *Mode switch from kernel to user needs a privileged instruction.* — **True.**
9. *A context switch can only occur in kernel mode.* — **True** (the scheduler/dispatcher is OS code).
10. *Device interrupts are synchronous with the executing program.* — **False** (asynchronous).

CHAPTER 2

Processes, PCB, Context Switch, fork / exec / wait

GATE IMPORTANCE ★★☆☆☆

◎ **GATE pattern:** *fork() / wait() — count processes or printed lines* (3 NAT: 2019, 2024 S1, 2026 S1); *context-switch contents, system calls* (MSQ). Fork questions are **mechanical** once you know the doubling rule — they are the cheapest marks in this subject.

2.1 Program vs Process

- A **program** is a passive file on disk (compiled bytecode/machine code).
- A **process** is a **program in execution** — it *contests for resources*: CPU, memory, I/O. NOTES

How a program becomes a process NOTES:

1. You compiled the program and it sits on disk.
2. You ask the OS to run it (a system call).
3. The OS runs the **loader**, which copies the code from disk into some **memory location**.
4. The OS **schedules** it for execution.

Context of a process NOTES = everything that makes "this process running right now" different from another: CPU register values, PC, the MMU info (where it lives in memory), cache/TLB contents, open files, whether it's waiting for I/O...

Every process has a unique **Process ID (PID)**.

2.2 A process in memory

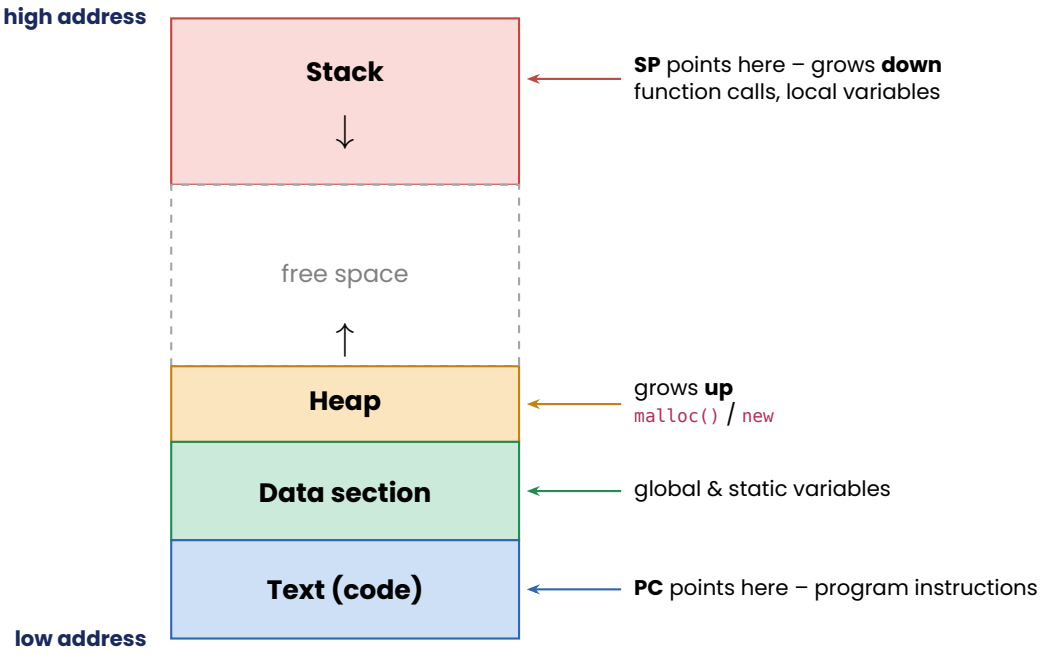


Figure 2.1. Layout of a process in memory (high addresses at the top).

NOTES When the OS loads a process it allocates a bit *more* memory than currently needed so the process doesn't have to ask for memory every time.

2.3 Process Control Block (PCB)

The OS must remember each process's context. It does this with a **PCB** (a.k.a. task control block), **one per process**, stored in RAM (the OS usually keeps them in a linked list / process table). NOTES It contains:

1. Process ID
2. **Program counter**
3. **Open files** list
4. **Register values**
5. **Process state**
6. **Memory limits / page-table info** (base-limit or page-table pointer)
7. Accounting, priority, scheduling info, parent/child pointers, I/O status ...

> The **page table of a process is also kept with its PCB** (Ch 8).

2.4 Process states and the schedulers

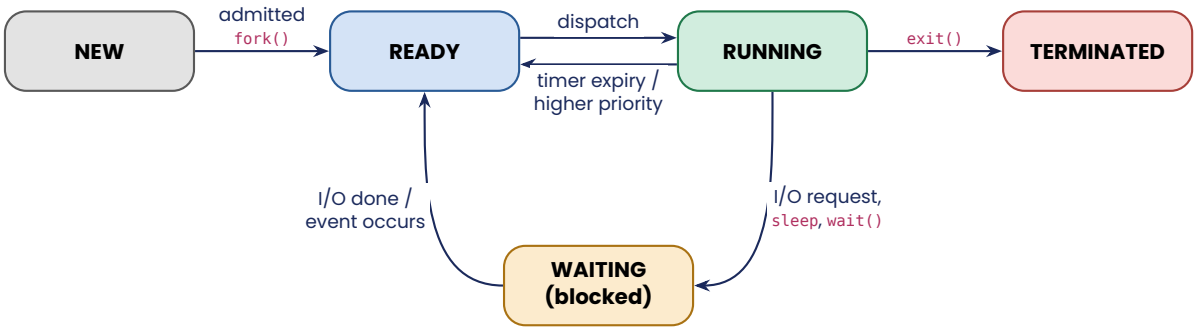


Figure 2.2. The five-state process model and the events that move a process between states.

NOTES *"New is just the initial state where no process exists. It exists only when the fork() call succeeds."*

Transition	Cause
new → ready	`fork()` succeeds (PCB created, added to ready queue)
ready → running	**dispatch** — the scheduler picks it (context switch)
running → ready	**preemption**: timer interrupt (Round Robin), higher-priority process arrives
running → waiting	process asks for I/O, `sleep`, `wait`, or a busy lock/semaphore
waiting → ready	the awaited event/I/O completes
running → terminated	`exit()` or killed

⚠ **Impossible transitions** (classic MCQ bait): *waiting* → *running* (a blocked process must first become ready), *ready* → *waiting* (a process that isn't running can't start I/O).

Queues kept by the OS NOTES: **Job queue** (all submitted tasks), **Ready queue** (in memory, ready for CPU), **Waiting/device queues**.

Three schedulers NOTES

Scheduler	Chooses...	Runs	Controls
Long-term (job)	which submitted *tasks* are brought into memory	rarely	**Degree of multiprogramming** = number of processes in memory
Short-term (CPU)	which ready process runs next on the CPU	very often (ms)	CPU allocation
Mid-term	moves a process out of memory to disk (**swap-out**) and later back (**swap-in**)	occasionally	reduces memory load / thrashing

NOTES A program in "task form" (job queue) doesn't occupy any CPU resource until the long-term scheduler admits it.

Dispatcher NOTES — the module that gives the CPU to the chosen process. It does 3 things: (**1**) context switch, (**2**) switch to user mode, (**3**) jump to the right location in the program. The time it takes = **dispatch latency**.

2.5 Context switch

NOTES Suppose the CPU is running P_2, and P_2 needs a file from disk. If the CPU just waits, CPU time is wasted. So replace P_2 with P_{10} for now.

> **Context switch = saving the state (registers, PC, cache/TLB info, open files, memory-management info) of the currently running process into *its PCB*, and loading the state of another process from *its PCB*.**

Facts for MSQs:

- A context switch is **pure overhead** — no useful user work is done during it. It is **slow**. NOTES
- It happens in **kernel mode** (the scheduler/dispatcher is OS code).
- It happens when: the running process blocks (I/O), terminates, its time quantum ends, a higher-priority process becomes ready, or an interrupt makes the OS reschedule.
- **What is saved/restored:** PC, general registers, stack pointer, status flags, **memory-management registers (e.g., page-table base)**, I/O status, accounting. **Not** copied: the process's code/data/heap — they stay in memory.
- If the TLB has no ASID/PID tag, the **TLB must be flushed** on a process switch (Ch 8).
- *Switching between two **threads of the same process** is cheaper* than between two processes (no address-space change).
- ⚠ *A process switch always needs a mode switch*, but a mode switch doesn't need a process switch.

2.6 `fork()`, `exec()`, `wait()` — the heart of this chapter

fork

NOTES *"When fork is executed by a user program the call creates a new **child** process which is a **replica** of the original program."*

- **Return value:** **0** to the child; the **child's PID (positive)** to the parent; **negative** if creation failed (e.g., not enough memory).
- **Both parent and child continue from the instruction just after `fork()`** (they have the same PC value).
- "Exact replica" means everything the parent had run up to that point *has already happened* in the child's world (variables hold the same values); from here on they may diverge.
- Fork normally uses **copy-on-write** NOTES: a new PCB is created, but the memory pages are **not** copied until either process *writes*; then that page is duplicated.
- NOTES **There is no specific order** in which parent and child run. (Hence "possible outputs" questions.)
- Each process has its **own copy of variables** after the fork: changing `x` in the child does not change `x` in the parent.

```
int main() {
    int c_id = fork();        // L1
    printf("%d", c_id);       // L2
}
// Parent prints e.g. 2537 (child's PID);  Child prints 0.
```

exec

- `exec` **replaces the process's memory** (text + data; fresh stack and heap) with a new program. The **PID and PCB stay the same → open files stay open** (stdin/stdout/stderr unchanged). NOTES
- **On success, exec never returns** to the old program; the statements after `exec` run only if exec **failed**.

The loader's recipe NOTES

> To run a new program the OS does **fork (create process) + exec (load the program into it)**.

How the shell runs `ls` NOTES

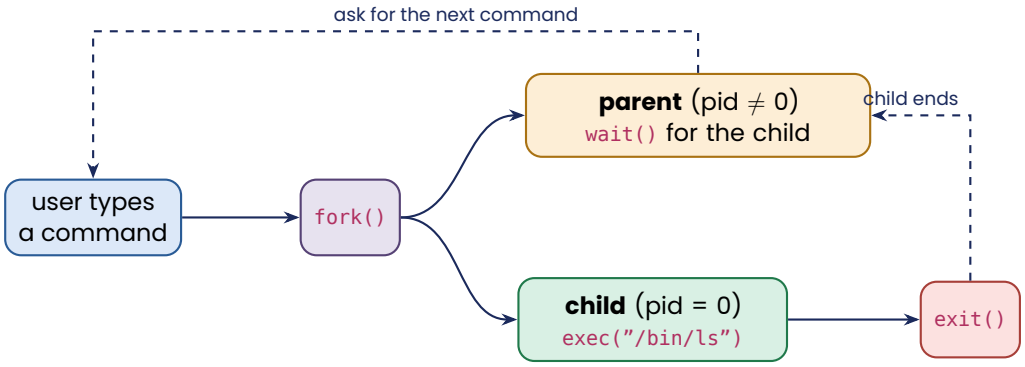


Figure 2.3. How a shell runs a command: fork creates the child, exec loads the program, wait lets the parent block.

1. Shell calls `fork`. 2. Parent calls `wait` (blocks). 3. Child calls `exec("/bin/ls")`, replacing its memory. Open files unchanged so output goes to the same terminal. 4. `ls` runs and terminates. 5. Parent unblocks and prompts again.

wait

NOTES `wait()` **blocks the caller until one of its children terminates**, returns that child's PID. **If the caller has no children, wait has no effect** (returns -1 immediately). To reap k children you need k `wait()` calls.

Zombie and orphan NOTES

	Definition	Why it matters
Zombie	Child has **terminated**, but parent has **not called** `wait`. All resources are freed *except its PCB/entry in the process table*	Wastes a process-table slot until parent waits (or parent dies and `init` reaps it)
Orphan	**Parent terminated** while the child is still running	Child is adopted by `init` (PID 1)

2.7 How to count processes (the mechanical method)

Rule 1 — the doubling rule

> **Every *unconditional* `fork()` executed by k processes produces k new processes → the count doubles.**

- n unconditional forks in a straight line → 2^n **processes** ($= 2^n - 1$ *children*).
- `for (i=0; i<n; i++) fork();` → also 2^n. NOTES The notes' example `for(i=0;i<3;++i) fork(); printf("HW\n");` prints **8** times (2^3).

Rule 2 — conditions decide *which* processes fork

- `if (fork() == 0) { … }` — only the **child** runs the body.
- `if (fork()) { … }` (non-zero) — only the **parent** runs the body.
- Draw the tree; count the leaves that reach the print.

Rule 3 — short-circuit operators

- `fork() && fork()` : parent gets a **non-zero** (true) → evaluates the second `fork`; child gets **0** (false) → **skips** it. → **3 processes**.
- `fork() || fork()` : parent (true) **skips** the second; child (0) evaluates it. → **3 processes**.
- Precedence: `&&` binds tighter than `||`.

Rule 4 — number of printed lines

times a `printf` runs = # of processes that reach it (times how many times each reaches it, if inside a loop).

Worked examples (all verified by actually running the programs)

Code	Processes	Why
`fork(); fork(); fork();`	**8**	2^3
`for(i=0;i<4;i++) fork();`	**16**	2^4
`if(fork()==0) fork();`	**3**	P forks C; only C forks again
`fork(); if(fork()==0) fork();`	**6**	after first fork 2 procs; each yields 3
`if(fork()==0){ fork(); fork(); }`	**5**	P + C; C's two forks make 4
`if(fork()) { fork(); } fork();`	**6**	P, C_1, P_2 = 3, then final fork doubles
`fork() && fork();`	**3**	
`fork() || fork();`	**3**	
`for(i=0;i<3;i++){ if(fork()==0) break; }`	**4**	each child breaks out after being created: parent + 3 children
`for(i=0;i<3;i++){ if(fork()!=0) break; }`	**4**	a chain: only the child continues looping
`fork(); fork() && fork() || fork(); fork();`	**20**	see below

The "20" example, step by step. After the first `fork()`: 2 processes. Each of them evaluates `(fork() && fork()) || fork()`. Follow *one* process X:

- X calls fork#a → parent X (true) and child Y (false).
  - X (true) calls fork#b → X (true) and Z (false). For X: `true && true = true` → `||` short-circuits, **no third fork**. → 1 process.
  - Z (false from fork#b): `true && false = false` → must evaluate `|| fork#c` → splits in 2. → 2 processes.
  - Y (false from fork#a): `&&` short-circuits to false → evaluates `|| fork#c` → splits in 2. → 2 processes.
- So one process becomes 1 + 2 + 2 = **5**. Two processes → **10**. The last `fork()` doubles → **20**.

NOTES | Lecture Example A — printing "A", fork, "B", wait, "C"

```c
int main() {
    printf("A");
    int ch = fork();
    printf("B");
    if (ch) wait(NULL);     // only the parent waits
    printf("C");
}
```

Possible outputs (notes): **ABBCC** and **ABCBC**.
Reasoning: "A" is printed once, before the fork. After the fork, parent P and child C each print **B**. The child never waits, so it prints **C** right after its B. The parent prints B, **waits until the child terminates**, then prints C. So the parent's C always comes **after the child's C**. If the child runs first it can print B, C before the parent has printed its own B → **A B C B C**. If the parent prints B first → **A B B C C**.

NOTES | Lecture Example B — counting forks *and* checking an output sequence

```c
int main() {
    int c = 0;
    for (int i = 0; i < 3; i++) {
        if (fork() == 0) {
            printf("%d ", c + i);
            fflush(stdout);
        } else {
            c = c + i + 3;
        }
    }
}
```

(a) How many times is `fork` called? Round i=0: 1 process forks; i=1: 2 processes; i=2: 4 processes $\rightarrow 1 + 2 + 4 = \mathbf{7}$ calls.
(b) Which of these is *not* a valid output? (1) 0 1 4 6 5 9 2 (2) 4 0 1 6 9 2 5 (3) 9 4 0 5 1 2 6 (4) 0 1 5 4 9 6 2
Method — build the "who prints what" table. Name processes A (original), B, C, ... in creation order. Remember: *only the child prints*, right after its own fork, and the child keeps the **old** c.

Process	Created by	At loop i	c at that moment (parent's c before update)	Prints c+i
B	A	0	0	**0**
C	A	1	3 (A: 0+0+3)	**4**
E	A	2	7 (3+1+3)	**9**
D	B	1	0	**1**
F	B	2	4 (B: 0+1+3)	**6**
G	C	2	3	**5**
H	D	2	0	**2**

So 7 values are printed: {0, 1, 2, 4, 5, 6, 9}.
Ordering constraints (a child can print only *after* its parent has forked it, and a process's own later forks come after its own print):

- B's print (0) must come before everything B's descendants print: D's (1), F's (6) $\rightarrow$ **0 < 1, 0 < 6**.
- D's print (1) before its child H's (2) $\rightarrow$ **1 < 2**.
- C's print (4) before its child G's (5) $\rightarrow$ **4 < 5**.
- 9 has no constraint.

Test options: (1) 0 1 4 6 5 9 2 ✔ (2) 4 0 1 6 9 2 5 ✔ (3) 9 4 0 5 1 2 6 ✔ (4) 0 1 **5 4** ... ✘ (5 before 4).
Answer: (4). ● matches the notes ("5 can't be printed before 4").

> ϟ **Technique for "valid output sequence" questions:** (1) label every process; (2) write the value each print outputs; (3) write only the *must-come-before* pairs (parent-fork $\rightarrow$ child-print $\rightarrow$ child's later actions); (4) check each option against those pairs only.

NOTES | Lecture Example C — `while` with fork

```c
int i = 0;
while (i < 2) { fork(); printf("%d", i); i++; }
```

Which sequences are possible? (A) 0101 (B) 001111 (C) 010111 (D) 011011 (E) 011101
Process picture: round 0: original **a** forks child **b**; both print **0**. Round 1: a forks c, b forks e; a, b, c, e each print **1**. Total prints = 2 zeros + 4 ones = **6 characters**. Constraints: a's 0 before a's 1 and c's 1; b's 0 before b's 1 and e's 1. Each zero can only "release" two ones.

- (A) 0101 has only 4 characters $\rightarrow$ ✘.

- (B) 001111 ✔. (C) 010111 ✔. (D) 011011 ✔ (a0, a1, c1, b0, b1, e1).
- (E) 011101: three 1s before the second 0 — but only two 1s (a's and c's) can appear before b prints its 0 → ✘. **Answer: (A) and (E) impossible.** (Checked by brute force.)

2.8 Quick-revision capsule

- Process = program in execution; PCB stores its context; one PCB per process.
- States: new → ready ⇄ running → terminated; running → waiting → ready.
- Context switch: save into old PCB, load from new PCB; pure overhead; kernel mode.
- fork returns **0 to child, child's PID to parent, −1 on error**; both resume after the fork.
- exec replaces memory but **keeps PID/PCB/open files**; doesn't return on success.
- wait blocks until *a* child ends. Zombie = terminated child not waited for. Orphan = parent died first.
- n unconditional forks → 2^n processes. `&&`/`||` short-circuit — the child (value 0) behaves differently from the parent.

2.9 ADDED Practice set (answers verified by running the programs)

Q1. How many processes (including the original) exist after `fork(); fork(); fork(); fork();`?

Q2. `main(){ fork(); fork(); printf("A"); fork(); printf("B"); }` How many A's and B's are printed in total?

Q3. `for (i=0;i<n;i++) fork();` creates how many *new* processes?

Q4. What is always printed by the following (no other possibility)?

```
main(){ if(fork()==0) printf("C"); else { wait(NULL); printf("P"); } printf("E"); }
```

Q5. In `main(){ int x=5; if(fork()==0){ x=x+10; printf("c%d ",x);} else { wait(NULL); printf("p%d ",x);} }` what is printed?

Q6.

```
main(){ if(fork()==0){ printf("A"); if(fork()==0) printf("B"); } else printf("C"); }
```

How many different output strings are possible?

Q7.

```
int pid = fork();
if (pid == 0) { printf("1"); fork(); printf("2"); }
else { wait(NULL); printf("3"); }
printf("4");
```

How many characters are printed in total?

Q8. After `fork()` in a process whose parent never calls `wait()`, the child exits. What is the child called? What if the parent exits first?

Q9. After a successful `execl("/bin/ls","ls",NULL);` does the next statement `printf("done")` execute?

Q10. Which of the following are **true**? (a) Parent and child share the same copy of the variable `x` after fork. (b) `exec` creates a new process. (c) `fork` returns 0 in the child. (d) The PID changes after `exec`.

Solutions

- **Q1:** $2^4 = \mathbf{16}$.
- **Q2:** After 2 forks: 4 processes $\rightarrow$ **4 A's**. Then each forks $\rightarrow$ 8 processes $\rightarrow$ **8 B's**.
- **Q3:** $2^n - 1$.
- **Q4: CEPE** — parent waits for the child, so child's "C","E" come first; then parent's "P","E".
- **Q5:** Output is **c15 p5** — variables are *separate copies*; the child's change doesn't affect the parent.
- **Q6:** Process P prints C. Child C_1 prints A, then forks C_2 which prints B. Only constraint: A before B. Orders of {A,B,C} with A before B: ABC, ACB, **CAB** $\rightarrow$ **3**.
- **Q7:** Child prints 1, forks (grandchild); both print 2, then both print 4 $\rightarrow$ "1","2","4" (child) + "2","4" (grandchild) = 5 chars. Parent waits for the *child* (its only child), prints 3, 4 $\rightarrow$ 2 chars. Total = **7**. (The grandchild's output can interleave anywhere after the fork.)
- **Q8: Zombie**. If parent exits first the child becomes an **orphan** (adopted by init).
- **Q9: No** — exec replaces the program; it returns only on failure.
- **Q10:** (c) is the only true one. (a) false – separate copies (copy-on-write). (b) false – exec doesn't create a process. (d) false – PID unchanged.

CHAPTER 3

Threads

GATE IMPORTANCE ★★☆☆☆

◎ **GATE pattern:** *thread properties* (MSQ 1-markers: 2021 S1, 2023, 2024 S1, 2024 S2) and *threads & shared variables — race conditions / possible outcomes* (2021 S2, 2024 S1). Learn the comparison table in §3.5 and the shared-counter result in §3.6.

3.1 The problem threads solve

Recall the process layout (Ch 2): stack, heap, data, text. The **address space** of a process NOTES is *the set of all addresses from start to end in memory that the process can access*.

Two CPUs can run in different ways NOTES:

1. **Two CPUs, two different processes** → different address spaces *and* different contexts.
2. **Two CPUs, the *same* address space but different contexts** (different register values / PCs) → what we want for, say, **multiple tabs in a browser**: same program, same data, but two independent flows of execution.

Observation NOTES: **the number of execution contexts does not depend on the number of CPUs.** Even on a single CPU you can interleave two contexts of the same process with Round Robin.

> **Thread = one execution context (own PC, registers, stack) inside a process, sharing the process's code, data, heap and open files with the other threads.**

NOTES The picture the notes draw: *two stacks in the same process memory.* "Stack 1 + Heap + Data + Text" forms one thread; "Stack 2 + Heap + Data + Text" forms another.

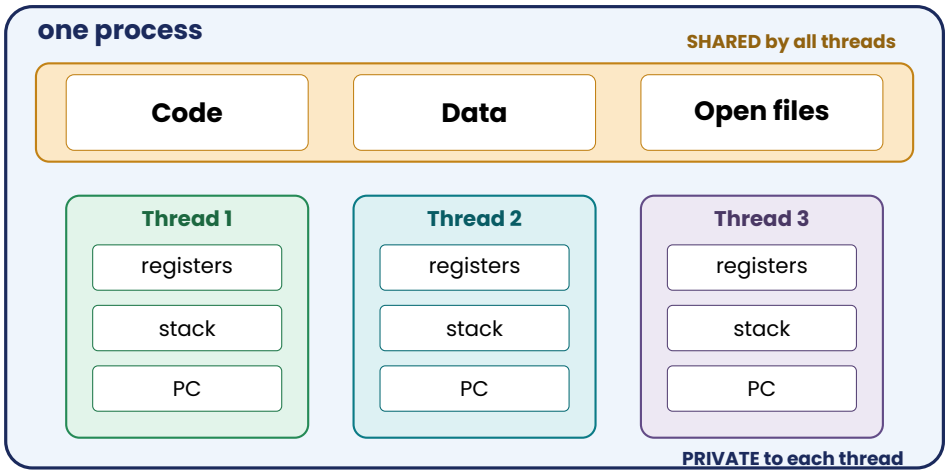


Figure 3.1. Threads of one process share code, data and open files; each has its own registers, stack and PC.

3.2 Parallelism vs concurrency NOTES

	Concurrency	Parallelism
CPUs	**one** CPU suffices	**more than one** CPU
Execution	contexts are all "in progress", but run **interleaved** (C1 C2 C1 C2...); never at the same instant	contexts **overlap in time** (really simultaneous)

Threads are about **both**: concurrent (interleaved on one CPU) and parallel (on many CPUs).

Typical use case NOTES **— an HTTP server:** every client request runs the *same code* with a *different context*. Creating a whole process per request is heavy; threads are light.

3.3 Why a process is "heavy" and a thread is "light"

- A **process** has its **own address space** and doesn't share OS resources. NOTES Two processes **cannot access each other's address space**; communication needs the OS (**pipes, FIFOs, sockets, shared memory, message queues**) → **expensive**.
- **Threads** live in the same address space → communicating is just reading/writing shared variables → **cheap**. NOTES

Benefits: responsiveness, resource sharing, economy (cheap creation and context switch), scalability on multiprocessors.

3.4 How are threads implemented? — User-level vs Kernel-level

User-level threads (ULT) NOTES

- A **library inside the process** creates and schedules threads. The **OS sees only one process** and schedules it.
- Managing threads needs **no OS intervention / no system calls** → **fast** creation and switching.
- ⊗ If **one thread makes a blocking call** (I/O, sleep), the OS blocks the **whole process** (it doesn't know about the other threads).
- ⊗ Cannot use **multiple CPUs** for the threads of one process (OS schedules the process on one CPU at a time).

Kernel-level threads (KLT) NOTES

- **The kernel manages the threads**; creating one needs a **system call**. A common design: **one PCB (kernel structure) per thread**, all pointing to the same address space, each with its own context.
- ✅ Kernel can schedule different threads of the same process on **different CPUs** (one of the reasons to use them — CPUs are owned by the kernel).
- ✅ If one thread blocks, the others can still run.
- ❌ Slower to create/switch than ULT (system call / mode switch).

Multilevel / hybrid threading NOTES

Many user threads are **mapped onto** some kernel threads (many-to-many):

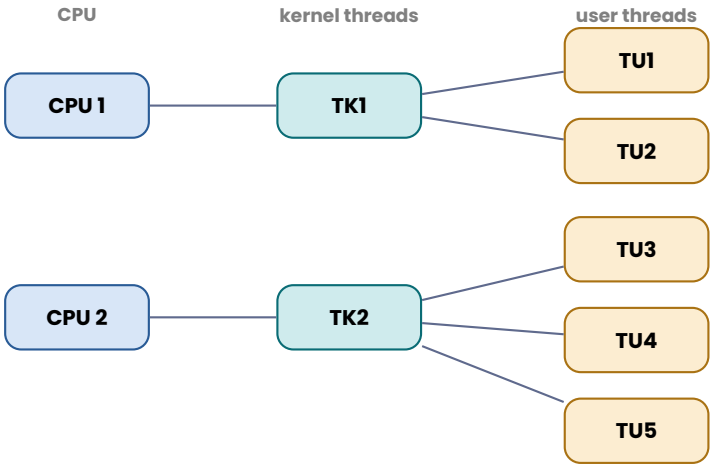


Figure 3.2. Hybrid (many-to-many) threading: user threads are multiplexed onto kernel threads.

- Fast switching among user threads mapped to the same kernel thread (no OS involvement).
- Multi-CPU execution via the kernel threads.
- If one kernel thread blocks, the other kernel thread can still run — even on a single CPU.

Mapping models (names GATE may use): **many-to-one** (= pure user-level), **one-to-one** (= pure kernel-level), **many-to-many** (hybrid).

3.5 ⚡ Thread vs Process — the comparison table to memorise

Property	Per thread (private)	Shared among threads of a process
Program counter	✔	
CPU registers	✔	
Stack	✔	
Thread ID / state	✔	
Code (text)		✔
Global/static data		✔
Heap		✔
Open files, signal handlers (mostly)		✔
Address space		✔ (same)

	Process	Thread
Creation / termination / context-switch cost	High	Low
Communication	via OS (IPC)	via shared memory (cheap, but needs synchronization)
Isolation	Strong (crash of one doesn't corrupt the other)	Weak (a bug in one thread can corrupt the whole process)
Switching same-process threads: TLB flush / address-space change?	Needed for process switch	**Not** needed

⚠ **Trap:** *"A user-level thread switch requires a system call."* → **False.** *"All threads of a process share the stack."* → **False.** *"On a single-CPU system, kernel-level threads can run in parallel."* → **False** (concurrently yes, parallel no).

3.6 Threads + shared variables = race conditions (preview of Chapter 5)

Because threads **share memory**, unsynchronized updates collide. NOTES The notes' first example:

```
int count = 0;
void foo() { count++; }
int main() {
    T1 = create_thread(foo);
    T2 = create_thread(foo);
    wait(T1, T2);
    printf(count);
}
```

`count++` is not one step. It compiles to three machine instructions NOTES:

	Instruction
I_1	`LOAD R1, [2000]` (read count)
I_2	`INCR R1`
I_3	`STORE [2000], R1` (write back)

If the scheduler interleaves **T1I1 → T2I1 → T1I2 → T2I2 → T1I3 → T2I3**, both threads read 0, both write 1 → **count = 1** instead of 2. NOTES Other schedules (T1 completes fully, then T2) give 2. The notes count **6!/(3!·3!) = 20 possible interleavings** of the six instructions (3 per thread) — that's the number of ways to merge two sequences of 3 steps.

The classic "min and max" result (frequently asked: *"possible final values of x"*)

Two threads each execute `x = x + 1` inside a loop `n` times, x starts at 0, no synchronization.

n (per thread)	Possible final values (brute-force checked)
1	{1, 2}
2	{2, 3, 4}
3	{2, 3, 4, 5, 6}
4	{2, 3, ..., 8}

Rule: for **n ≥ 2**: **minimum = 2, maximum = 2n**, and every integer in between is possible. For n = 1: min = 1.

Why can the minimum be 2 and not n? Thread 1 loads 0 and is stalled; thread 2 runs n−1 full iterations (x = n−1); thread 1 stores 1; thread 2 loads 1, then thread 1 runs all its remaining iterations up to x = n

(overwritten later); thread 2 stores 2 and finishes. Final value 2.

> ⚠ **Trap:** "min = n" is the usual wrong guess. With 3 threads and n $\geq$ 2 the minimum is also 2 (checked).

The cure is *synchronization* (locks / semaphores) — Chapter 5.

3.7 Quick-revision capsule

- Thread = execution context sharing code, data, heap, files; private PC, registers, stack.
- Concurrency = interleaving (1 CPU ok); parallelism = simultaneous ($\geq$ 2 CPUs).
- ULT: library-managed, fast, **one block blocks all**, no multi-CPU. KLT: kernel-managed, slower, true parallelism, block affects only one thread.
- Thread switch inside a process is cheaper than a process switch.
- Unsynchronized `count++` from two threads: 1 or 2. For n $\geq$ 2 increments each: min 2, max 2n.

3.8 ADDED Practice set

Q1. Which are shared by all threads of a process? (a) global variables (b) stack (c) heap (d) program counter (e) open file table (f) register values.

Q2. A process has 3 user-level threads on a single-CPU system. Thread A calls a blocking `read()`. What happens to threads B and C?

Q3. Two threads each run `for(i=0;i<5;i++) x = x+1;` on shared x = 0 without locking. What are the minimum and maximum final values of x?

Q4. In how many ways can the 3-instruction sequences of two threads be interleaved?

Q5. *True/False:* (a) Kernel-level threads of one process can run on different CPUs simultaneously. (b) A user-level thread library needs a system call for every thread switch. (c) Context switch between threads of the same process doesn't need a change of address space.

Solutions

- **Q1:** (a), (c), (e).
- **Q2:** With ULT the OS sees one blocked process $\rightarrow$ **B and C also block** (that is the major drawback of ULT).
- **Q3: min = 2, max = 10** (n=5 $\geq$ 2).
- **Q4:** C(6,3) = **20**.
- **Q5:** (a) True, (b) False, (c) True.

PART II

CPU Scheduling

CHAPTER 4

CPU Scheduling

GATE IMPORTANCE ★★★★☆ *≈ 1.5 marks/paper, asked almost every year*

◎ **GATE pattern (from your analysis):** a *numerical* 2-mark question nearly every year — "draw the Gantt chart, find average WT / TAT / number of context switches / CPU idle %" for **FCFS, SJF, SRTF, RR, priority**, sometimes with **I/O bursts** or **multiple CPUs**. Add the occasional 1-mark conceptual statement (starvation, convoy effect, which algorithm is optimal). **Scheduling is a pure-practice topic: after this chapter, do every Gantt chart by hand until it takes < 2 minutes.**

4.1 Why do we need a scheduler?

Many processes are *ready* (Ch 2) but there is **one CPU**. Somebody must decide **who runs next** — that is the **CPU scheduler** (the *short-term scheduler*). The **dispatcher** is the piece that actually does the job after the decision: context switch → switch to user mode → jump to the right place in the chosen process. NOTES

Recap of the three schedulers NOTES:

Scheduler	Moves processes ...	Frequency	Controls
Long-term (job scheduler)	job pool → memory (ready queue)	rare	**degree of multiprogramming** (= number of processes in memory)
Short-term (CPU scheduler)	ready queue → CPU	every few ms	who runs
Medium-term	swap a process out of / back into memory	occasional	memory pressure

When is the scheduler invoked? (the four decision points NOTES)

1. Running → **Waiting** (I/O request, `sleep`, `wait`)
2. Running → **Ready** (timer interrupt, or a higher-priority process arrives)
3. Waiting → **Ready** (I/O finished)
4. Running → **Terminated**

- If scheduling happens **only at 1 and 4** → **non-preemptive** (process keeps the CPU until it blocks or ends).
- If it can also happen at **2 and 3** → **preemptive** (the OS *forcibly* takes the CPU away: by a **timer interrupt** (time slice) or a **system call** that makes a higher-priority process ready). NOTES

All these events are system calls or hardware interrupts → the mode bit becomes 0 → the scheduler code (in the kernel) runs.

4.2 The terms (learn these 7 and the formulas — everything else follows)

Term	Meaning	Formula
AT (Arrival Time)	when the process enters the ready queue	given
BT (Burst Time)	CPU time it needs	given
CT (Completion Time)	when it finishes	read from Gantt chart
TAT (Turn-Around Time)	submit → finish	**TAT = CT − AT**
WT (Waiting Time)	total time spent **waiting in the ready queue**	**WT = TAT − BT**
RT (Response Time)	arrival → **first** time it gets the CPU	**RT = (first CPU time) − AT**
Throughput	processes completed per unit time	n / total time
CPU utilisation	% of time CPU is busy	busy time / total time × 100

Relations worth remembering

- **Non-preemptive algorithm ⇒ RT = WT** (a process gets the CPU once and keeps it).
- **Total WT = Σ CT − Σ AT − Σ BT.** So you can get the *average* WT from the completion times alone — no need to compute each WT.
- CPU busy time = **Σ BT** (when no I/O). Utilisation = Σ BT / (finish time of the last process − first start) if there are idle gaps.
- If all AT = 0: **TAT = CT**.

4.3 The universal recipe to solve any scheduling numerical

1. **Sort processes by AT** and write them in a small table: `P | AT | BT`.
2. **Advance a clock `t`.** At every decision point, make the **ready set** = processes with AT ≤ t that are not finished.
3. Pick from the ready set by the algorithm's rule (below). **If the ready set is empty → CPU is idle: jump `t` to the next arrival** (draw "idle").
4. Write each run as a segment of the **Gantt chart** `[Pi : start–end]`.
5. For **preemptive** algorithms, re-decide **at every arrival** (SRTF/priority) or **at every quantum expiry** (RR) — and *only* at those instants.
6. Fill the table **CT → TAT → WT → RT** and average.
7. **Check:** Σ BT = total busy time in the Gantt chart; every CT ≥ AT + BT.

Algorithm	Choose from the ready set	Preemptive?
FCFS	smallest AT	no
SJF	smallest BT	no
SRTF	smallest **remaining** BT	yes
RR	front of a FIFO queue, run ≤ q	yes
LJF	largest BT	no
LRTF	largest **remaining** BT	yes
Priority	highest priority	either
HRRN ADDED	highest (W+BT)/BT	no

Tie-breaking (if the question does not say): smaller AT first, then smaller process number. (Details in Appendix B.)

4.4 FCFS — First Come First Served NOTES

Serve strictly in arrival order. **Non-preemptive.** Implemented with a FIFO queue. Simple, but suffers the **convoy effect**: one long process at the front makes all short ones wait, lowering CPU/I-O utilisation.

NOTES | Example 4.1 — FCFS

P	AT	BT
P1	3	24
P2	1	3
P3	10	6

GANTT CHART



(CPU is idle 0–1 — nobody has arrived).

P	CT	TAT = CT−AT	WT = TAT−BT	RT
P1	28	25	1	1
P2	4	3	0	0
P3	34	24	18	18

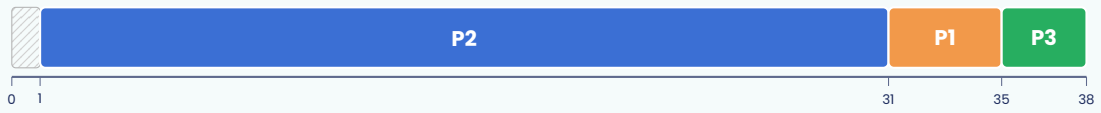
Avg WT = 19/3 = 6.33; avg TAT = 52/3 = 17.33. (Non-preemptive ⇒ RT = WT, as you see.)

NOTES | Example 4.2 — Convoy effect

P	AT	BT
P1	3	4
P2	1	30
P3	5	3

At t = 1 only P2 is present, so FCFS (non-preemptively) runs it for 30 units.

GANTT CHART



P	CT	TAT	WT	RT
P1	35	32	28	28
P2	31	30	0	0
P3	38	33	30	30

Average RT = (28+0+30)/3 = 19.33. The two tiny jobs wait behind the giant one — that *is* the convoy effect.

> ϟ **FCFS shortcut for AT = 0:** if processes run in the order b_1, b_2, ..., b_n then WT of the k-th = b_1+...+$b_{(k-1)}$ (sum of all earlier bursts).

4.5 SJF — Shortest Job First (non-preemptive) NOTES

Among the **ready** processes pick the **smallest burst time**. **Non-preemptive.** Gives the **minimum average waiting time among all non-preemptive algorithms** (proved by an exchange argument: putting a short job before a long job reduces total wait). Drawbacks: (i) **starvation** of long jobs (short ones keep coming), (ii) the burst time is **not known in advance** → must be *predicted*.

NOTES | Example 4.3 — SJF

P	AT	BT
P1	1	8
P2	2	6
P3	4	2
P4	10	3

Reasoning (how the notes do it):

- t = 0 → nobody → **idle 0–1**.
- t = 1 → only P1 present → **P1 1–9** (non-preemptive!).
- t = 9 → P2 (6) and P3 (2) are waiting; P4 not yet → pick P3 → **P3 9–11**.
- t = 11 → P2 (6) and P4 (3) → **P4 11–14**.
- t = 14 → only P2 → **P2 14–20**.

GANTT CHART



P	CT	TAT	WT	RT
P1	9	8	0	0
P2	20	18	12	12
P3	11	7	5	5
P4	14	4	1	1

Avg WT = 18/4 = 4.5, total time 20.

> ϟ **SJF shortcut when all AT = 0:** just **sort by BT**. WT of the k-th job = sum of the earlier (smaller) bursts.

4.5.1 Predicting the next CPU burst NOTES

The OS cannot know the next burst, so it **guesses from history**. Let `tₙ` = actual length of the n-th burst, `Sₙ` = predicted value.

1. **Simple average** (all past bursts weigh equally): `S(k+1) = (k/(k+1))·S(k) + (1/(k+1))·t(k)` — only the last prediction and last actual are needed.
2. **Exponential average** (recent history matters more), with `0 ≤ α ≤ 1`:

S(n+1) = α·t_n + (1−α)·S_n

- α = 0 → `S(n+1) = Sₙ` → recent behaviour is **ignored**.
- α = 1 → `S(n+1) = tₙ` → **only the last burst** counts.
- α = ½ is the usual choice.
- Expanded: S(n+1) = α·t_n + α(1−α)·t(n−1) + α(1−α)²·t(n−2) + … + (1−α)^(n+1)·S_0 — older terms fade geometrically (hence "exponential").

NOTES | Example 4.4 — Predictions

Previous actual bursts: **4, 7, 8, 16**; initial guess S_0 **= 10**; **α = 0.5**.

- **Simple average** = (4+7+8+16)/4 = 35/4 = **8.75**
- **Exponential**: S_1 = 0.5·4 + 0.5·10 = **7** S_2 = 0.5·7 + 0.5·7 = **7** S_3 = 0.5·8 + 0.5·7 = **7.5** S_4 = 0.5·16 + 0.5·7.5 = **11.75** ← the predicted next burst.

(Notice the jump 8 → 16 pulls the exponential prediction up quickly.)

4.6 SRTF — Shortest Remaining Time First (preemptive SJF) NOTES

Preemption = the OS **forcibly stops** the running process and moves it to the ready queue. SRTF: whenever a process **arrives**, compare its BT with the **remaining time** of the running process; if it is smaller, **preempt**. It gives the **provably minimum average waiting time overall** (but long jobs may **starve**).

NOTES | Example 4.5 — SRTF

P	AT	BT
P1	1	8
P2	2	5
P3	4	2
P4	10	3

Walk-through (decide at arrivals/completions only):

- t=1 P1 starts (rem 8). t=2 P2 arrives, BT 5 < P1's remaining 7 → **preempt**, P2 runs (rem 5).
- t=4 P3 arrives, BT 2 < P2's remaining 3 → **preempt**, P3 runs 4–6.
- t=6 P3 done. Ready: P1 (7 rem), P2 (3 rem) → P2 runs 6–9.
- t=9: ready P1 (rem 7). P1 runs 9–10. t=10 P4 arrives (3) < P1's remaining 6 → preempt; P4 10–13.
- t=13: P1 runs 13–19.

GANTT CHART



P	CT	TAT	WT	RT
P1	19	18	10	0
P2	9	7	2	0
P3	6	2	0	0
P4	13	3	0	0

Avg WT = 12/4 = 3.0 (compare with SJF's 4.5 on the *same* data — SRTF is better). **Number of context switches = 6** (count the boundaries P1→P2, P2→P3, P3→P2, P2→P1, P1→P4, P4→P1; the first dispatch after idle and the final termination are not counted — see 4.15).

> ϟ **Speed trick for SRTF:** keep a one-line "remaining table". At each arrival write the new remaining values; only compare **new arrival vs. current runner** — all others in the queue are already larger than the runner.

4.7 Round Robin (RR) NOTES

Why? Interactive systems need quick **response**. RR = FCFS **+ a time quantum (slice) q**. Each process runs at most q; if it isn't finished it is moved to the **end** of the ready queue. Preemptive.

How the hardware helps: a **timer** = a counter + a countdown register + a comparator. While doing the context switch the OS loads the quantum δ into the timer; when it reaches 0 a **hardware interrupt** fires and the OS takes the CPU back (this is how the OS keeps control — *CPU protection*).

Steps NOTES

1. Pick the process at the **front** of the ready queue.
2. Set the timer to q and dispatch it.
3. If it finishes before q → go to step 1. If the timer expires → move it from running to the **end** of the ready queue → step 1.

Efficiency NOTES: if one context switch takes γ and the quantum is δ,

Useful CPU % = δ / (δ + γ) × 100 (if $\gamma \approx \delta$ the CPU wastes 50 %).

Rules of thumb NOTES

- **Smaller quantum → smaller (better) response time** (but more context switches and more overhead).
- **If q > the largest burst, RR = FCFS.**
- If q → very small, each of n processes seems to run on its own CPU n times slower ("processor sharing").
- If a process's remaining BT becomes **exactly 0 when the quantum expires**, whether it is called a "preemption" is ambiguous (depends which event the CPU sees first) — but a **context switch happens either way**.

> ⚠ **Queue-order convention (very important!):** if a **new process arrives at the very same instant** a running process's quantum expires, the **new arrival is placed in the ready queue BEFORE the preempted process**. In Example 4.6 the notes use exactly this (P3 arrived at 4, before P1 was re-queued at 5).

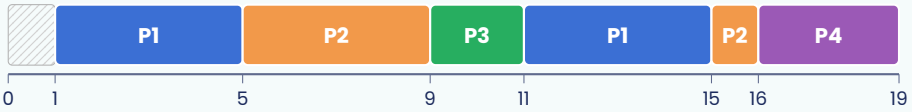
NOTES | Example 4.6 — RR, q = 4

P	AT	BT
P1	1	8
P2	2	5
P3	4	2
P4	10	3

Track the ready queue:

- t=1: P1 runs 1–5 (rem 4). During this, P2 (t=2) and P3 (t=4) arrive → queue [P2, P3]; at t=5 P1 is added → **[P2, P3, P1]**.
- t=5: P2 runs 5–9 (rem 1) → queue [P3, P1, P2] (P4 arrives at 10).
- t=9: P3 runs 9–11 (done). At t=10 P4 arrived → queue [P1, P2, P4].
- t=11: P1 runs 11–15 (done exactly).
- t=15: P2 runs 15–16 (done). t=16: P4 runs 16–19 (done).

GANTT CHART



P	CT	TAT	WT	RT
P1	15	14	6	0
P2	16	14	9	3
P3	11	7	5	5
P4	19	9	6	6

Avg WT = 26/4 = 6.5. Context switches = 5 (P1→P2, P2→P3, P3→P1, P1→P2, P2→P4). **Preemptions = 2** (P1 at t=5, P2 at t=9). *If there is a preemption there is a context switch, but not every context switch is a preemption* (a process finishing also gives one).

NOTES | Example 4.7 — Effect of the quantum

Five processes, all AT = 0, BT = **200, 2, 2, 2, 2** (P1 is the long one).

q = 100: `P1 0–100 | P2 100–102 | P3 102–104 | P4 104–106 | P5 106–108 | P1 108–208`

- WT: P1 = 8 (it waited 100→108), P2 = 100, P3 = 102, P4 = 104, P5 = 106 → **Avg WT = 420/5 = 84**
- RT: 0, 100, 102, 104, 106 → **Avg RT = 412/5 = 82.4**
- Context switches = 5.

q = 2: `P1 0–2 | P2 2–4 | P3 4–6 | P4 6–8 | P5 8–10 | P1 10–208`

- WT: 8, 2, 4, 6, 8 → **Avg WT = 28/5 = 5.6**
- RT: 0, 2, 4, 6, 8 → **Avg RT = 4**
- Context switches = 5 (P1 then runs alone, so no more switches); preemptions = 1.

Lesson: a smaller quantum slashes response time for the short jobs; a huge quantum makes RR behave like FCFS (the short jobs wait behind P1's 100).

⚡ **Counting RR context switches quickly:** write the Gantt chart, **merge consecutive segments of the same process**, and count the boundaries between *different* processes. (If every process gets $\lceil BT/q \rceil$ slots and no slot is followed by the same process, switches = total slots − 1.)

⚡ **RR with all AT = 0, n processes, quantum q:** the first response times are 0, q, 2q, ..., (n−1)q (for processes with BT ≥ q); a process waits at most $\mathbf{(n-1)\cdot q}$ between two turns. With context-switch overhead s, successive turns of a process are about $\mathbf{n\cdot(q+s)}$ apart. ADDED

4.8 LJF and LRTF NOTES

Longest Job First: pick the **largest** burst (non-preemptive; implemented with a max-heap). Its preemptive version is **LRTF** (Longest Remaining Time First). Rarely useful in practice but loved by GATE as a "trick" Gantt chart because the running process becomes *shorter* and everyone else eventually overtakes it.

NOTES | Example 4.8 — LRTF (preemptive LJF)

P	AT	BT
P1	0	2
P2	1	1
P3	2	3
P4	3	6

- t=0: only P1 → runs 0–1; at t=1, P1 rem 1, P2 (1) arrives: tie → keep P1 (don't switch on ties): P1 runs to 2.
- t=2: P1 done; P2 (1) and P3 (3) → P3 runs from 2; at t=3 P3 rem 2, P4 (6) arrives → **preempt**, P4 runs 3–...
- P4's remaining shrinks: 6 → 5 → 4 → 3 → 2 ...; at t=8 P4 rem 1; P3 rem 2 > 1 → P3 runs 8–10 (rem 2 → 0). Meanwhile P2 (1) is the shortest and goes last.

GANTT CHART



P	CT	TAT	WT	RT
P1	2	2	0	0
P2	11	10	9	9
P3	10	8	5	0
P4	12	9	3	0

Context switches = **5**, preemptions = **2** (P3 at t=3, P4 at t=8; as the notes say). (Tie rule: when remaining times are equal, GATE normally says to continue the running process or choose the earlier arrival.)

4.9 Priority scheduling NOTES

Each process has a priority number; the CPU goes to the **highest priority**. (By default **lower number = higher priority** — read the question!). **Preemptive** version: a newly arrived higher-priority process takes the CPU at once. **Non-preemptive**: it only goes to the front of the ready queue.

- **Explicit priority:** given by the user/administrator. **Implicit priority:** OS-derived (e.g., SJF is priority scheduling where priority = 1/burst time; interactive > CPU-bound; memory needs).
- Can be implemented with **multilevel queues** (a queue per priority).
- **Problem: starvation** — a low-priority process may wait forever. **Solution: aging** — gradually *raise* the priority of a process the longer it waits.

ADDED | Example 4.9 — Priority (lower number = higher priority)

P	AT	BT	Priority
P1	0	10	3
P2	1	5	1
P3	2	8	4
P4	3	6	2

Preemptive: `P1 0–1 | P2 1–6 | P4 6–12 | P1 12–21 | P3 21–29` → WT: P1 = 11, P2 = 0, P3 = 19, P4 = 3 → **avg WT = 33/4 = 8.25**. **Non-preemptive:** `P1 0–10 | P2 10–15 | P4 15–21 | P3 21–29` → WT: 0, 9, 19, 12 → **avg WT = 40/4 = 10.0**.

4.10 ADDED Three more algorithms GATE may mention (brief)

HRRN — Highest Response Ratio Next (non-preemptive)

Response ratio = (W + BT) / BT where W = time already waited. Pick the highest. Short jobs get high ratios; but **long jobs' ratio grows as they wait, so there is no starvation**. Example: P1(0,3) P2(2,6) P3(4,4) P4(6,5) P5(8,2): P1 0–3; P2 3–9 (only choice); at t=9 ratios: P3 = (5+4)/4 = 2.25, P4 = (3+5)/5 = 1.6, P5 = (1+2)/2 = 1.5 → P3 9–13; at t=13: P4 = (7+5)/5 = 2.4, P5 = (5+2)/2 = 3.5 → P5 13–15; P4 15–20. Order **P1, P2, P3, P5, P4**; WT = 0,1,5,9,5 → **avg 4.0**.

Multilevel Queue (MLQ) NOTES

Split the ready queue by *type of process* (e.g., **foreground/interactive** and **background/batch**). The OS first picks a **queue** (fixed priority — foreground always before background — or time-sliced between queues, e.g., 80 % / 20 %), and then uses **that queue's own algorithm** (e.g., RR for foreground, FCFS for background). A process **stays in its queue**. Starvation of lower queues is possible with fixed priority.

Multilevel Feedback Queue (MLFQ)

Like MLQ but a process can **move between queues**: new job → top queue (small quantum); if it uses its full quantum it is demoted (CPU-bound jobs sink); if it blocks for I/O early it stays high (interactive jobs stay on top); aging promotes starving jobs. It approximates SRTF without knowing burst times.

4.11 Scheduling with I/O bursts NOTES

So far every process was "pure CPU". Real processes alternate **CPU burst → I/O burst → CPU burst ...**. The CPU has **one** core, but **I/O devices are separate**, so the **I/O bursts of different processes overlap** with each other and with another process's CPU burst. (*Standard GATE assumption: I/O of different processes proceeds in parallel; the CPU never waits for another's I/O if something else is ready.*)

⚠ **Poor design (no overlap)** NOTES**:** P1 (AT 0: CPU 3, IO 21), P2 (AT 1: CPU 1, IO 3), P3 (AT 2: CPU 2, IO 3). If the system treats a burst as "CPU+IO" and runs them one after another, the total time is $24 + 4 + 5 = \mathbf{33}$, of which the CPU works only $3{+}1{+}2 = \mathbf{6}$ → **utilisation 6/33 ≈ 18 %**. The cure: while P1 is in I/O, give the CPU to the next ready process (the **convoy/I-O-bound** problem).

Method for I/O problems: keep **two timelines**. (1) The CPU Gantt chart chosen by the scheduling algorithm using only *ready* processes. (2) For each process note the instant it **returns from I/O** (becomes ready again). **CPU idle time = gaps in the CPU timeline.**

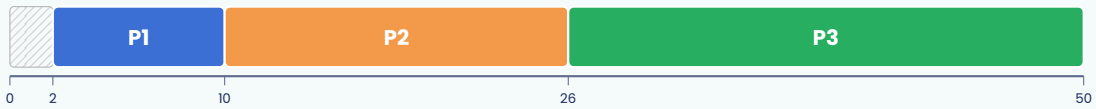
NOTES | Example 4.10 — FIFO with I/O first, then CPU

Each process does **I/O first, then CPU**. All arrive at 0.

P	I/O burst	CPU burst
P1	2	8
P2	4	16
P3	6	24

I/Os run in parallel: P1 becomes ready at 2, P2 at 4, P3 at 6.

CPU GANTT CHART



CT = 10, 26, 50 (= TAT, AT = 0). Total time 50; CPU busy = 8 + 16 + 24 = 48. **CPU utilisation = 48/50 = 96 %.** (Idle only for the first 2 units — until the first process finishes its I/O.)
Waiting in the ready queue = TAT − (CPU + I/O): P1 = 10 − 10 = **0**, P2 = 26 − 20 = **6**, P3 = 50 − 30 = **20**. These are exactly the response-time values the notes list (**0, 6, 20**), i.e., the notes measure RT from the moment the process *became ready* (first CPU time − its I/O burst).

NOTES | Example 4.11 — SRTF with CPU sandwiched between I/Os; "% idle CPU"

All arrive at 0; each process = `I/O₁ → CPU → I/O₂`.

P	I/O_1	CPU	I/O_2
P1	2	7	1
P2	4	14	2
P3	6	21	3

Timeline: CPU idle 0–2 (everyone is in their first I/O); P1 CPU 2–9; P2 CPU 9–23; P3 CPU 23–44; then P3's last I/O 44–47 (CPU has nothing to do). Total time = **47**; idle = 2 + 3 = **5**. **Idle % = 5/47 ≈ 10.6 %** (utilisation 42/47 = 89.4 %). (Why this order? At every moment only one process had its first I/O

finished — P1 at 2; when it ends at 9, P2 and P3 are ready; SRTF picks the shorter, P2, etc. The final I/Os of P1 and P2 overlap with others' CPU, so only P3's last I/O is exposed.)

ADDED | Example 4.12 — Two CPU bursts around an I/O (FCFS)

P	CPU_1	I/O	CPU_2
P1	2	5	2
P2	3	1	1

All arrive at 0, FCFS.
P1 CPU 0–2 → I/O 2–7. P2 CPU 2–5 → I/O 5–6. CPU idle **5–6**. P2 (ready at 6) CPU 6–7. P1 (ready at 7) CPU 7–9. Completion: P2 = 7, P1 = 9. Busy = 8 of 9 → **utilisation 88.9 %**.

> ϟ **I/O-problem checklist:** (1) mark the instant each process becomes ready; (2) run the algorithm only on ready processes; (3) idle = gaps; (4) the **last I/O of the last finisher** can add to total time with an idle CPU; (5) utilisation = Σ CPU bursts / total time.

4.12 ADDED Multi-CPU (multiprocessor) scheduling

GATE sometimes gives **2 CPUs**. Same recipe, but **at each decision point a process can go to *any* free CPU** (take the lowest-numbered CPU if a tie).

Example 4.13 — FCFS on 2 CPUs. P1(0,5), P2(1,3), P3(2,8), P4(3,2), P5(4,4).

- t=0 P1 → CPU1 (0–5). t=1 P2 → CPU2 (1–4). P3, P4, P5 queue up (in arrival order).
- t=4 CPU2 free → P3 (4–12). t=5 CPU1 free → P4 (5–7). t=7 CPU1 free → P5 (7–11).

CT = 5, 4, 12, 7, 11; TAT = 5, 3, 10, 4, 7 (avg **5.8**); WT = 0, 0, 2, 2, 3 (avg **1.4**). Total finish = 12.

The same idea works for SJF/SRTF/RR: *"whenever any CPU becomes free (or a process arrives and a CPU is free) assign the best ready process"*; for preemptive algorithms, the arriving process may preempt the **running process with the largest remaining time** among all CPUs.

4.13 ϟ Master cheat-sheet: which algorithm is best / worst at what?

	FCFS	SJF	SRTF	RR	Priority	LJF/LRTF	HRRN
Preemptive	no	no	**yes**	**yes**	either	no / yes	no
Starvation	no	**yes**	**yes**	**no**	**yes** (cure: aging)	**yes**	no
Convoy effect	**yes**	less	no	no	–	**yes**	no
Min avg WT	–	min among non-preemptive	**min overall**	–	–	–	–
Needs BT knowledge	no	**yes**	**yes**	no	no	yes	yes
Good for	batch	batch	batch	**interactive**, best RT	real-time/critical	–	batch
Best response	–	–	–	**yes (small q)**	–	–	–

- **Overhead:** RR and preemptive algorithms have more context switches.
- **RR with q → ∞ = FCFS.**
- **Same BT for all, AT = 0:** FCFS = SJF (all give the same average WT); RR is worse.
- **Which has the lowest avg WT?** SRTF (overall), SJF (non-preemptive).

4.14 MSQ / statement bank (with reasons)

1. *FCFS can cause starvation.* — **False** (everyone is served in finite time; only the *convoy effect*).
2. *SJF (non-preemptive) minimises average waiting time among all scheduling algorithms.* — **False** — only among **non-preemptive** ones; SRTF is better overall.
3. *SRTF can cause starvation of long processes.* — **True.**
4. *In RR with a very large quantum the algorithm behaves like FCFS.* — **True.**
5. *Decreasing the quantum in RR always decreases average waiting time.* — **False** (it lowers response time but raises context-switch overhead; avg WT can go up).
6. *Round Robin gives the lowest average response time for interactive jobs.* — **True** (in practice: small q).
7. *Priority scheduling may lead to starvation; aging solves it.* — **True.**
8. *Throughput is the number of processes completed per unit time.* — **True.**
9. *Under non-preemptive scheduling RT = WT.* — **True.**
10. *With preemptive scheduling, a process can be moved from running to ready by a timer interrupt.* — **True.**
11. *The dispatcher chooses the next process to run.* — **False** — the **scheduler** chooses; the **dispatcher** performs the switch.
12. *Long-term scheduler controls the degree of multiprogramming.* — **True.**
13. *Context-switch time is "useful work".* — **False** — pure overhead.
14. *CPU utilisation can be increased by overlapping I/O of one process with CPU of another.* — **True.**
15. *Exponential averaging with $\alpha = 0$ ignores the most recent burst.* — **True** ($S(n+1) = S_n$).

4.15 Conventions and traps in scheduling questions

1. **Idle CPU at the start:** if no process has arrived at t = 0, the CPU idles until the first AT. (Examples 4.1, 4.3, 4.5, 4.6.)
2. **Counting context switches:** count the boundaries between *different* processes in the **merged** Gantt chart. **Do not count** the first dispatch from idle, the end of the last process, or "switches" from a process to itself. If the question says *"including the first and last"*, add them. Different books vary — the safe approach is the "Pi→Pj" count used in the notes (SRTF example = 6, RR example = 5, LRTF example = 5).
3. **Preemption vs context switch:** a *preemption* happens only when a process is forcibly removed **before it finishes**. A process finishing → context switch without preemption.
4. **RR tie at the same instant:** new arrival is queued before the pre-empted process.
5. **SRTF/LRTF ties:** continue the running process (don't switch on equal remaining time).
6. **Priority number:** lower number = higher priority *unless stated otherwise*.
7. **TAT is measured from AT**, not from 0.
8. **Response time ≠ waiting time** for preemptive algorithms (RT is only until the *first* run).
9. **WT includes only ready-queue time** — not I/O time.

4.16 Quick-revision capsule

- TAT = CT − AT; WT = TAT − BT; RT = first run − AT. Non-preemptive ⇒ RT = WT.
- FCFS: convoy effect. SJF: min WT (non-preemptive), starvation, predict BT: $\mathbf{S(n+1) = \alpha \cdot t_n + (1-\alpha) \cdot S_n}$. SRTF: min WT overall, preempts on arrival.
- RR: queue + quantum; small q → good response, high overhead; q ≥ max BT → FCFS; efficiency = q/(q+s).
- Priority: aging fixes starvation. HRRN: (W+BT)/BT. MLQ: queues fixed; MLFQ: processes move.
- With I/O: overlap I/O with other CPU bursts; utilisation = ΣCPU / total time; idle = gaps.

4.17 ADDED Practice set (GATE style) — try on paper first

Q1. P1(AT 0, BT 7), P2(2, 4), P3(4, 1), P4(5, 4). Find the **average waiting time** under (a) FCFS (b) SJF (c) SRTF (d) RR with q = 3.

Q2. Four processes arrive at 0 with BT = 6, 8, 7, 3 (A, B, C, D). Find the average WT under (a) FCFS in the order A,B,C,D, (b) SJF, (c) RR with q = 3.

Q3. P1(0,10,pr 3), P2(1,5,pr 1), P3(2,8,pr 4), P4(3,6,pr 2); lower number = higher priority. Average WT under preemptive and non-preemptive priority?

Q4. In which order does **HRRN** schedule P1(0,3), P2(2,6), P3(4,4), P4(6,5), P5(8,2), and what is the avg WT?

Q5. Two CPUs, FCFS: P1(0,5), P2(1,3), P3(2,8), P4(3,2), P5(4,4). Find avg WT and avg TAT.

Q6. Two processes, FCFS, both AT 0. P1: CPU 2 → I/O 5 → CPU 2. P2: CPU 3 → I/O 1 → CPU 1. Find CPU utilisation and each CT.

Q7. Previous CPU bursts of a process: 6, 4, 6, 4. $S_0 = 10$, $\alpha = 0.5$. Predict the next burst using exponential averaging.

Q8. In RR, the quantum is 4 ms and a context switch costs 1 ms. (a) What fraction of time is useful? (b) What minimum quantum gives ≥ 90 % efficiency?

Q9. RR (q = 2): P1(0,5), P2(1,3), P3(2,1). Draw the Gantt chart, avg WT, number of context switches.

Q10. P1(0,8), P2(1,4), P3(2,9), P4(3,5): avg WT under SRTF and under SJF?

Q11. Which of these can lead to starvation: FCFS, RR, SJF, SRTF, priority, HRRN?

Q12. A system has n = 10 processes, all CPU-bound, RR with q = 20 ms and a context switch of s = 2 ms after every quantum. How long does a process wait between the end of its own quantum and the start of its next turn?

Solutions

Q1. (a) FCFS: `P1 0–7 | P2 7–11 | P3 11–12 | P4 12–16` → WT = 0, 5, 7, 7 → **4.75**. (b) SJF: at t=7 ready P2(4), P3(1), P4(4) → `P1 0–7 | P3 7–8 | P2 8–12 | P4 12–16` → WT = 0, 6, 3, 7 → **4.0**. (c) SRTF: `P1 0–2 | P2 2–4 | P3 4–5 | P2 5–7 | P4 7–11 | P1 11–16` → WT: P1 = 9, P2 = 1, P3 = 0, P4 = 2 → **3.0**. (d) RR q=3: `P1 0–3 | P2 3–6 | P1 6–9 | P3 9–10 | P4 10–13 | P2 13–14 | P1 14–15 | P4 15–16` → WT: P1 8, P2 8, P3 5, P4 7 → **7.0**. (*Order of avg WT: SRTF < SJF < FCFS < RR. RR pays for good response time with larger WT.*)

Q2. (a) WT = 0, 6, 14, 21 → **10.25**. (b) SJF order D(3), A(6), C(7), B(8): WT = D 0, A 3, C 9, B 16 → **7.0**. (c) RR q=3: `A 0–3 | B 3–6 | C 6–9 | D 9–12 | A 12–15 | B 15–18 | C 18–21 | B 21–23 | C 23–24` → WT: A 9, B 15, C 17, D 9 → **12.5**.

Q3. Preemptive `P1 0–1 | P2 1–6 | P4 6–12 | P1 12–21 | P3 21–29`: WT = 11, 0, 19, 3 → **8.25**. Non-preemptive `P1 0–10 | P2 10–15 | P4 15–21 | P3 21–29`: WT = 0, 9, 19, 12 → **10.0**.

Q4. Order **P1, P2, P3, P5, P4**; WT = 0, 1, 5, 9, 5 → **avg 4.0** (computation in §4.10).

Q5. CT = 5, 4, 12, 7, 11 → TAT avg **5.8**; WT avg **1.4** (§4.12).

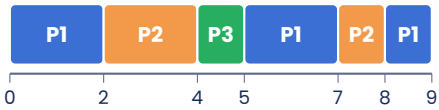
Q6. P1 CPU 0–2, P2 CPU 2–5, CPU idle 5–6, P2's 2nd burst 6–7 (done at 7), P1's 2nd burst 7–9 (done at 9). Busy = 8 of 9 → **88.9 %**; CT(P1) = 9, CT(P2) = 7.

Q7. $S_1 = 0.5{\cdot}6 + 0.5{\cdot}10 = 8$; $S_2 = 0.5{\cdot}4 + 0.5{\cdot}8 = 6$; $S_3 = 0.5{\cdot}6 + 0.5{\cdot}6 = 6$; $S_4 = 0.5{\cdot}4 + 0.5{\cdot}6 = \mathbf{5}$ → predicted next burst = **5**.

Q8. (a) 4/(4+1) = **80 %**. (b) $q/(q+1) \geq 0.9 \Rightarrow q \geq$ **9 ms**. (General: $q \geq s{\cdot}e/(1-e)$.)

Q9. Queue at t=2: [P2, P3, P1].

GANTT CHART



CT = 9, 8, 5; TAT = 9, 7, 3; WT = 4, 4, 2 → avg **3.33**. Context switches (P1→P2, P2→P3, P3→P1, P1→P2, P2→P1) = **5**.

Q10. SRTF: `P1 0–1 | P2 1–5 | P4 5–10 | P1 10–17 | P3 17–26` → WT = 9, 0, 15, 2 → avg **6.5**. SJF: `P1 0–8 | P2 8–12 | P4 12–17 | P3 17–26` → WT = 0, 7, 15, 9 → avg **7.75**. (SRTF wins again.)

Q11. **SJF, SRTF, priority** (and LJF/LRTF). FCFS, RR and HRRN do not starve.

Q12. Timeline after your quantum ends: switch (2) → 9 other processes each run 20 ms with a switch (2) after each. So wait = 9 × 20 + 10 × 2 = **200 ms**. (Start-to-start of your turns = n(q+s) = 10 × 22 = 220 ms = 20 ms run + 200 ms wait.)

PART III

Concurrency

CHAPTER 5

Process Synchronization

GATE IMPORTANCE ★★★★☆ *≈ 1.2 marks/paper, "Tier 1"*

◎ **GATE pattern (from your analysis):** (a) *"Does this proposed critical-section solution satisfy mutual exclusion / progress / bounded waiting?"* (b) **Semaphore programs** — "what output can be printed", "max/min value of a shared variable", "how many processes can be blocked" (c) classic problems — *producer-consumer, readers-writers, dining philosophers* — "what is wrong if we swap two lines?" (d) 1-mark MSQ on spinlock / test-and-set / Peterson. If you master **§5.3 (the checklist)** and **§5.9 (semaphore puzzles)**, you can collect most of these marks.

5.1 The problem: shared data + interleaving = wrong answers

NOTES **The chef story.** Two chefs share a **store** and an **inventory list (ILIST)**. To cook they do three steps: (1) *check the list* for butter, (2) *collect butter* from the store, (3) *update the list*. If chef C1 checks ("butter available"), then C2 checks ("butter available"), both go to collect, and the second finds the store empty. **Fix: "only one chef at a time may do check–collect–update."** That one sentence *is* the whole subject of synchronization.

> **Synchronization** = the set of rules (protocols) that make *concurrent* contexts which *share resources* still execute **correctly**. NOTES

Race condition NOTES — the final result depends on the **order in which concurrent contexts run**. If the shared data is only *read* there is no issue; **even one writer** makes order matter.

Recall from Chapter 3: `count++` is `LOAD` → `INCR` → `STORE`; interleaving T1I1, T2I1, T1I2, T2I2, T1I3, T2I3 leaves **count = 1** instead of 2 (and *any one wrong interleaving is enough* to show a solution is wrong). NOTES Different sequences:

Interleaving	count
T1I1 T1I2 T1I3 T2I1 T2I2 T2I3 (one after the other)	2 ✔
T1I1 T2I1 T2I2 T2I3 T1I2 T1I3	**1** ✘
T1I1 T2I1 T1I2 T2I2 T1I3 T2I3	**1** ✘

Total interleavings = $6!/(3!{\cdot}3!) = \mathbf{20}$.

5.2 The Critical Section (CS) problem

Critical section NOTES = the bunch of instructions that manipulate **shared data** and whose result depends on the order of execution (here: `count++`).

The CS problem: *design the code around the CS so that when one context is inside its CS, no other context is inside its CS on the same data.*

Structure of every participant NOTES:

```
do {
    ENTRY SECTION      // ask permission
        CRITICAL SECTION
    EXIT SECTION       // announce leaving
    REMAINDER SECTION  // everything else
} while (true);
```

The three requirements (memorise exactly — GATE asks them over and over) NOTES

#	Requirement	Plain meaning
1	**Mutual Exclusion (ME)**	At most **one** context in the CS at a time.
2	**Progress**	If nobody is in the CS and some contexts want to enter, then **only those not in their remainder section** take part in deciding who goes next, and this decision **cannot be postponed forever** (⇒ no deadlock; a process sitting in its remainder section must not block others).
3	**Bounded Waiting (BW)**	After a process has **requested** entry, there is a **fixed upper bound** on the number of times others can enter before it is granted (⇒ no starvation). The bound may be huge (1 sec or 10 years) but must exist.

(*Plus assumptions: no assumption about relative speeds; each CS is finite.*)

> ϟ **Easy way to remember:** ME = **safety** (nothing bad); Progress = **no deadlock / no unnecessary blocking**; BW = **fairness / no starvation**.

NOTES The first (wrong) attempt — a single flag

```
int flag = 0;
foo() {
    while (flag == 1);   // wait
    flag = 1;            // I am in
    count++;             // CS
    flag = 0;            // I am out
}
```

- **Analysis 1:** if T1 runs completely and then T2 → count = 2 ✔ (a *sequential* test proves nothing).
- **Analysis 2 (counter-example):** T1 checks `flag` (0) and exits the loop; **before it sets flag=1**, T2 checks `flag` (0) and exits the loop. T1 sets flag=1, enters (count=1); T2 sets flag=1 …; T1 resets flag=0; T2 enters → **count = 1** ✘. *One counter-example is enough to reject a solution.*
- **Why it failed:** *checking* and *setting* are **two** separate steps. The entry section must complete in **one indivisible (atomic)** step → we need hardware help (§5.5).

5.3 How to check a CS solution (the 5-minute checklist) NOTES

Mutual exclusion — try to find a schedule where two are inside: (a) one is in the CS and another tries → the other must **block at entry**; (b) two or more are **in the entry code simultaneously** → **at most one** gets through.

Progress (= absence of deadlock/blocking): (a) nobody is in the CS and Pi arrives (the other being in its remainder section, or not interested) → Pi **must be able to enter**; (b) two or more in the entry section → **at least one** enters (no deadlock). *Always test the corner cases:* "only Pi wants the CS", "the other process never wants it", "both set their flags simultaneously".

Bounded waiting (fairness) — assume progress holds and someone else is contending; count how many times the others can enter between Pi's request and Pi's entry (each Pi in a loop). If you can give a **constant K** it holds. *Example: in a deadlock nobody enters at all → BW holds trivially with K = 0, but progress is violated.* NOTES

> ⚠ **Standard exam logic:** *Disprove* with a single bad schedule; *prove* only by reasoning over all schedules.

5.4 Software solutions for two contexts (T0 and T1)

Algorithm 1 — Strict Alternation (shared `turn`) NOTES

```
int turn = 0;
T0: while (turn != 0);  CS0;  turn = 1;
T1: while (turn != 1);  CS1;  turn = 0;
```

- **ME** ✔ (`turn` is read atomically; it is 0 or 1, never both).
- **Progress** ✘ — if `turn = 0` but T0 does not want the CS (stays in its remainder section), **T1 waits forever** although the CS is empty.
- **BW** ✔, bound = **1** (they must alternate: the other can enter at most once in between).

Algorithm 2 — `want[]` flags NOTES

```
int want[2] = {0,0};
T0: while(1){ want[0]=1; while(want[1]); CS; want[0]=0; }
T1: while(1){ want[1]=1; while(want[0]); CS; want[1]=0; }
```

- **ME** ✔.
- **Progress** ✘ — **DEADLOCK:** both set `want = 1` at the same time, each spins on the other's flag forever.
- If one never wants the CS, the other can enter unlimited times (fine). If no deadlock occurs they alternate → BW = 1.

Algorithm 3 — Peterson's solution NOTES (the only classic *software* solution that satisfies all three)

```
int want[2] = {0,0};  int turn = 0;
T0: want[0] = 1; turn = 1;  while (want[1] && turn == 1);  CS;  want[0] = 0;
T1: want[1] = 1; turn = 0;  while (want[0] && turn == 0);  CS;  want[1] = 0;
```

How it works (i = me, j = the other): *"I want in (want[i]=1) — but I politely give the **turn to the other** (turn=j). I wait only while **the other also wants in AND it is the other's turn**."*

- **ME** ✔ in every interleaving (if both want, `turn` — a single variable — is either 0 or 1, so only one loop condition stays true).
- **Progress** ✔ — if Pj does not want the CS, `want[j] = 0`, so Pi enters at once; if both want, `turn` breaks the tie.
- **BW** ✔**, bound = 1** — after Pi exits and tries again it sets `turn = j` *before* waiting, so Pj goes first.

Peterson "what if we change…" (verified by exhaustive model-checking)

Modification	Result
Original	ME ✔, no deadlock ✔
Set `turn` BEFORE `want` (`turn=j; want[i]=1;`)	**ME violated** ✘
`while (want[j] || turn == j)` (**OR** instead of AND)	**Deadlock** ✘
Drop `turn` (Algorithm 2)	**Deadlock** ✘
Check first, then set the flag (`while(want[j]); want[i]=1;`)	**ME violated** ✘
`turn = i; while(want[j] && turn == i);` (relabelled)	Still correct (just the opposite convention) ✔
Single lock flag, `while(flag); flag=1;`	**ME violated** ✘
Exit section forgets `want[i]=0`	Other process blocks forever → **progress** ✘

NOTES Worked classification examples

EX 1. `T1: while(x==1); x=1; CS` — `T2: while(x==0); x=0; CS`

- **ME** ✘ (counter-example from the notes): x = 0 → T1 exits its loop and **sets x = 1**, entering its CS. Now T2's loop condition (`x == 0`) is false, so T2 exits its loop, **sets x = 0** and enters *its* CS — **both are in the CS** at the same time. ✘
- **Progress** ✘**:** if x=1 and T2 doesn't want the CS, T1 can never enter (needs x=0, which only T2 sets).
- BW: by definition at most 1 process enters before Pi (they alternate) → **BW = 1**.

EX 2. `bool S1=S2` initially. `T1: while(S1==S2); CS; S1=S2;` — `T2: while(S1!=S2); CS; S2=!S1;`

- The two loop conditions are **complements** (S1==S2 vs S1≠S2), evaluated atomically ⇒ **ME** ✔.
- Initially S1=S2 → T1 is blocked, T2 may enter. If T2 never wants the CS, T1 waits forever ⇒ **Progress** ✘ (this is strict alternation in disguise).
- **BW = 1** (turns strictly alternate).

5.5 Hardware support — atomic instructions

Option A: disable interrupts NOTES

Entry: disable interrupts (no preemption!) … exit: enable. ⊗ A user program should never be allowed to (a bug → system hang; e.g., a segmentation-fault inside the CS), and ⊗ on a **multi-CPU** machine it does **not** work (each CPU has its own interrupt line; another CPU can still enter).

Option B: Test-and-Set (TSL) — an *atomic* instruction NOTES

```
int test_set(int *flg) {      // executed as ONE indivisible step
    int v = *flg;
    *flg  = 1;
    return v;                 // returns the OLD value
}
```

Solution: **spinlock**

```
int lk = 0;                          // 0 = free, 1 = locked
lock(addr)   { while (test_set(addr)); }   // spin until old value was 0
unlock(addr) { *addr = 0; }

foo() { lock(&lk); count++; unlock(&lk); }
```

Trace NOTES: T1's `test_set` returns 0 atomically and sets 1 → T1 enters. T2's `test_set` returns 1 → spins. T1 finishes, `lk=0`. T2's next `test_set` returns 0 → T2 enters → count = 2 ✔.

Properties of the simple TSL spinlock

- **ME** ✔ (guaranteed by atomicity). Progress ✔ (if lock is free the first `test_set` gets it).
- **Bounded waiting** ✘ NOTES: with `foo(){while(1){lock(&f); CS1; unlock(&f);}}` and `bar(){while(1){lock(&f); CS2; unlock(&f);}}` T1 can re-acquire the lock again and again — CS1 may run K times while CS2 runs 0 times, for *any* K (unbounded) ⇒ **starvation possible**.
- ⊗ **Busy waiting** (spinning) wastes CPU cycles.
- ⊗ **Priority-inversion deadlock** NOTES: low-priority Pi holds the lock and is pre-empted by a high-priority P2 that now spins on the lock forever (Pi can never run to release it) → deadlock on a single CPU with strict priority scheduling.
- ✔ Spinlocks are good only when the CS is **very short** (cheaper than a context switch) and there are multiple CPUs.

Option C: bounded-waiting solution with Test-and-Set (n processes) NOTES

```
bool waiting[n];   bool lock = false;     // shared

do {                                       // process i
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* ---- critical section ---- */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j]) j = (j + 1) % n;   // look for next waiting process in cyclic order
    if (j == i) lock = false;              // nobody waiting: free the lock
    else        waiting[j] = false;        // hand the CS directly to Pj
    /* ---- remainder section ---- */
} while (true);
```

- **ME:** Pi enters only if `key == false` (its own TSL found the lock free — only one can) **or** `waiting[i] == false` (set by the exiting process — which keeps the lock locked, so only one is released).
- **Progress:** the exiting process either unlocks the lock or releases one `waiting[j]` — so someone always gets in.
- **BW = n − 1:** the exiting process scans **cyclically** starting from i+1, so every waiting process is served within **n − 1** turns.

5.6 Blocking locks (mutex) — don't waste the CPU NOTES

A spinlock burns CPU while waiting. Better: if the lock is taken, **move the thread to a waiting queue (blocked state)** and **wake it when the lock is released** (needs OS support: kernel threads; a user-level library can do it for user-level threads).

```
acquire(lock) { while (test_and_set(lock)) move_self_to_wait_queue(); }
release(lock) { *lock = 0; move_one_or_more_threads_to_ready_queue(); }
```

- "Blocked" = *waiting* state (Ch 2); it does **not** hold the CPU.
- **ME ✔, Progress ✔, no busy waiting ✔, Bounded waiting ✘** (the awoken thread may lose the race again).

5.7 Semaphores NOTES (the most important tool)

A **semaphore** is an integer variable `S` that can be changed **only** through two **atomic** operations:

```
wait(S)   (also P / down):   while (S <= 0) ;   S--;
signal(S) (also V / up):     S++;
```

(Busy-wait version. Both operations are atomic — implement them by wrapping with `acquire(lock)/release(lock)`.)

Blocking implementation NOTES (what people mean by "semaphore" in GATE): instead of spinning, the process **sleeps**.

```
wait(S):    S.value--;  if (S.value < 0) { add this process to S.queue; block(); }
signal(S):  S.value++;  if (S.value <= 0) { remove a process P from S.queue; wakeup(P); }
```

> ϟ **Reading the value:**

- `S.value` $\geq$ `0` → number of **free units** (how many more `wait`s will pass without blocking).
- `S.value` = $-$`k` (k > 0) → exactly **k processes are blocked** on S.

Two kinds: | | Values | Use | |—|—|—| | **Binary semaphore** (mutex semaphore) | 0 or 1 | mutual exclusion | | **Counting semaphore** | any integer $\geq$ 0 (initial value = number of units) | resources with several identical instances; event counting |

> ⚠ **Binary semaphore trap:** calling `signal` on a binary semaphore that is already 1 leaves it at **1** (no stacking). On a *counting* semaphore each `signal` adds one.

The three patterns you will use 99 % of the time

1. **Mutual exclusion:** `S = 1`: `wait(S); CS; signal(S);`
2. **Ordering / signalling** (A must run before B): `S = 0`: A: `…; signal(S);` B: `wait(S); …`
3. **Resource counting:** `S = number of resource units`.

NOTES Deadlock with semaphores (two locks)

```
T1: wait(S1); wait(S2); CS; signal(S2); signal(S1);
T2: wait(S2); wait(S1); CS; signal(S1); signal(S2);
```

Sequence `T1:wait(S1)`, `T2:wait(S2)`, `T1:wait(S2)` → blocked, `T2:wait(S1)` → blocked: **deadlock** (each waits for what the other holds). Cure: **same acquiring order in both** (exhaustively verified: then no deadlock). (More in Chapter 6.)

5.8 The three classic problems NOTES

5.8.1 Dining Philosophers

Setup: 5 philosophers sit round a table; between each pair there is **one chopstick** (5 chopsticks, an array of binary semaphores `C[0..4]`). To eat, a philosopher needs **both** adjacent chopsticks. Requirements: *no chopstick is held by two philosophers; nobody starves.* (Philosophers = processes, chopsticks = resources.)

Protocol 1 (naive): every philosopher picks **right (Ci) then left (C(i+1)%5)**. Failure: all five pick their right one simultaneously → everybody waits for the left → **deadlock**, everyone starves. All four deadlock conditions hold: *mutual exclusion* (a chopstick is used by one), *hold and wait* (holds one, waits for the other), *no preemption* (can't snatch), *circular wait* (P1 holds C5 wants C1 held by P2, ... P5 wants C5 held by P1).

Protocol 2 (asymmetric): odd-indexed philosophers pick **left first, then right**; **even-indexed** pick **right first, then left**:

```
odd i :  wait(C[left]);  wait(C[right]);  eat;  signal(C[right]); signal(C[left]);  think
even i:  wait(C[right]); wait(C[left]);   eat;  signal(C[left]);  signal(C[right]); think
```

→ breaks the circular wait ⇒ **no deadlock, no starvation**.

Protocol 3: pick chopsticks **only if both are available**; otherwise **put down** whatever you picked (relinquish) — breaks *hold and wait*. (Without more care this can lead to livelock/starvation.)

ADDED **Protocol 4:** allow at most **4** philosophers to sit at the table at once (counting semaphore `room = 4`) → by pigeonhole at least one gets two chopsticks ⇒ no deadlock.

NOTES **GATE-style statement:** *"A dining-philosophers system with 5 chopsticks and **4** philosophers: (A) deadlock is not possible — **TRUE** (pigeonhole: some philosopher can always get both). (B) starvation is not possible — **FALSE** (it depends on the algorithm; the system may let two philosophers alternate and starve a third)."*

5.8.2 Producer–Consumer (bounded buffer) NOTES

A producer puts items into a buffer of **N** slots; a consumer removes them.

```
int buff[N];  int in = -1, out = -1;          // as written in the notes (pre-increment)
semaphore empty = N;   // counts EMPTY slots   (counting)
semaphore full  = 0;   // counts FULL slots    (counting)
semaphore mu    = 1;   // mutual exclusion on the buffer / in / out (binary)

Producer:                                     Consumer:
 do {                                          do {
   wait(empty);   // a free slot?                wait(full);    // an item?
   wait(mu);                                    wait(mu);
     in = (in+1) % N; buff[in] = item();           out = (out+1) % N; item = buff[out];
   signal(mu);                                  signal(mu);
   signal(full);  // one more item               signal(empty); // one more free slot
 } while(true);                                } while(true);
```

Why three semaphores? `empty` blocks the producer when the buffer is **full**; `full` blocks the consumer when it is **empty**; `mu` protects the indices.

> ⚠ **GATE traps (very common):**

1. **Swap `wait(empty)` and `wait(mu)`** in the producer → **deadlock**: buffer full (empty = 0), producer takes `mu` and blocks on `empty`; the consumer needs `mu` to free a slot → both stuck. **Rule: always wait for the counting semaphore *first*, the mutex *last*.**
2. Swapping the two `signal`s is harmless (no deadlock) — signals never block.
3. **Remove `mu`** ⇒ wrong with **multiple** producers (or multiple consumers) — two producers could write the same slot. With exactly **one producer and one consumer** `mu` is **not** needed (they touch different indices).
4. Initial values: `empty = N`, `full = 0`. If `empty = N−1`, only N−1 slots are usable.

5.8.3 Readers–Writers NOTES

Many readers may read together; a writer needs **exclusive** access (no readers, no other writer).

```
binary_semaphore mu = 1, write = 1;   int reader_count = 0;

Reader:                                   Writer:
 do {                                       do {
   wait(mu);                                  wait(write);
   reader_count += 1;                         /* WRITE */
   if (reader_count == 1) wait(write);   // first reader locks out writers
   signal(mu);                                signal(write);
   /* READ */                               } while(true);
   wait(mu);
   reader_count -= 1;
   if (reader_count == 0) signal(write); // last reader lets writers in
   signal(mu);
 } while(true);
```

- `mu` protects `reader_count`; `write` is the lock for the data.
- The **first** reader locks `write` for all readers; the **last** one releases it.
- **Properties:** readers + readers ✔ concurrent; reader + writer ✘; writer + writer ✘.
- This is the **readers-preference** version → a **writer may starve** if readers keep arriving (BW ✘ for writers). A writers-preference version avoids that but can starve readers.

5.9 ⚡ ADDED Semaphore puzzles — how to solve them

Method: (1) write the initial values; (2) find a process that can run *without* a `wait` blocking — only it can start; (3) follow who signals whom; (4) a **cycle of waits with all semaphores 0 = deadlock**.

All outputs below were verified by exhaustive search of all interleavings.

Puzzle 1 — Alternation. `S=1, T=0` `P1: repeat 3×: wait(S); print A; signal(T)` `P2: repeat 3×: wait(T); print B; signal(S)` → the **only** possible output is **ABABAB**.

Puzzle 2 — Ring of three. `S1=1, S2=0, S3=0` `P1: wait(S1); print 1; signal(S2)` `P2: wait(S2); print 2; signal(S3)` `P3: wait(S3); print 3; signal(S1)` → only **123**. If **all** three semaphores start at 0 → **deadlock**, nothing is printed.

Puzzle 3 — Signal after print. `S = 0`: `P: wait(S); print X` `Q: print Y; signal(S)` → only **YX**.

Puzzle 4 — Chain. `s=0, t=0`: `P: print A; signal(s)` `Q: wait(s); print B; signal(t)` `R: wait(t); print C` → only **ABC**.

Puzzle 5 — Counting semaphore as "k at a time". `S = 2` and three processes each doing `wait(S); [use]; signal(S)` → **at most 2** are inside simultaneously; the third blocks (S = −1 when all three tried).

Puzzle 6 — Lock order. `S1 = S2 = 1`; `T1: wait(S1); wait(S2); signal(S2); signal(S1)` `T2: wait(S2); wait(S1); signal(S1); signal(S2)` → **deadlock possible**. Make T2 use the *same* order as T1 → no deadlock.

Puzzle 7 — Value arithmetic. A counting semaphore starts at **10**. **6** `wait`s and **4** `signal`s are completed (none blocked) → **10 − 6 + 4 = 8**. If it starts at 2 and 5 `wait`s are issued with no `signal`s → value = **−3**, i.e., **3 processes blocked**.

ADDED Puzzle 8 — "Max and min of x" with a counting semaphore

Shared `x = 0`. Four processes W, X, Y, Z each run `wait(S); r = x; r = r + k; x = r; signal(S);` (load, add, store are **separate** steps). W and X use k = **+1**, Y and Z use k = **−2**. What are the maximum and minimum final values of x if **S = 1**? if **S = 2**?

- **S = 1:** it behaves like a mutex → the result is always the plain sum: $1 + 1 - 2 - 2 = -2$ (max = min = -2).
- **S = 2:** two processes may be inside together, so a **stale load** is possible.
- **Max = 2:** W runs completely (x = 1). X loads 1 and stalls. Y and Z run completely (x = −3). X now stores $1 + 1 = 2$.
- **Min = −4:** W loads 0 and stalls; Y loads 0; W stores 1; X runs (x = 2); Y stores $0 - 2 = -2$; then Z runs: $-2 - 2 = -4$.
- So with S = 2: **max = 2, min = −4**.
- ADDED A variant used in practice: W, X add **+3**, Y, Z add **−2**. S = 1 → always **2**. S ≥ 2 → **max 6, min −4** (possible values −4, −2, −1, 1, 2, 3, 4, 6).

> **General idea for such max/min problems:** the *maximum* comes from the schedule in which the *positive* updates are the **last stores** and the negative ones are lost; the *minimum* from the opposite. A stale load = the "lost update".

5.10 Fork/threads + shared variables (link back to Chapter 3)

- Two threads each doing `x = x + 1` n times without sync: **x** ∈ **[2, 2n]** for n ≥ 2 (n = 1: {1, 2}).
- k threads (k ≥ 2, n ≥ 2): min is still **2**, max is **k·n**.
- With a correct mutex, the result is exactly **k·n**.

5.11 MSQ / statement bank (with reasons)

1. *Peterson's algorithm works for any number of processes.* — **False** (the basic version is for **two**).
2. *Strict alternation satisfies progress.* — **False.**
3. *Test-and-Set spinlock guarantees bounded waiting.* — **False.**
4. *Test-and-Set guarantees mutual exclusion.* — **True** (atomicity).
5. *A spinlock is preferable to a blocking lock when the critical section is very short on a multiprocessor.* — **True.**
6. *Disabling interrupts is a good way to achieve ME on multiprocessors.* — **False.**
7. *`wait` and `signal` must be executed atomically.* — **True.**
8. *If S = −3 for a blocking semaphore, three processes are blocked.* — **True.**
9. *A binary semaphore can implement a counting semaphore.* — **True** (with extra variables/semaphores) — but not directly.
10. *Using semaphores can lead to deadlock if the acquiring order is inconsistent.* — **True.**
11. *In the producer–consumer solution, `empty` is initialised to 0.* — **False** (it's N).
12. *In readers–writers (readers preference) writers may starve.* — **True.**
13. *Mutual exclusion + progress already imply bounded waiting.* — **False** (the TSL spinlock has ME and progress but not BW).
14. *Deadlock ⇒ progress violated; bounded waiting may still hold (K = 0).* — **True.**
15. *A semaphore initialised to 1 and used by two processes gives exactly the behaviour of a mutex lock.* — **True** (ownership aside).
16. *Busy waiting happens in a blocking semaphore implementation.* — **False.**
17. *If one producer and one consumer share a buffer, the mutex `mu` is unnecessary.* — **True.**
18. *In Peterson's algorithm, a process can enter the CS even if the other process has `want = 1`.* — **True** (when `turn` is in its favour).

5.12 Quick-revision capsule

- **3 requirements:** ME, Progress, BW. Counter-example kills a solution.
- Strict alternation: ME ✔, Progress ✘, BW = 1. `want[]` only: ME ✔, deadlock. **Peterson: all three (n = 2), BW = 1.**
- TSL spinlock: ME ✔ progress ✔ **BW** ✘, busy waiting; bounded-waiting TSL algorithm: **BW = n−1**.
- Blocking lock: no busy waiting, still BW ✘.
- Semaphore: wait = down = P, signal = up = V. **S = −k ⇒ k blocked.** Binary = 0/1; counting = any.
- **Producer–consumer:** `empty = N, full = 0, mu = 1`; counting wait **before** mutex wait. **Reader–writer:** `mu`, `write`, `reader_count` — first locks, last unlocks.
- **Dining philosophers:** naive → deadlock; fix by asymmetry (odd/even), "both or none", or at most N−1 at the table.

5.13 ADDED Practice set (GATE style)

Q1. Two processes: `P1: while(turn != 1); CS; turn = 2;` `P2: while(turn != 2); CS; turn = 1;` with `turn = 1` initially. Which of ME/Progress/BW hold?

Q2. In Peterson's algorithm we interchange the first two statements of the entry section (`turn = j; want[i] = 1;`). Is ME still guaranteed?

Q3. A counting semaphore S starts at 7. 20 `wait`s and 15 `signal`s are executed (in some order). What is the final value, and how many processes are blocked at the end?

Q4. What output(s) can `P: print 'A'; signal(s); print 'D'` / `Q: wait(s); print 'B'; print 'C'` (s = 0) produce?

Q5. In the producer–consumer solution the **consumer** is changed to `wait(mu); wait(full); …` (instead of `wait(full); wait(mu);`). Is deadlock possible?

Q6. Philosophers all pick the left chopstick first, then the right (same direction), each needing 2 chopsticks. (a) How many chopsticks are needed for 5 philosophers to be certain of no deadlock? (b) With only 5 chopsticks, at most how many philosophers can be seated?

Q7. Bounded-waiting TSL algorithm with n = 6 processes: what is the maximum number of times others can enter the CS after P3 has started waiting?

Q8. Three threads each run `x = x + 1` twice with no locking (x = 0). What is the range of final values?

Q9. Semaphores `a = 1, b = 0`. `X: wait(a); print 1; signal(b)` `Y: wait(b); print 2; signal(a)` run in infinite loops. What is the output?

Q10. Which of these makes readers–writers "writer-friendly"? (a) Make `write` a counting semaphore (b) Block new readers once a writer is waiting (c) Remove `mu`.

Solutions

Q1. This is strict alternation: **ME** ✔, **Progress** ✘ (if P2 never wants the CS, P1 waits forever after its first turn), **BW** ✔ **(bound 1)**.

Q2. **No — ME can be violated** (verified): e.g., P0: `turn=1`; P1: `turn=0`; P1: `want[1]=1`, sees `want[0]=0` → enters; P0: `want[0]=1`, sees `turn=0` $\neq$ `1` → enters. Both inside. (Order matters: flag first, then turn.)

Q3. Net = 7 − 20 + 15 = **2**. Final value 2 ≥ 0 ⇒ **0 blocked** (if all 20 waits completed). (If the signals come late, some may block temporarily, but at the end the value is 2.)

Q4. `P` prints A then signals; `Q` can only print B C after the signal; P prints D concurrently with Q's B and C. **Possible outputs:** A followed by any interleaving of "D" with "BC": **ADBC, ABDC, ABCD**.

Q5. **Yes.** If the buffer is empty (full = 0), the consumer takes `mu` then blocks on `full`; the producer needs `mu` to put an item → deadlock. (Rule: counting semaphore first, mutex last — for both.)

Q6. Use the no-deadlock formula (Chapter 6): minimum resources = $\Sigma(m_i - 1) + 1$. (a) 5 philosophers, each needs 2 → 5·1 + 1 = **6 chopsticks**. (b) n·1 + 1 ≤ 5 ⇒ n ≤ 4 → **4 philosophers** (with 4 seated, by pigeonhole someone gets both).

Q7. BW = n − 1 = **5**.

Q8. Min 2, max 6 ⇒ **x** ∈ **{2, 3, 4, 5, 6}** (verified).

Q9. Only X can start (a = 1). Output: **1 2 1 2 1 2 ...** (strict alternation, ever-repeating).

Q10. **(b)** — block new readers while a writer waits (writers-preference). (a) and (c) don't help; (c) breaks correctness of `reader_count`.

CHAPTER 6

Deadlocks

GATE IMPORTANCE ★★★☆☆ *≈ 1.0 mark/paper, "Tier 2"*

◎ **GATE pattern (from your analysis):** (a) *Minimum number of resources / maximum number of processes for a deadlock-free system* (formula, 1 mark, appears often); (b) **Banker's algorithm** — "is this state safe? give a safe sequence, can this request be granted?" (2 marks); (c) *Resource-allocation graph* — "is there a deadlock?" (d) 1-mark statements — the **four necessary conditions**, prevention vs avoidance vs detection. All the arithmetic is short once you know the *method*.

6.1 What is a deadlock?

Chapter 5 already showed one: **T1 waits for T2 and T2 waits for T1 — and both wait forever.** NOTES Formally:

> A set of processes is in **deadlock** if **every** process in the set is waiting for an event (a resource release) that **only another process in the set can cause**.

Philosophers = processes, chopsticks = resources NOTES. Resources that need mutual exclusion: CPU, critical sections, printers, keyboards, OS data structures (locks).

Deadlock vs starvation

Deadlock	Starvation (indefinite blocking)
Nobody in the group can ever progress	Some process waits forever while **others progress**
Needs a cycle of waiting	e.g., low priority never scheduled
Needs ME + HW + NP + CW	Cure: aging, fair queues

6.2 The four necessary conditions (Coffman) NOTES — all four must hold together

#	Condition	Meaning (dining philosophers)
1	**Mutual exclusion**	a resource can be used by only one process at a time (a chopstick is held by one philosopher)
2	**Hold and wait**	a process **holds** some resources while **waiting** for others (holds left, waits for right)
3	**No preemption**	a resource can be released only **voluntarily** by its holder (no snatching a chopstick)
4	**Circular wait**	a cycle $P_1 \to P_2 \to \ldots \to P_n \to P_1$ where each waits for a resource held by the next (P1 holds C5, wants C1 held by P2 … P5 wants C5 held by P1)

Deadlock ⇒ ME ∧ HW ∧ NP ∧ CW. Therefore **if even one of the four is false, deadlock is impossible.** (The converse is *not* true — all four being present means deadlock is *possible*, not certain, for multi-instance resources.)

⚠ **Trap:** these conditions are **necessary, not sufficient**. "Cycle in the graph" is also necessary-only for multi-instance resources (§6.5).

NOTES **GATE statement:** *"Deadlock cannot occur in a system where every process needs at most one resource to execute."* — **TRUE**: with one resource a process never *holds one while waiting for another* ⇒ no hold-and-wait.

6.3 Strategies to handle deadlock NOTES

1. **Prevention** — design so that at least one necessary condition can **never** hold.
2. **Avoidance** — allow all four conditions but **grant a request only if** the resulting state is safe (needs advance information, e.g., Banker's).
3. **Detection and recovery** — let deadlocks happen; detect; recover.
4. **Ignorance (Ostrich algorithm)** — pretend it never happens; used by most modern OSes (UNIX, Windows) because deadlocks are rare and the other methods are expensive.

6.4 Deadlock prevention NOTES — break one condition

Break...	How	Practical?
Mutual exclusion	make resources sharable (read-only files)	Not always possible (printers, locks)
Hold and wait	a process must request **all** resources at once, or **release all held resources** before asking for more (check if it *can get* what it needs, else release everything)	Low utilisation, starvation
No preemption	the OS **takes away** resources from a waiting process	Not viable for locks/printers; OK for CPU, memory
Circular wait	impose a **total order** on resource types; a process holding R_j may request only R_k with **k > j**	**Most feasible and widely used** NOTES

ϟ **Why ordering works:** every wait goes "upward" in the numbering, so a cycle (which must come back down somewhere) is impossible. (*This is exactly what the fix "same locking order in both threads" is.*)

6.5 Resource-Allocation Graph (RAG) NOTES

- **Process** = circle. **Resource type** = rectangle, with a **dot for each instance**.
- **Request edge** $P_i \to R_j$ (P_i is waiting for an instance of R_j).
- **Assignment edge** R_j(instance dot) $\to P_i$ (an instance is held by P_i).

Reading a RAG — rules ϟ

- **No cycle ⇒ no deadlock.**
- **Cycle and every resource has a single instance ⇒ deadlock** (cycle ⇔ deadlock).
- **Cycle and some resource has several instances ⇒ deadlock *possible*, not certain** — check by **graph reduction**:

1. Find a process that is **not waiting** (all its requests can be satisfied by free instances, or it has no request edges). Remove it, give back its held instances (free them).
2. Repeat. If **all processes can be removed → no deadlock**. If some remain → those are **deadlocked**.

NOTES RAG example (cycle but NO deadlock)

R1 has 2 instances (held by **P2** and **P3**); R2 has 2 instances (held by **P1** and **P4**). P1 requests R1; P3 requests R2. There is a cycle P1 → R1 → P3 → R2 → P1, **but** P2 and P4 are not waiting for anything. They finish and release their instances → P1 gets an R1 instance and finishes → releases R2 → P3 gets R2. **No deadlock** — a cycle is necessary, not sufficient, with multi-instance resources.

Wait-for graph NOTES

A simplified RAG for **single-instance** resources: only processes as nodes. **Edge $P_i \to P_j$ means "P_i waits for a resource held by P_j"** (collapse $P_i \to$ R $\to P_j$ into one edge; e.g., P2 → R3 → P3 gives P2 → P3). **A cycle in the wait-for graph ⇔ deadlock** (single-instance case).

6.6 Deadlock avoidance — the min-resources formulas

The simplest avoidance: have **enough resources that at least one process can always obtain everything it needs and finish** (then it releases them for the others). NOTES

Formula 1. n processes share one resource type; process i needs at most m_i instances. The system is **guaranteed deadlock-free** if

$$\mathbf{R \geq \Sigma(m_i - 1) + 1} \text{ i.e. } \mathbf{R \geq \Sigma\, m_i - n + 1}$$

Why: in the worst case every process holds $m_i - 1$ (one short of finishing); we need **one more** instance so that someone can finish. The **minimum R for guaranteed freedom = $\Sigma(m_i - 1) + 1$**; with $\Sigma(m_i - 1)$ instances a deadlock is possible.

Formula 2. K instances, every process needs at most x ($x \leq K$); the **largest number of processes** for which deadlock is impossible: $\mathbf{K \geq P \cdot (x-1) + 1 \Rightarrow P_max = \lfloor (K-1)/(x-1) \rfloor}$.

Formula 3 (NOTES GATE-style form). n processes, P_i needs s_i max, m resources: deadlock-free *if* $\Sigma(s_i - 1) + 1 \leq m \Leftrightarrow \mathbf{\Sigma\, s_i < m + n}$ (option (C) in the notes).

Examples

Situation	Answer
3 processes each need max 2 tape drives	3·1 + 1 = **4** drives
4 processes each need 3	4·2 + 1 = **9**
3 processes need 3, 4, 5	2+3+4+1 = **10**
K = 10 instances, each process needs 3	$P \leq (10-1)/2 = 4.5 \to$ **4** processes
K = 7, each needs 3	$\lfloor 6/2 \rfloor$ = **3**
K = 6, each needs 2	$\lfloor 5/1 \rfloor$ = **5** safe; **N = 6** can deadlock

NOTES **Notes' example (corrected):** *"A system has 6 instances of resource R; N processes each need at most 2. For which N can a deadlock occur?"* — by Formula 2, $N \leq 5$ is **always safe** (e.g., N = 4: even if P1, P2 hold 2 each and P3, P4 hold 1, P1 and P2 finish and release). **N = 6 is the smallest value for which deadlock is possible** (each holds 1, all 6 used, each wants one more). (*The notes' option "N = 4 could deadlock" is a copying slip — see Appendix C.*)

6.7 Banker's algorithm (avoidance with multiple instances) NOTES

Needs a-priori information: each process declares in advance the **maximum number of instances of each resource type** it may ever need. When a request arrives, the OS asks: *"if I grant this, will the system still be in a **safe state**?"* If yes → grant; else the process waits.

The data (m resource types, n processes)

Name	Shape	Meaning
Available	vector of m	free instances of each resource
Max	n × m	maximum demand of P_i for R_j over its whole life
Allocation	n × m	instances currently held
Need	n × m	**Need = Max − Allocation** (what it can still ask for)

Safe state NOTES

A state is **safe** if there exists a **safe sequence** $\langle P_1, P_2, ..., P_n \rangle$ such that, for each P_i, its **future needs can be met by Available + the resources held by all P_j with j < i** (those earlier processes finish and give everything back).

- **Safe ⇒ no deadlock.**
- **Unsafe ⇒ deadlock *may* happen** (not certain).
- **Deadlock ⇒ unsafe** (the deadlocked states are a subset of unsafe states; safe and unsafe are disjoint).

The safety algorithm (what you do on paper)

1. `Work = Available`. Mark all processes "unfinished".
2. Find an unfinished P_i with **Need[i]** $\leq$ **Work** (component-wise). If none → go to 4.
3. Pretend it runs to completion: **Work = Work + Allocation[i]**; mark finished; (write i in the sequence); go to 2.
4. If **all finished** → **SAFE** (the order you wrote is a safe sequence); otherwise **UNSAFE**.

Resource-request algorithm (when P_i asks for Request):

1. If Request > Need[i] → error (it exceeded its declared maximum).
2. If Request > Available → P_i **must wait** (not enough free).
3. **Pretend to grant:** Available −= Request; Allocation[i] += Request; Need[i] −= Request. Run the safety algorithm.
4. **Safe → grant. Unsafe → undo and make P_i wait.**

NOTES Two facts: when a process **releases** resources, a safe state stays safe. When a process **requests**, the new state may or may not be safe → test it.

NOTES | Example 6.1 — one resource type, total 12 instances

P	Allocation	Max	Need
P0	5	10	5
P1	2	4	2
P2	2	9	7

Available = 12 − 9 = **3**.

- P1 (need 2 ≤ 3) can finish → Available = 3 + 2 = **5**.
- P0 (need 5 ≤ 5) → Available = 5 + 5 = **10**.
- P2 (need 7 ≤ 10) → Available = 10 + 2 = **12**. **Safe sequence: P1 → P0 → P2.** (It is the only one: after P1, P2 needs 7 > 5.)

Now P2 requests 1 more. Pretend: Allocation P2 = 3, Available = 2. Only P1 (need 2) can run → Available = 4. P0 needs 5 > 4; P2 needs 9 − 3 = 6 > 4. **No safe sequence → request NOT granted** (P2 must wait) NOTES.

NOTES | Example 6.2 — four resource types (A, B, C, D), five processes

P	Allocation (A B C D)	Max	Need = Max − Alloc
P0	3 0 1 4	5 1 1 7	**2 1 0 3**
P1	2 2 1 0	3 2 1 1	**1 0 0 1**
P2	3 1 2 1	3 3 2 1	**0 2 0 0**
P3	0 5 1 0	4 6 1 2	**4 1 0 2**
P4	4 2 1 2	6 3 2 5	**2 1 1 3**

Available = (1, 0, 0, 2).

- Who can go first? Compare each Need with (1,0,0,2): only **P1** (1,0,0,1) ✔ → Work = (1,0,0,2)+(2,2,1,0) = **(3,2,1,2)**.
- Now P2 (0,2,0,0) ✔ → Work = (3,2,1,2)+(3,1,2,1) = **(6,3,3,3)**.
- Now P3 (4,1,0,2), P0 (2,1,0,3), P4 (2,1,1,3) — all fit. Take P3 → (6,8,4,3); P0 → (9,8,5,7); P4 → (13,10,6,9).

Safe sequence: ⟨P1, P2, P3, P0, P4⟩. State is SAFE. After P1, P2 the remaining three can go in **any order**, so there are 3! = **6 safe sequences** (P1, P2 are forced first). ✔

Speed tips for Banker's questions

- Compute **Need first**, then scan the processes **top to bottom repeatedly**; each pass picks everything that fits.
- Work only increases, so *a process that fits once will always fit later* — you never need to backtrack. This also means: **to test safety it suffices to find *one* complete order**.
- **Counting safe sequences:** at each step count how many unfinished processes fit; if after some point everyone fits, multiply factorials (as above).
- If a request is "Request ≤ Need" but "> Available", the answer is "wait", not "unsafe".

NOTES | ADDED Example 6.3 — the standard textbook state (A, B, C = 10, 5, 7 instances)

P	Allocation	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available = (10,5,7) − (7,2,5) = **(3, 3, 2)**. Safety check: P1 (1,2,2) fits → (5,3,2); P3 (0,1,1) → (7,4,3); P4 (4,3,1) → (7,4,5); P0 (7,4,3) → (7,5,5); P2 (6,0,0) → (10,5,7). **Safe:** ⟨**P1, P3, P4, P0, P2**⟩ (16 safe sequences exist).
Requests (in sequence):

1. **P1 requests (1,0,2):** ≤ Need (1,2,2) ✔, ≤ Available (3,3,2) ✔ → pretend: Available (2,3,0), P1 Alloc (3,0,2), Need (0,2,0) → safety check still finds ⟨P1,P3,P4,P0,P2⟩ → **GRANTED**.
2. **P4 requests (3,3,0):** Available is now (2,3,0) → **3 > 2** → **not enough available** → **P4 must WAIT** (this is not a safety test failure).
3. **P0 requests (0,2,0)** (in the state after request 1): fits Need (7,4,3) and Available (2,3,0). Pretend → Available (2,1,0). No process has Need ≤ (2,1,0) → **unsafe** → **NOT granted**. (*Note: if P0's request were examined in the **original** state, before P1's grant, the result would be safe — the order of requests matters.*)

6.8 Deadlock detection and recovery NOTES

If we neither prevent nor avoid, the OS runs a **detection algorithm periodically (or whenever a request can't be granted)**.

- **Single instance per resource:** build the **wait-for graph**; a cycle = deadlock.
- **Multiple instances:** run the **Banker-like detection algorithm**. Use **Allocation, Available** and a **Request matrix** (Request[x][y] = instances of R_γ that P_x is *currently* waiting for). It is the safety algorithm with **Need replaced by the current Request**: a process whose Request $\leq$ Work is assumed to finish and release; processes that can never be satisfied are **deadlocked**.
- Difference from safety: detection uses *actual outstanding requests*, not *maximum possible future needs* — it is optimistic.

ADDED | Example 6.4 — detection (A, B, C; Available = (0,0,0))

P	Allocation	Request
P0	0 1 0	0 0 0
P1	2 0 0	2 0 2
P2	3 0 3	0 0 0
P3	2 1 1	1 0 0
P4	0 0 2	0 0 2

P0 (0,0,0) $\leq$ Work (0,0,0) $\rightarrow$ Work (0,1,0); P2 (0,0,0) $\rightarrow$ (3,1,3); P3 (1,0,0) $\rightarrow$ (5,2,4); P4 (0,0,2) $\rightarrow$ (5,2,6); P1 (2,0,2) $\rightarrow$ (7,2,6). All finish $\Rightarrow$ **no deadlock** (sequence P0,P2,P3,P4,P1). **Now let P2 also request one more C: Request P2 = (0,0,1).** Work (0,1,0) after P0; no other process fits (P2 needs (0,0,1), P1 needs (2,0,2), P3 needs (1,0,0), P4 needs (0,0,2)) $\Rightarrow$ **P1, P2, P3, P4 are deadlocked** (P0 is not).

Recovery NOTES

1. **Resource preemption** — take a resource from one process and give it to another (different from *prevention*: it is done **after** the deadlock occurs); pick a victim (min cost), roll it back, avoid starving the same victim.
2. **Rollback** to a **checkpoint** — restart the process from a saved safe state.
3. (Also common) **Terminate** all deadlocked processes, or one at a time until the cycle breaks.

6.9 Comparison: prevention / avoidance / detection

	Prevention	Avoidance (Banker's)	Detection + recovery
Idea	break one of 4 conditions	keep the state **safe**	allow, find, fix
Needs advance info?	no	**yes** (max demands)	no
Resource utilisation	low	medium	high
Run-time cost	low	high (safety test per request)	periodic, high
Deadlock possible?	never	never (safe state)	yes, then recovered

6.10 MSQ / statement bank

1. *If a RAG has no cycle, there is no deadlock.* — **True.**
2. *If a RAG has a cycle, deadlock has occurred.* — **False** (only for single-instance resources).
3. *An unsafe state always leads to deadlock.* — **False** ("may").
4. *A safe state can lead to deadlock.* — **False** (as long as processes obey declared maxima).
5. *Banker's algorithm needs to know each process's maximum demand in advance.* — **True.**
6. *Resource ordering prevents deadlock by breaking circular wait.* — **True.**
7. *Preempting resources is a way to break "no preemption".* — **True.**
8. *All four Coffman conditions are sufficient for deadlock with multi-instance resources.* — **False** (necessary).
9. *A system with one process can never deadlock on resource requests.* — **True** (a single process can't wait on another).
10. *Deadlock detection is part of the Banker's algorithm (safety algorithm).* — **True** (the same loop).
11. *The Ostrich algorithm is used in most modern OSes.* — **True.**
12. *In a wait-for graph with single-instance resources, a cycle ⇔ deadlock.* — **True.**
13. *Deadlock prevention may reduce resource utilisation.* — **True.**
14. *Starvation and deadlock are the same.* — **False.**
15. *If every process needs at most one resource to execute, deadlock cannot occur.* — **True** (NOTES).

6.11 Quick-revision capsule

- **4 conditions: ME, Hold & Wait, No Preemption, Circular Wait** — all necessary.
- Prevention: break one (ordering of resources = easiest). Avoidance: Banker's, safe state. Detection: wait-for graph / detection matrix. Ostrich.
- **Min resources = $\Sigma(m_i - 1) + 1$.** Max processes = $\lfloor (K-1)/(x-1) \rfloor$.
- RAG: single instance → cycle = deadlock; multi → reduce the graph.
- Banker's: **Need = Max − Alloc**; safe sequence by repeated "Need ≤ Work → Work += Alloc". **Request ≤ Need, ≤ Available, then test safety.**
- Safe ⇒ no deadlock; deadlock ⇒ unsafe; unsafe ⇏ deadlock.

6.12 ADDED Practice set

Q1. 4 processes each need at most 3 units of one resource. Minimum units that guarantee no deadlock? What if the maximum demands are 2, 3, 4, 5?

Q2. 8 identical units; each process needs at most 3. Max number of processes for which the system is deadlock-free?

Q3. One resource, total 10 units. Allocation/Max: P0 3/9, P1 3/4, P2 2/7. (a) Is the state safe? Give a sequence. (b) Can P0's request for 1 more be granted? (c) Can P2's request for 1 more be granted?

Q4. In Example 6.2, how many safe sequences exist? Could the system be safe if Available were (1,0,0,1)?

Q5. (Detection) Allocation: P0 (0,1,0), P1 (2,0,0), P2 (3,0,3), P3 (2,1,1), P4 (0,0,2); Available (0,0,0); Requests: P0 (0,0,0), P1 (2,0,2), P2 (0,0,1), P3 (1,0,0), P4 (0,0,2). Which processes are deadlocked?

Q6. True/False: (a) A process in an unsafe state is deadlocked. (b) With 6 resource instances and 5 processes each needing at most 2, deadlock is impossible. (c) With 6 instances and 6 such processes it is possible.

Q7. Three threads each acquire two locks A and B: T1 A→B, T2 B→A, T3 A→B. Can deadlock occur? Give a fix.

Q8. RAG: P1 holds R1, requests R2; P2 holds R2, requests R3; P3 holds R3, requests R1; all single instance. Deadlock?

Solutions

Q1. $4\cdot(3-1)+1 = \mathbf{9}$. For demands 2,3,4,5: $(1+2+3+4)+1 = \mathbf{11}$.

Q2. $P\cdot(3-1)+1 \le 8 \rightarrow P \le 3.5 \rightarrow \mathbf{3}$.

Q3. Available $= 10 - 8 = 2$; Need = 6, 1, 5. (a) P1 $(1 \le 2) \rightarrow 5$; P2 $(5 \le 5) \rightarrow 7$; P0 $(6 \le 7)$ ✔ → **safe: P1, P2, P0**. (b) P0 +1: Available 1, P0 Alloc 4, Need 5: P1 $(1 \le 1) \rightarrow 4$; P2 needs $5 > 4$, P0 needs $5 > 4$ → **unsafe** → **not granted**. (c) P2 +1: Available 1, P2 Alloc 3, Need 4: P1 → 4; P2 $(4 \le 4) \rightarrow 7$; P0 $(6 \le 7)$ → **safe** → **granted**.

Q4. **6** safe sequences. With Available (1,0,0,1): P1 needs D = 1 ✔ fits (1,0,0,1)! → Work (3,2,1,1); P2 (0,2,0,0) ✔ → (6,3,3,2); P3 (4,1,0,2): D $2 \le 2$ ✔ → (6,8,4,2); P0 (2,1,0,3): D $3 > 2$ ✘; P4 (2,1,1,3) ✘ D $3 > 2$. After P3 nothing more (P0, P4 need D = 3) → try P0/P4 first: also $D = 3 > 2$. **Unsafe.** (One unit of D decides everything.)

Q5. Work (0,0,0): P0 finishes → (0,1,0). Nobody else fits (P2 needs C = 1, P1 needs (2,0,2), P3 needs (1,0,0), P4 needs (0,0,2)). **Deadlocked: P1, P2, P3, P4.**

Q6. (a) **False.** (b) **True** $(5\cdot1 + 1 = 6 \le 6)$. (c) **True** $(6\cdot1 + 1 = 7 > 6)$.

Q7. **Yes** — T1 takes A, T2 takes B, then each waits for the other's lock. Fix: **impose a global order** (always A before B) in all three threads (breaks circular wait).

Q8. Single instances, cycle P1 → R2 → P2 → R3 → P3 → R1 → P1 ⇒ **deadlock**.

PART IV

Memory

CHAPTER 7

Memory Management Basics: Address Spaces, Protection, Contiguous Allocation

GATE IMPORTANCE ★☆☆☆☆ — *≈ 0.2 marks/paper — short chapter; its ideas feed the paging chapters*

◎ **GATE pattern:** *Memory allocation* (first/best/worst fit, fragmentation) is "Tier 4" — it appears rarely, as a 1-mark placement question or a statement about fragmentation. **Spend little time here, but read §7.1–7.3 carefully: logical vs physical address, relocation, and "why do we need paging" are the foundation of Chapters 8 and 9.**

7.1 What is an address space?

NOTES The **address space of a process** = the **range of addresses [start, end]** it is allowed to access; its size is `end − start + 1`. It contains the **text (code), data, heap and stack** areas.

Why restrict a process to its own range? *Security and safety.* The CPU's fetch–decode–execute cycle must not jump into another program's instructions or read its data. (Hardware recap: the CPU talks to memory through an **address bus** (which location), a **data bus** (the value) and a **read/write control line**.)

Logical address NOTES = the address **generated by the CPU** while process P runs. **Logical address space** = the range of addresses the CPU may generate for P.

Physical address NOTES = the address **seen by the memory unit** (what actually goes on the address bus).

> A user program only ever deals with **logical** addresses; it never sees physical addresses.

7.2 The MMU and the base–limit scheme NOTES

A program could generate an address outside its range (a bug, or a hack). Checking this by **software** before every memory access would be far too slow ⇒ the check is done by hardware: the **Memory Management Unit (MMU)**.

Simplest MMU design — base and limit registers:

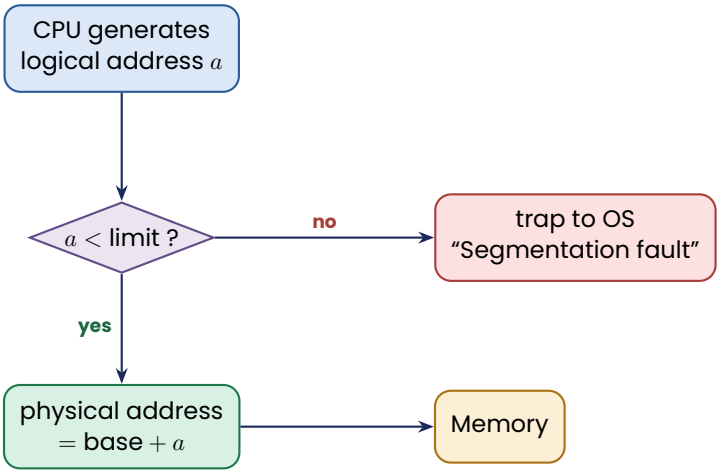


Figure 7.1. Base–limit protection: every logical address is checked against the limit register.

- **base** = starting physical address of the process; **limit** = size of its address space.
- The process's *logical* space is **[0, limit−1]**; its *physical* space is **[base, base+limit−1]**.
- The OS decides `base` when it creates the process. For the OS itself, base = 0 and limit = total memory.
- The **MMU registers are part of the process context**: on a **context switch** the OS **loads the new process's base/limit** (and saves the old one's in its PCB). Loading these registers is a **privileged** instruction.
- An illegal access raises an interrupt/exception; the OS handles it (usually kills the process: **segmentation fault**).

Relocatable code NOTES

At **compile time the compiler does not know where in memory the process will be placed** (e.g. `int x = 0;` compiles to `STORE a, 0` — what is `a`?). Solution: the compiler always generates addresses in the fixed logical range **[0, limit−1]**; the hardware adds the base at run time: **a' = base + a**. Such code is called **relocatable**. It can be moved anywhere in memory (even while the process is not running) by just changing `base`.

> ϟ **Quick numeric template:** *base = 3000, limit = 500.* Logical 100 → **3100**; 499 → **3499**; **500** → **trap** (500 ≥ limit); 600 → trap.

7.3 Contiguous memory allocation NOTES

With **multiprogramming** many processes live in memory at once. In the simplest scheme each process gets **one contiguous block** (from `base` to `base + limit`). The OS maintains a list of **holes** (free blocks). (The size is given when the process is created, e.g., via `fork`; if no hole is big enough, `fork` returns a negative value — failure.)

Placement algorithms

Algorithm	Rule	Remarks
First fit	scan the hole list from the beginning; take the **first** hole that is big enough	fast; works well in practice
Best fit	take the **smallest** hole that is big enough (scan the whole list)	leaves tiny useless leftovers
Worst fit	take the **largest** hole	leftover is big (maybe usable) but kills big holes
(Next fit ADDED)	like first fit but continues from where the last search ended	

⚡ **How to do a placement question:** keep a list of holes; for each request in the *given order* choose a hole by the rule, **subtract the request from that hole** (the remainder stays as a smaller hole), and continue.

ADDED | Example 7.1 — placement

Holes (in address order): **100, 500, 200, 300, 600 KB**. Requests in order: **212, 417, 112, 426 KB**.

Request	First fit	Best fit	Worst fit
212	500 → leaves 288	300 → leaves 88	600 → leaves 388
417	600 → leaves 183	500 → leaves 83	500 → leaves 83
112	288 → leaves 176	200 → leaves 88	388 → leaves 276
426	**no hole ≥ 426 → cannot be placed**	600 ✔	**cannot be placed**

Best fit places all four; first fit and worst fit fail on 426. (Not a general rule — best fit is not always best; it just happens here.)

Fragmentation NOTES

- **External fragmentation:** total free memory is enough for a request, but it is **split into holes that are each too small**. The wasted memory is **outside** every process. (First-fit/best-fit/worst-fit all suffer.)
- **Internal fragmentation:** memory **inside** an allocated block that the process does not use (e.g., a 9 KB process put in a 10 KB chunk wastes 1 KB; nobody else can use that 1 KB).

Solution 1 — Compaction NOTES: slide all processes to one end so the holes merge into one big hole. Since the code is **relocatable**, only the `base` register of each moved process needs updating. ⊗ But copying lots of memory is **extremely expensive**.

Solution 2 — Fixed-size chunks (partitions) NOTES: divide memory into pre-defined chunks, each large enough for some programs. Example in the notes: chunks of 0.5 KB, 10 KB, 7 KB, 3 KB, 2 KB — a 10 KB process goes into the 10 KB chunk; a 9 KB process uses chunks 3+4 (= 10 KB) and wastes 1 KB inside → **internal fragmentation**. Fixed partitions remove external fragmentation but create internal fragmentation.

Solution 3 — Paging NOTES: divide memory into **equal-size chunks (frames)** and let a process live in **non-contiguous** frames. → Chapter 8. *Idea:* three free chunks that are not adjacent can still hold a 3-chunk process if the process itself is cut into 3 pieces (pages).

	External fragmentation	Internal fragmentation
Variable partitions (contiguous)	**Yes**	negligible
Fixed partitions	No	**Yes**
Paging	**No**	**Yes**, only in the **last page** of each process (avg. half a page)
Segmentation (pure)	**Yes**	No

7.4 MSQ / statement bank

1. *The MMU performs the limit check in hardware on every access.* — **True.**
2. *The base and limit registers can be loaded by user programs.* — **False** (privileged).
3. *With relocatable code a process can be moved in memory by changing only its base register.* — **True.**
4. *Best fit always gives the least fragmentation.* — **False.**
5. *Compaction is possible only if the code is relocatable at run time.* — **True.**
6. *Paging removes external fragmentation.* — **True.**
7. *A logical address is the one generated by the CPU; a physical address is the one seen by the memory.* — **True.**
8. *A page-table-less base/limit scheme allows a process to be non-contiguous in memory.* — **False.**

7.5 Quick-revision capsule

- Logical = CPU-generated; physical = memory-seen; MMU does `if a < limit then base + a else trap`.
- Relocatable code: compile for [0, limit−1]; relocate by changing `base`.
- First/Best/Worst fit; external vs internal fragmentation; compaction (expensive); paging cures external fragmentation.

7.6 ADDED Practice

Q1. base = 1200, limit = 800. Which of the logical addresses 0, 799, 800, 1500 are legal, and what are their physical addresses? **Q2.** Holes: 150, 350, 100, 600, 200 KB. Place 300, 120, 250, 400 with (a) first fit, (b) best fit, (c) worst fit. Which requests fail? **Q3.** 10 processes live in a system with fixed 64 KB partitions, each using on average 50 KB. How much memory is lost to internal fragmentation?

Solutions

Q1. 0 → **1200**; 799 → **1999**; 800 → **trap**; 1500 → **trap**. **Q2.** (a) First fit: 300→350 (leaves 50); 120→150 (leaves 30); 250→600 (leaves 350); 400→ holes now 30,50,100,350,200: none ≥ 400 → **fails**. (b) Best fit: 300→350 (50); 120→150 (30); 250→600 (350); 400→ **fails** (largest hole 350). (c) Worst fit: 300→600 (300 left); 120→350 (230); 250→300 (50); 400→ holes 150,230,100,50,200 → **fails**. Total free is 1400 KB but only 1070 requested — still 400 fails: *external fragmentation*. **Q3.** $10 \times (64 - 50) = \mathbf{140\ KB}$.

CHAPTER 8

Paging, TLB, Multilevel Paging, Segmentation

GATE IMPORTANCE ★★★★★ *the top-scoring cluster: "Virtual memory & paging" ≈ 2.4 marks/paper*

◎ **GATE pattern (from your analysis):** a *numerical* "address-translation geometry" question almost every year:

- *How many bits in the physical/virtual address, page number, offset, PTE?*
- *Size of the page table; number of levels needed; size of the top-level table.*
- *Effective memory access time (EAT) with a TLB / with multilevel paging.*
- 1-mark statements on TLB, ASID, inverted page tables, segmentation, dynamic linking.

This is a formula chapter. Learn the 12 formulas of §8.3 and the "level-counting recipe" of §8.7 and you can solve almost every question in under 2 minutes.

8.1 Why paging? (continuing Chapter 7)

Contiguous allocation suffers from **external fragmentation**. Paging NOTES divides:

- **physical memory** into equal-size chunks called **frames**,
- **a process's logical address space** into chunks of the **same size** called **pages**,

and puts **any page into any free frame**. A process of size L bytes needs $\lceil \mathbf{L} / \mathbf{K} \rceil$ **frames** (K = page size = frame size); only the **last page** may be partly empty ⇒ **internal fragmentation only in the last page**.

NOTES *Example.* Memory split into 256 KB chunks. P1 = 512 KB → chunks 0, 1. P2 = 420 KB → chunks 2, 3 with 512 − 420 = **92 KB unused inside chunk 3** (internal fragmentation). Three free chunks that are **not adjacent** can now hold a 3-page process.

8.2 How an address is translated NOTES

Logical address = page number (p) | offset (d). Physical address = frame number (f) | offset (d) — the **offset is unchanged**. The **page table (PT)** — one per process, kept in memory (its location in the PCB / a **page-table base register**) — maps *page number* → *frame number*. The translation is done by the **MMU in hardware**.

For a byte at logical address a (page size K):

- **page number** j = ⌊a / K⌋, **offset** d = a mod K
- **PA = f_j · K + d**, where f_j = PT[j]. NOTES (In the notes: "PA(a) = (a − j·K) + Start(f_j)".)

NOTES *Mini-example:* 5 frames f0…f4 of 256 B; process P has page p0 → frame 3 and p1 → frame 1. Logical 100 (page 0, offset 100) → 3·256 + 100 = **868**. Logical 300 (page 1, offset 44) → 1·256 + 44 = **300**.

NOTES Worked example 8.1 — the standard geometry problem

*Memory 4 GB, **page = 4 KB**, **word-addressable with 2-byte words**, process size 8 MB. Find: physical address bits, logical address bits, page-table size.*

1. Memory = 2^{32} B = 2^{31} words. Page = 2^{12} B = 2^{11} **words**.
2. **#frames** = 2^{31} / 2^{11} = 2^{20} → physical address = **20 (frame) + 11 (offset) = 31 bits**.
3. Process = 8 MB = 2^{23} B = 2^{22} words → **#pages** = 2^{22} / 2^{11} = 2^{11} → logical address = **11 (page) + 11 (offset) = 22 bits**.
4. PT entry must hold a frame number = **20 bits**; #entries = #pages = 2^{11} → **PT size = 2^{11} × 20 bits = 40 Kb (= 5 KB)**.

> ⚠ **Always check "byte-addressable" or "word-addressable"** — it changes the offset bits and the number of address bits.

8.3 ⚡ The formula sheet (memorise)

Let **VAS** = virtual address space size, **PAS** = physical address space size, **P** = page size, **e** = PTE size (bytes).

Quantity	Formula
Offset bits	$\log_2$ P
Virtual address bits	$\log_2$ VAS
Physical address bits	$\log_2$ PAS
#pages (virtual)	VAS / P = 2^(VA bits − offset bits)
#frames (physical)	PAS / P = 2^(PA bits − offset bits)
Bits in a frame number	PA bits − offset bits
PTE size (min)	frame-number bits + control bits (valid, dirty, reference, R/W/X protection) — usually rounded up to bytes
Single-level PT size	#pages × e
PTEs per page	P / e
Pages occupied by the PT	(#pages × e) / P
Internal fragmentation per process (avg)	**P / 2** (last page)
Frames needed for a process of L bytes	⌈L / P⌉
TLB reach	#TLB entries × P
Inverted PT size	#frames × (entry size)

> ⚡ If the question says *"the PTE contains only the frame number"*, e = frame bits (not rounded) unless told "rounded to bytes".

8.4 What is inside a page-table entry?

Field	Purpose
Frame number	where the page is
Valid/Invalid bit NOTES	*valid* = the page belongs to the process **and is in memory**; *invalid* = not in memory **or** illegal address. Only valid pages get frames ⇒ **a process can have a large logical space but few used pages**
Protection (R / W / X)	read-only code, etc. → violation raises an exception
Dirty (modified) bit	page written since loaded? — if dirty it must be written back to disk on replacement (Ch 9)
Reference bit	accessed recently? — used by LRU approximations / clock (Ch 9)

The **first few PTEs of every process** point to the **OS frames** (marked as accessible only in privileged/kernel mode) — so when a system call switches the mode bit to kernel, the CPU can still reach OS code through the *same* page table. NOTES

8.5 TLB — Translation Lookaside Buffer NOTES

Problem: every memory reference needs **two memory accesses** — one for the page-table entry, one for the data ⇒ the machine runs at **50 % speed**. **Solution:** a small, fast **cache of recent page-table entries**, the **TLB**, inside the CPU/MMU. It stores (page number → frame number) pairs and is searched in parallel/associatively.

- **TLB hit:** frame found in TLB → only **1 memory access**.
- **TLB miss:** read the PT from memory (extra access), then (usually) load the entry into the TLB.
- Each **process has its own page table**, so on a **context switch** the TLB must be **flushed** (otherwise the new process would use the old process's translations) — **OR** the TLB entries carry an **ASID (address-space identifier = PID)** so entries of several processes can **coexist** and no flush is needed. NOTES

⚡ Effective Memory Access Time (EAT) — the formulas

Let **m** = memory access time, **t** = TLB lookup time, **h** = TLB hit ratio, **k** = number of page-table levels.

Case	EAT
No TLB, 1-level paging	**2m**
No TLB, k-level paging	**(k+1)·m**
TLB, 1-level (lookup first, then memory)	h·(t + m) + (1−h)·(t + m + m) = **t + (2 − h)·m**
TLB, k-level	h·(t + m) + (1−h)·(t + k·m + m) = **t + m + (1−h)·k·m**
TLB lookup time **ignored** (t = 0)	1-level: (2 − h)·m; k-level: m + (1−h)·k·m
TLB lookup **in parallel** with PT access (rare, only if stated)	h·m + (1−h)·2m

Numerical check (t = 10 ns, m = 100 ns): | h | EAT (1-level) = 10 + (2−h)·100 | t ignored | |—|—|—| | 0.9 | **120 ns** | 110 ns | | 0.8 | **130 ns** | 120 ns | | 2-level paging, h = 0.9 | 10 + 100 + 0.1·2·100 = **130 ns** | |

> ⚡ **To find h from EAT:** solve the linear equation (e.g., t = 20, m = 100, target EAT 130 → 20 + (2−h)·100 = 130 → **h = 0.9**).

TLB reach = (#entries) × (page size) = the memory the TLB can map at once. 64 entries × 4 KB = **256 KB**. Larger pages (or more entries) increase the reach.

> ⚠ **Read the wording:** "TLB access time is negligible" ⇒ t = 0. "Page table is in memory and TLB lookup takes 20 ns" ⇒ include t. "Each level of page table access takes m" ⇒ k·m.

8.6 Sharing pages and dynamic linking NOTES

- **Static linking:** library code is **copied into every executable** → n processes × k library pages = **n·k frames** wasted.
- **Dynamic linking / shared libraries:** the library's frames are loaded **once**; the **PTEs of different processes point to the same frames** → only **k frames**. Needs compiler support (stub code) and an OS module (the dynamic linker) that finds/loads library pages. Shared pages are normally **read-only**.

8.7 Multilevel (hierarchical) paging NOTES — the highest-yield part of the chapter

Why? Two reasons NOTES

1. **The page table may be too big to sit in one contiguous block** (hardware keeps only its *start address* in a register and assumes the PT is contiguous). Solution: **page the page table itself** — repeat until the top-level table fits in **one frame**.
2. **Sparse address spaces:** a 2^{10}-page logical space may use only 2^8 pages (stack at the top, text/data at the bottom, a big invalid gap). Lower-level tables for the gap are **never allocated**; their entry in the upper level is marked invalid → **saves memory**.

The level-counting recipe (use this first — it answers "how many levels?")

1. Offset bits = $\log_2$ P. **Page-number bits** = VA bits − offset bits.
2. **One page can hold P/e entries** ⇒ each level **indexes** $\log_2(\mathbf{P/e})$ **bits**.
3. **#levels =** $\lceil$ **page-number bits** / $\log_2(\mathbf{P/e})$ $\rceil$**.** The **top level gets the remaining (fewer) bits**, and (to fit in one frame) never needs more than $\log_2(P/e)$ bits.

VA	Page	PTE	page-no bits	bits/level	Levels	Split
32 bit	4 KB	4 B	20	10	**2**	10 | 10 | 12
48 bit	4 KB	8 B	36	9	**4**	9 | 9 | 9 | 9 | 12
64 bit	4 KB	8 B	52	9	**6** (5.78→6)	7 | 9×5 | 12
33 bit	4 KB	2 B	21	11	**2**	10 | 11 | 12

NOTES Worked example 8.2 — PAS 256 MB, VAS 8 GB, page 4 KB

- PA = 28 bits, VA = **33 bits**, offset = 12 bits.
- #frames = 2^{16}, #pages = 2^{21}. PTE must hold a 16-bit frame number → **2 B**.
- Flat PT = 2^{21} × 2 B = **4 MB** — too big for a 4 KB frame ⇒ **page the PT**: 4 MB / 4 KB = 2^{10} pages of PT; each PT page holds 4 KB / 2 B = 2^{11} **PTEs**.
- The upper-level table has 2^{10} entries × 2 B = **2 KB** → fits in a frame ⇒ **2 levels**.
- **VA = 10 | 11 | 12** (top index | second-level index | page offset).
- *Walk:* the MMU register holds the **start of the top-level table** → index X selects an entry = frame of a second-level table → index Y selects an entry = **frame of the data page** → add offset Z.

NOTES Worked example 8.3 (A) — 64-bit address, page 1 MB, PTE 16 B

- offset = 20 bits; #pages = $2^{64}/2^{20} = 2^{44}$. Flat PT = $2^{44} \times 2^4 = 2^{48}$ B ≫ page.
- PTEs per page = $2^{20}/2^4 = 2^{16}$. Level-1 table needs $2^{44}/2^{16} = 2^{28}$ entries = 2^{32} B ≫ page → page again.
- Level-2 table: $2^{28}/2^{16} = 2^{12}$ entries × 16 B = 2^{16} B = **64 KB < 1 MB** → fits.

- **VA = 12 | 16 | 16 | 20** (= 64). → **3 levels**.

Variant (design choice, NOTES): *fill each page of the next table with only* 2^{10} *entries.* Then level-2 table = $2^{28}/2^{10}$ = 2^{18} entries × 16 B = 2^{22} B > 1 MB → page again with 2^{12} per page → level-3 table = $2^{18}/2^{12} = 2^6$ entries × 16 B = **1 KB**. **VA = 6 | 12 | 10 | 16 | 20 = 64** (4 levels + offset). (*The notes' "*2^{23}*" in this derivation is a slip for* 2^{18} ⇒ *Appendix C.*)

NOTES Worked example 8.4 (B) — "how many bits in the PTEs of L1, L2, L3?"

PAS = 36 bits, VAS = 32 bits, PTE = 4 B, page 4 KB; 3-level paging with VA = 3 | 8 | 9 | 12 (L1 | L2 | L3 | offset).

- **L3 PTE** points to a **data page frame**: #frames = $2^{36}/2^{12} = 2^{24}$ → **24 bits**.
- An **L3 table** has 2^9 entries × 4 B = 2^{11} B = 2 KB. **L2 PTE** points to an L3 table: positions of 2 KB tables in memory = $2^{36}/2^{11} = 2^{25}$ → **25 bits**.
- An **L2 table** has 2^8 × 4 B = 2^{10} B = 1 KB. **L1 PTE** points to an L2 table: $2^{36}/2^{10} = 2^{26}$ → **26 bits**.
- **Answer: L1 = 26, L2 = 25, L3 = 24 bits.**

> ϟ **Rule:** *a PTE must be able to address the thing it points to.* #bits = $\log_2$(PAS / size of the pointed-to object). Page tables need not be page-sized — "memory is not physically partitioned into fixed frames; it is logically partitioned into chunks of whatever size each user needs" NOTES (like `char` = 1 B, `int` = 4 B).

NOTES Worked example 8.5 (C) — chunk sizes at each level

PAS 32 bits, VAS 38 bits, page 2^{16} *B, VA = 4 | 10 | 8 | 16; the PTE holds only the address of the next-level chunk.*

- Data pages: **chunk =** 2^{16} **B**, #such chunks in memory = $2^{32}/2^{16} = 2^{16}$ → **L3 PTE = 16 bits**.
- **L3 chunk** = 2^8 entries × 16 bits = 2^9 B = **512 B**; #chunks = $2^{32}/2^9 = 2^{23}$ → **L2 PTE = 23 bits**.
- **L2 chunk** = 2^{10} entries × 23 bits = 23 552 bits = **2944 B**; #chunks ≈ $2^{32}/2944 \approx 1.46 \times 10^6$ → needs $\lceil \log_2 \rceil$ = **21 bits** (the notebook rounds to 20).
- **L1 chunk** = 2^4 entries × 21 bits = 336 bits = **42 B** (notebook: 40 B with 20 bits). (*Such "ugly" numbers are unlikely in GATE; the method is what matters. See Appendix C.*)

NOTES Worked example 8.6 (D) — PT size of a *given process* (the "count the chunks" technique)

32-bit machine, VA = 10 | 8 | 6 | 8 (L1 | L2 | L3 | offset), 3-level paging, PTE = 2 B at every level.

- **(1) Page size** = 2^8 words.
- **(2) A process with 256 K words** (2^{18}) **starting at address 0:** pages = $2^{18}/2^8 = 2^{10}$ **= 1024**; each L3 chunk holds 2^6 = 64 PTEs → **L3 chunks = 1024/64 = 16**; one L2 chunk holds 2^8 = 256 entries → L2 chunks = $\lceil 16/256 \rceil$ = **1**; L1 chunks = **1**. **PTEs stored** = 16 × 2^6 + 1 × 2^8 + 1 × 2^{10} = 1024 + 256 + 1024 = **2304** → **PT size = 2304 × 2 B = 4608 B.** (*Chunks that exist are stored in full — even if only 16 of the 256 L2 entries are used.*)
- **(3) A process with three segments** (page = 256 words, PTE 2 B): | Segment | Size | Start | Pages | |—|—|—|—| | code | 48 K | 0x01000000 | 192 | | data | 600 K | 0x80000000 | 2400 | | stack (grows up) | 64 K | 0xF0000000 | 256 |
- **L3 chunks** (64 PTEs each): code $\lceil 192/64 \rceil$ = 3; data $\lceil 2400/64 \rceil$ = 38; stack 256/64 = 4 → **45**.
- The top 10 bits (L1 index) of the three starts are **4, 512 and 960** — all different — so **3 separate L2 chunks**; **1 L1 chunk**.
- **PTEs** = 45 × 2^6 + 3 × 2^8 + 1 × 2^{10} = 2880 + 768 + 1024 = **4672** → **9344 B.**
- (Computing each segment separately and adding would **double-count** the shared L1 chunk — the notebook crossed this out.)

⚡ **"Count the chunks" recipe for sparse processes:** (i) pages used → (ii) L3 chunks = Σ ⌈pages per region / entries-per-L3-chunk⌉ → (iii) L2 chunks = number of *distinct* L2-chunk regions touched → (iv) L1 chunks (usually 1) → total entries = Σ chunks × entries-per-chunk → × PTE size. **Alignment matters:** check whether segments start on chunk boundaries.

Memory cost of translation (⚠)

With k-level paging a TLB miss costs **k + 1** memory accesses; that is why a good TLB hit ratio is essential.

8.8 Inverted page table NOTES

Problem: a flat PT must cover the **whole virtual address space** for each process — even a small process's PT can be bigger than the process. **Inverted PT: one entry per physical FRAME**, storing ⟨**PID, page number**⟩ of the page currently in it. The system has **one** such table (not one per process).

- ✔ **Space** ∝ physical memory (#frames × entry size) — independent of the number of processes and of VAS size.
- ⊗ **Slower lookup:** to translate (pid, page) the table must be **searched** (usually through a hash) instead of an O(1) index; the **frame number is the index of the matching entry**. Sharing pages between processes is harder.

⚡ Size = #frames × (bits for PID + page number + control bits). E.g., 64 MB memory, 4 KB pages → 2^{14} frames; with 8 B entries → **128 KB**.

8.9 Segmentation NOTES

A program's logical space is naturally a set of **segments** (text, data, stack, heap, libraries). **Logical address = (segment number s, offset d)** — "give me the d-th byte of the s-th segment" — which is convenient for the **compiler**. **Segment table:** ST[s] = (**base**, **limit**). Translation: `if d < limit[s] then PA = base[s] + d else trap`.

ADDED | Example 8.7 — segment translation

Segment table: S0 (base 219, limit 600); S1 (2300, 14); S2 (90, 100); S3 (1327, 580); S4 (1952, 96). | Logical (s, d) | Result | |—|—| | (2, 53) | 53 < 100 → 90 + 53 = **143** | | (3, 852) | 852 > 580 → **trap** | | (0, 430) | 219 + 430 = **649** | | (1, 10) | 2300 + 10 = **2310** | | (1, 20) | 20 > 14 → **trap** |
Pure segmentation: variable-size segments ⇒ **external fragmentation** (no internal), but supports sharing/protection per segment naturally.

Segmented paging NOTES

Combine both: **logical address = segment no | page no | page offset. Each segment is paged and has its own page table**; the segment-table entry holds the **base of that segment's PT** (and its length in pages). Translation: segment table → that segment's page table → frame → + offset. Result: **no external fragmentation** + the segment view. (*e.g., 4 + 8 + 12 = 24-bit address: 16 segments, each up to 256 pages of 4 KB.*)

	Paging	Segmentation
Unit size	fixed (invisible to programmer)	variable (visible: code, stack...)
External fragmentation	none	yes
Internal fragmentation	yes (last page)	none
Address	page no, offset	segment no, offset
Table	page table	segment table (base + limit)

8.10 ADDED Optimal page size

Too small a page → huge page table; too large a page → more internal fragmentation. For an average process size **s** bytes and PTE size **e** bytes, overhead per process ≈ **s·e / p** (page table) **+ p / 2** (internal fragmentation). Minimising gives

p_opt = √(2·s·e)

Example: s = 1 MB (2^{20}), e = 8 B (2^3): p = √(2^{24}) = 2^{12} = **4 KB**. (If s = 4 MB, e = 8 B: √(2^{26}) = 2^{13} = **8 KB**.)

8.11 MSQ / statement bank

1. *Internal fragmentation occurs in paging only in the last page of a process.* — **True.**
2. *Page size must be a power of 2.* — **True** (so that offset/page-number split is a bit-slice).
3. *TLB must be flushed on every context switch in all systems.* — **False** (not with ASIDs).
4. *Inverted page tables reduce memory for page tables but increase lookup time.* — **True.**
5. *A larger page size always reduces the number of page faults.* — **False.**
6. *Increasing the number of levels reduces the page-table memory for sparse spaces but increases the cost of a TLB miss.* — **True.**
7. *In a k-level page table, a TLB miss needs k extra memory accesses.* — **True** (k+1 total with the data).
8. *Valid bit = 0 in a PTE always means the page is not in memory.* — **False** (also illegal pages).
9. *The offset is not translated.* — **True.**
10. *Dynamic linking lets many processes share library frames.* — **True.**
11. *Physical memory addresses can exceed the virtual address space.* — **True** (e.g., 36-bit PAS with a 32-bit VAS).
12. *Segmentation can lead to external fragmentation.* — **True.**
13. *Page table for a process must be stored contiguously in memory (single-level).* — **True** → motivates multilevel paging.
14. *The TLB reach increases if the page size increases.* — **True.**
15. *The memory-access protection check uses the PTE protection bits.* — **True.**

8.12 Quick-revision capsule

- Logical = (page, offset); physical = (frame, offset). #pages = VAS/P, #frames = PAS/P, PT = #pages × e.
- TLB: hit → 1 access. EAT = **t + (2−h)m** (1-level); **t + m + (1−h)·k·m** (k-level).
- Levels = ⌈(VA bits − offset) / $\log_2$(P/e)⌉. PTE bits in a pointer = $\log_2$(PAS / size of the pointed-to object).
- Sparse processes: count chunks per level; chunks that exist count fully.
- Inverted PT: one entry per frame (small, slow). Segmentation: (s, d), base+limit, external fragmentation.
- p_opt = √(2se).

8.13 ADDED Practice set (GATE style)

Q1. 32-bit VA, 4 KB pages, 4 B PTE. (a) single-level PT size? (b) split of a 2-level scheme? (c) size of the top-level table?

Q2. PAS = 40 bits, page = 8 KB; the PTE holds a frame number + 3 control bits (valid, dirty, R/W) and is rounded up to whole bytes. (a) PTE size? (b) For a 48-bit VA, flat PT size? (c) How many levels are needed?

Q3. t = 20 ns, m = 100 ns, h = 0.95. Find EAT for (a) 1-level paging (b) 3-level paging. (c) What h gives EAT = 130 ns for 1-level?

Q4. TLB with 64 entries, page 4 KB. TLB reach? If pages become 2 MB?

Q5. Average process size 4 MB, PTE 8 B. Optimal page size?

Q6. Using the segment table of Example 8.7, translate (4, 95), (4, 96), (2, 100), (0, 0).

Q7. 64 MB physical memory, 4 KB pages, inverted PT with 8 B entries — size? How big would a flat per-process PT be if VA is 32 bits and PTE 4 B?

Q8. VA = 10 | 10 | 12, PTE 4 B. A process uses 3000 contiguous pages starting at page 0. What is the page-table memory used (top table + second-level tables)?

Q9. A system has 36-bit PAS, 4 KB pages and 3-level paging with VA = 6 | 9 | 9 | 12 (PTE 4 B). How many bits are needed for the PTE of the last level if it also has 4 control bits?

Q10. Process of 1050 KB, page = 256 KB: frames needed and internal fragmentation?

Q11. *True/False:* in a system with ASIDs, two processes can have TLB entries for the same virtual page number at the same time.

Q12. TLB hit 98 %, TLB lookup 10 ns, memory 80 ns, 2-level paging. EAT?

Solutions

Q1. (a) $2^{20} \times 4$ B = **4 MB**. (b) page-number bits 20; one page holds $4096/4 = 1024 = 2^{10}$ PTEs $\Rightarrow$ **10 | 10 | 12**. (c) top table = $2^{10} \times 4$ B = **4 KB** (one frame).

Q2. (a) frames = $2^{40}/2^{13} = 2^{27} \rightarrow 27 + 3 =$ **30 bits** $\rightarrow$ **4 B**. (b) pages = 2^{35}; PT = $2^{35} \times 4 = 2^{37}$ B = **128 GB**. (c) PTEs per page = $2^{13}/4 = 2^{11} \rightarrow 11$ bits per level; $\lceil 35/11 \rceil =$ **4 levels**.

Q3. (a) $20 + (2 - 0.95) \times 100 = 20 + 105 =$ **125 ns**. (b) $20 + 100 + 0.05 \times 3 \times 100 =$ **135 ns**. (c) $20 + (2 - h)100 = 130 \Rightarrow$ **h = 0.9**.

Q4. 64 × 4 KB = **256 KB**; with 2 MB pages 64 × 2 MB = **128 MB**.

Q5. $\sqrt{(2 \cdot 2^{22} \cdot 2^3)} = \sqrt{2^{26}} = 2^{13} =$ **8 KB**.

Q6. (4, 95): $95 < 96 \rightarrow 1952 + 95 =$ **2047**. (4, 96): **trap** (offset must be < limit). (2, 100): **trap**. (0, 0): **219**.

Q7. Frames = $2^{26}/2^{12} = 2^{14} \rightarrow 2^{14} \times 8$ B = **128 KB**. Flat PT per process: $2^{20} \times 4$ B = **4 MB** per process — the inverted table is far smaller (and shared by all processes).

Q8. Each second-level table covers $2^{10} = 1024$ pages and is 4 KB. 3000 pages $\rightarrow \lceil 3000/1024 \rceil =$ **3** second-level tables (3 × 4 KB = 12 KB) + the top table (4 KB) = **16 KB** (3 + 1 = 4 frames).

Q9. Last-level PTE points to a data frame: frames = $2^{36}/2^{12} = 2^{24} \rightarrow$ **24 + 4 = 28 bits** (fits in 4 B).

Q10. $\lceil 1050/256 \rceil =$ **5 frames**; 5 × 256 = 1280 KB; internal fragmentation = 1280 − 1050 = **230 KB** (all in the last page).

Q11. **True** — the ASID distinguishes them (that is the point of the ASID).

Q12. EAT = t + m + (1−h)·k·m = 10 + 80 + 0.02 × 2 × 80 = 90 + 3.2 = **93.2 ns**.

CHAPTER 9

Virtual Memory, Page Faults, Page Replacement, Thrashing

GATE IMPORTANCE ★★★★★ *same top cluster as Ch 8: ≈ 2.4 marks/paper together*

◎ **GATE pattern (from your analysis):** (a) **count page faults** for FIFO / LRU / OPTIMAL (and sometimes *Clock/second chance*) on a reference string — a guaranteed-type 1- or 2-mark question; (b) **EAT with page faults** — `(1−p)·m + p·S`; (c) **Belady's anomaly** statements; (d) conceptual MSQ: demand paging, pinned pages, dirty bit, thrashing, working set, global vs local replacement. Practise the tables in §9.5 by hand until you can fill one in 90 seconds.

9.1 The idea of virtual memory NOTES

Locality of reference: programs use only a small part of their pages at any moment. So **not all pages of a process need to be in memory** at once.

- **Demand paging** NOTES: load a page **only when it is actually needed**. *Example:* a program has 10 pages; plain paging gives it 10 frames. With demand paging the loader brings in only the first few pages (say 2); others come on demand.
- **Virtual memory** = the CPU can use an **address space larger than physical memory**. A process with N pages can run with M frames, **N ≥ M** (practically a process needs **at least 2 frames** — one for code, one for data/stack).
- If the virtual address has **K bits**, the process's address-space limit is **2^K − 1**; K can be much larger than the number of physical-address bits L — thanks to demand paging and **swapping**.
- Benefits: a process **bigger than RAM** can run; **more processes** fit in memory ⇒ a **higher degree of multiprogramming** and better CPU utilisation NOTES.

Valid/invalid bit in the PTE (recap) NOTES**:** *valid* = page is legal **and in memory**; *invalid* = **not in memory** *or* never used/illegal. In older designs invalid meant "illegal access" (→ hardware interrupt); with demand paging an access to a legal-but-absent page is a **page fault**.

9.2 Page fault NOTES

Page fault = the CPU references a page whose PTE is **valid in the process's address space but whose page is not in memory**. The hardware raises a **trap** (an *exception*, Ch 1); the **OS** brings the page in and **restarts the same instruction**.

Steps of handling a page fault ⚡ (this order is an exam favourite)

1. The reference to the absent page **traps to the OS** (mode switch to kernel).
2. The OS checks the PT/PCB: is the address legal? If not → **segmentation fault** (terminate). If legal-but-absent → continue.
3. **Pause the process** (running → **waiting/blocked**) and **find the disk location** of the page (executable file or swap area).
4. **Find a free frame.** If **none is free**, the **page-replacement algorithm picks a victim frame**; if the victim is **dirty** (modified), it must be **written to the swap area** first and its PTE updated (marked "in swap"/invalid).
5. **Issue a disk read** (I/O) for the needed page into the frame. (Other processes can use the CPU meanwhile.)
6. When the I/O completes (**interrupt**), **update the page table** (frame number, valid = 1) and **move the process to ready**.
7. When it is scheduled again, **restart the instruction that faulted** — it now succeeds.

Overheads of a fault: fault handling + swap-out of a dirty victim + swap-in + restart. Disk is ~10^5 times slower than RAM ⇒ page faults are *very* expensive.

NOTES **Why would a 10-page program with only 2 frames need page 6?** Because (1) data are read from the executable file on disk, (2) the stack grows, or (3) `malloc` (heap). If the victim (say page 5, the stack) is **dirty**, it is written to the **swap area** and its PTE records that it lives in swap.

Pinned pages NOTES: some pages must **never be swapped out** — e.g., the page of an **interrupt service routine** (if it were swapped out, the page-fault handler itself could fault → infinite loop) and pages being used for **DMA/I-O**. These are **pinned (locked)** in memory; the replacement algorithm must skip them.

Swap area: a dedicated disk area (old Unix layout: boot block, superblock, inodes, file data, **swap area**) where evicted dirty pages and anonymous memory (stack/heap) live.

Copy-on-write (COW) ADDED (link to `fork`, Ch 2)

After `fork`, parent and child **share the same frames** marked read-only; only when one **writes** is that page **copied**. That is why `fork` is cheap.

9.3 ⚡ Performance of demand paging — EAT with page faults

Let **p** = page-fault rate (probability that a reference faults), **m** = memory access time, **S** = **page-fault service time** (disk read + overheads; if the victim is dirty add the write-back).

EAT = (1 − p)·m + p·S

Because S ≫ m, even a tiny p ruins performance.

NOTES ADDED *Example:* m = 200 ns, S = 8 ms = 8 000 000 ns.

- p = 1/1000 → EAT = 0.999 × 200 + 0.001 × 8 000 000 = 199.8 + 8000 = **8199.8 ns** (≈ 41× slower!).
- For **at most 10 % slowdown** (EAT ≤ 220 ns): 200(1−p) + 8 000 000 p ≤ 220 → 7 999 800 p ≤ 20 → **p ≤ 2.5 × 10^{-6}** (one fault in 400 000 references).

⚡ **Dirty victims:** if a fraction *d* of victims are dirty and a clean fault costs S, a dirty fault costs 2S (write + read): **S_avg = (1−d)S + d·2S**. Example: S = 8 ms, d = 60 % → 12.8 ms; p = 10^{-6}, m = 100 ns → EAT ≈ 100 + 12.8 = **112.8 ns**.

⚡ **Combined with a TLB:** EAT = (1 − p)·[TLB/paging EAT of Ch 8] + p·S.

9.4 Page replacement — the common set-up

When a fault occurs and **no frame is free**, a **page-replacement algorithm** picks a **victim page**. NOTES Theoretical assumptions: the process has a **fixed number of frames**, and on a fault only **one of its own frames** is replaced (**local replacement**). (*Global replacement may take frames from any process.*)

> ϟ **Basic counting facts** (used to sanity-check answers)

- **Minimum possible faults = number of distinct pages** (each must be loaded once — "compulsory" faults).
- **Maximum = length of the reference string.**
- If #frames $\geq$ #distinct pages $\rightarrow$ faults = #distinct pages (for every algorithm).
- With **1 frame**: a reference is a hit only if it repeats the page just before it.
- **Hit ratio = 1 − faults/references; fault rate = faults/references.**
- **OPT $\leq$ LRU, FIFO (any algorithm) in faults** for the same number of frames.

From addresses to a page string: page number = $\lfloor$address / page size$\rfloor$; consecutive references to the **same page** can be merged (the repeats are hits). NOTES *Example:* process of 32 B with page size 8 B (4 pages), 2 frames; addresses 5, 13, 1, 24, 12, 20, ... $\rightarrow$ pages 0, 1, 0, 3, 1, 2, ... (the full string used below is 0, 1, 0, 3, 1, 2, 1, 0).

9.5 The algorithms with worked tables

(In the tables a **bold** entry is the page just loaded at a fault; **F** = fault; · = empty frame.)

9.5.1 FIFO — First In First Out NOTES

Replace the page that has been **in memory the longest**. Implement with a queue (or a pointer that cycles through the frames). Simple, but may evict a heavily used page and suffers from **Belady's anomaly**.

NOTES **Example (notes):** 2 frames, string **0, 1, 0, 3, 1, 2, 1, 0**

Ref	0	1	0	3	1	2	1	0
Frame 1	**0**	0	0	**3**	3	3	**1**	1
Frame 2	·	**1**	1	1	1	**2**	2	**0**
Fault?	F	F		F		F	F	F

6 faults, fault rate = 6/8 = **0.75**.

9.5.2 Optimal (OPT / MIN) NOTES

Replace the page that **will not be used for the longest time in the future**. Gives the **minimum** faults — but needs future knowledge, so it is only a **yard-stick**. (How to do it: at a fault, look **forward** in the string from the current position; the victim is the page whose next use is farthest — or never.)

Same string, 2 frames:

Ref	0	1	0	3	1	2	1	0
Frame 1	**0**	0	0	**3**	3	**2**	2	**0**
Frame 2	·	**1**	1	1	1	1	1	1
Fault?	F	F		F		F		F

5 faults, rate = 5/8. (At the fault on 3, pages 0 and 1 are in memory; 1 is used next (position 5) but 0 only at position 8 $\rightarrow$ evict 0.)

9.5.3 LRU — Least Recently Used NOTES

Replace the page **not used for the longest time in the PAST** — an approximation of OPT using the past instead of the future (works because of locality). How to do it by hand: at a fault, look **backward** from the current position; the victim is the page whose **last use is the earliest**.

Same string, 2 frames:

Ref	0	1	0	3	1	2	1	0
Frame 1	**0**	0	0	0	**1**	1	1	1
Frame 2	·	**1**	1	**3**	3	**2**	2	**0**
Fault?	F	F		F	F	F		F

6 faults (so here FIFO = LRU = 6, OPT = 5).

How LRU is implemented NOTES**:**

1. **Timestamps:** the hardware writes the clock value into the PTE on **every access**; the victim = smallest timestamp (needs a search; the clock may overflow).
2. **Stack of pages:** on each access move the page to the **top**; the **bottom = LRU victim** (updating a stack on every reference is costly). Both are too expensive per reference ⇒ real systems use **approximations**.

9.5.4 Belady's anomaly NOTES — "more frames, more faults"

For the string **1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5**:

FIFO, 3 frames — 9 faults

Ref	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	**1**	1	1	**4**	4	4	**5**	5	5	5	5	5
Frame 2	·	**2**	2	2	**1**	1	1	1	1	**3**	3	3
Frame 3	·	·	**3**	3	3	**2**	2	2	2	2	**4**	4
Fault?	F	F	F	F	F	F	F			F	F	

FIFO, 4 frames — 10 faults (MORE!)

Ref	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	**1**	1	1	1	1	1	**5**	5	5	5	**4**	4
Frame 2	·	**2**	2	2	2	2	2	**1**	1	1	1	**5**
Frame 3	·	·	**3**	3	3	3	3	3	**2**	2	2	2
Frame 4	·	·	·	**4**	4	4	4	4	4	**3**	3	3
Fault?	F	F	F	F			F	F	F	F	F	F

Faults by number of frames (this string):

Frames	1	2	3	4	5	6
FIFO	12	12	**9**	**10** ⚠	5	5
LRU	12	12	10	8	5	5
OPT	12	9	7	6	5	5

- **Belady's anomaly = more frames ⇒ more faults.** It affects **FIFO** (not all strings) but **never LRU or OPT**, which are **stack algorithms** (the set of pages in memory with n frames is always a subset of the set with n+1 frames).
- **LRU on the same string:** 3 frames → **10**, 4 frames → **8** (the notes' LRU example gives 10).

LRU 3 frames, string above (notes' example, 10 faults):

Ref	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	**1**	1	1	**4**	4	4	**5**	5	5	**3**	3	3
Frame 2	·	**2**	2	2	**1**	1	1	1	1	1	**4**	4
Frame 3	·	·	**3**	3	3	**2**	2	2	2	2	2	**5**
Fault?	F	F	F	F	F	F	F			F	F	F

9.5.5 Approximating LRU NOTES

(a) Reference bits (aging): several history bits per PTE. On every access the hardware sets the **leading/least-significant bit**; periodically the OS **shifts the bits** so older history moves left/right. At a fault the page with the **smallest history value** is the victim; a newly loaded page's bits are reset.

(b) Clock / Second-Chance algorithm NOTES — the one GATE likes:

- Every page has a **reference (recently used, RU) bit**; pages sit in a **circular list**, with a **clock hand**.
- When a page is **loaded**, RU = 1. When a page in memory is **accessed**, the hardware sets RU = 1.
- **On a page fault**, look at the page the hand points to:
- if **RU = 1** → set it to **0** (a "second chance") and **advance** the hand;
- repeat until a page with **RU = 0** is found → **replace it**, set RU = 1 for the **new** page, **advance** the hand beyond it.
- *If every page has RU = 1 the hand makes a full circle clearing all bits and then evicts the first page → it degenerates to FIFO.*

Worked trace (3 frames; string **7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1**):

- 7, 0, 1 load into frames 1–3 (all RU = 1), the hand wraps to frame 1.
- **Ref 2** (fault): hand sweeps 7 (RU 1→0), 0 (1→0), 1 (1→0), back at frame 1 with RU = 0 → **evict 7**, load 2 (RU = 1), hand → frame 2.
- **Ref 0:** hit → RU(0) = 1. **Ref 3** (fault): hand at frame 2: page 0 has RU = 1 → clear it, advance; frame 3: page 1 has RU = 0 → **evict 1**, load 3. And so on.

Fault counts on this standard 20-reference string with **3 frames**:

Algorithm	FIFO	LRU	OPT	Clock
Faults	15	12	9	14

Clock table (14 faults):

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	**7**	7	7	**2**	2	2	2	**4**	4	4	4	**3**	3	3	3	**0**	0	0	0	0
Frame 2	·	**0**	0	0	0	0	0	0	**2**	2	2	2	2	**1**	1	1	1	**7**	7	7
Frame 3	·	·	**1**	1	1	**3**	3	3	3	3	**0**	0	0	0	**2**	2	2	2	2	**1**
Fault?	F	F	F	F		F		F	F		F	F		F	F	F		F		F

(*Convention used: a loaded page gets RU = 1; the hand then moves past it. Different books vary the details, so GATE states them in the question — follow the question.*)

FIFO table (15 faults), LRU table (12), OPT table (9) for the same string:

FIFO	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
F1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	**7**	7	7
F2	·	**0**	0	0	0	**3**	3	3	**2**	2	2	2	2	**1**	1	1	1	1	**0**	0
F3	·	·	**1**	1	1	1	**0**	0	0	**3**	3	3	3	3	**2**	2	2	2	2	**1**

LRU	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
F1	**7**	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	**1**	1	1	1	1	1	1
F2	·	**0**	0	0	0	0	0	0	0	**3**	3	3	3	3	3	**0**	0	0	0	0
F3	·	·	**1**	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	**7**	7	7

OPT	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
F1	**7**	7	7	**2**	2	2	2	2	2	2	2	2	2	2	2	2	2	**7**	7	7
F2	·	**0**	0	0	0	0	0	**4**	4	4	**0**	0	0	0	0	0	0	0	0	0
F3	·	·	**1**	1	1	**3**	3	3	3	3	3	3	3	**1**	1	1	1	1	1	1

(*Frames are shown by slot, so the same page stays in the same row until replaced.*)

9.5.6 Enhanced Second Chance, LFU, MFU NOTES

- **Enhanced second chance** also uses the **dirty bit**: classes (RU, Dirty) → **(0,0)** best victim (not recently used, clean — no write-back), then (0,1), (1,0), (1,1). Replaces the first page found in the lowest non-empty class while sweeping.
- **LFU (Least Frequently Used):** replace the page with the **smallest usage count**. Problem: a page used heavily early on but never again stays forever.
- **MFU (Most Frequently Used):** argument — the page with the smallest count was probably **just brought in** and hasn't been used yet. Both are rare in practice.

Shortcuts for replacement problems

1. **Write the string with positions 1...n** — OPT looks forward, LRU looks backward.
2. **For a hit, FIFO does nothing; LRU/Clock update recency.** (Most errors: updating FIFO order on a hit.)
3. In FIFO, the victim is just "the page that arrived first" — maintain a **queue** in arrival order.
4. **Cyclic strings** (1,2,3,4,1,2,3,4...) with k frames and k+1 pages: **FIFO and LRU fault on every reference** (12 faults for 12 refs with 3 frames); OPT does much better (6).
5. **Check:** FIFO/LRU/Clock $\geq$ OPT; faults $\geq$ #distinct pages.

9.6 Allocation of frames

How many frames does a process get? NOTES At least **1 for code + 1 for data** (practically ≥ 2) — the exact minimum depends on the architecture (the max number of pages one instruction can touch, e.g., with indirect addressing). Too few frames ⇒ too many faults.

Scheme	Rule
Equal	m/n frames each
Proportional	$a_i = (s_i / S) \cdot m$ (s_i = size of process i, $S = \Sigma\, s_i$) — *e.g.*, m = 62, sizes 10 and 127 → a_1 = 10/137 × 62 ≈ **4**, a_2 = 127/137 × 62 ≈ **57**
Priority	high-priority processes may take frames from low-priority ones NOTES

Global vs local replacement: *local* — a process evicts only its own pages (predictable per-process fault rate, but may waste frames); *global* — any frame (better throughput but a process's fault rate depends on others, and a hog can steal frames).

9.7 Thrashing NOTES

> **Thrashing** = a process (or the system) spends **more time servicing page faults than executing**.

The vicious cycle NOTES — if the number of frames of a process drops below the number of pages it actively uses, it faults constantly → the CPU sits idle waiting for the disk → **CPU utilisation falls** → a naive scheduler thinks the CPU is under-used and **admits more processes** (higher multiprogramming) → each process gets **fewer frames** → **more faults** → utilisation falls further …

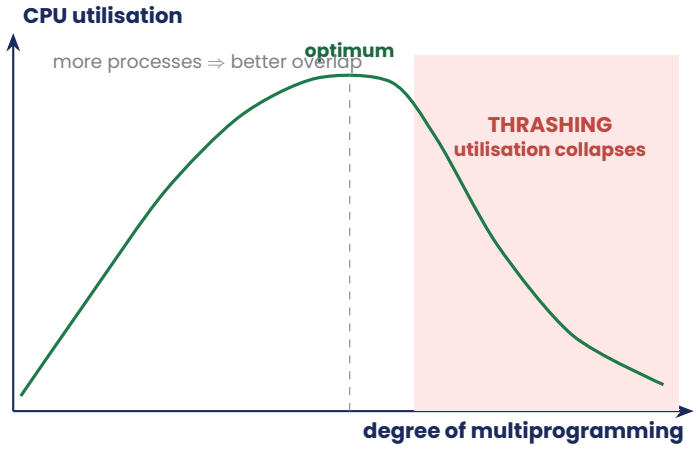


Figure 9.1. Beyond a certain degree of multiprogramming, CPU utilisation collapses: thrashing.

Remedies

- **Don't increase multiprogramming when utilisation is low because of paging; swap out** (suspend) some processes until the remaining ones have enough frames (the *medium-term scheduler*, Ch 4).
- Use **local replacement** so one thrashing process does not steal from others.
- **Working-set model** NOTES**:** a process needs a minimum number of frames for its current **locality**. **Working set WS(t, Δ) = the set of distinct pages referenced in the last Δ references (the window).** Give each process ≥ |**WS**| frames. With total demand **D = Σ**|WS$_i$| and F available frames: if **D > F** → thrashing is coming → **suspend** some (lower-priority) process; if D ≤ F you may admit more.
- **Page-Fault Frequency (PFF)** ADDED**:** monitor each process's fault rate; if above an upper bound → give it more frames; below a lower bound → take frames away; if no frames are free → suspend a process.

NOTES ADDED **Working-set example:** string 1,2,1,3,4,1,2,3,2,1 with Δ = 4: WS at t = 4 (last four refs 1,2,1,3) = {1,2,3} → size **3**; at t = 7 (3,4,1,2) = {1,2,3,4} → **4**; at t = 10 (2,3,2,1) = {1,2,3} → **3**.

9.8 MSQ / statement bank

1. *A page fault is an interrupt generated by the CPU hardware and handled by the OS.* — **True** (an exception/trap).
2. *After a page fault is serviced the instruction is restarted.* — **True.**
3. *Belady's anomaly can occur with LRU.* — **False** (stack algorithm).
4. *OPT is implementable in real systems.* — **False** (needs the future).
5. *Pages that hold the ISR must be pinned.* — **True.**
6. *A dirty victim page must be written to disk before reuse.* — **True.**
7. *Increasing the degree of multiprogramming always increases CPU utilisation.* — **False** (thrashing).
8. *Under demand paging a process can be larger than physical memory.* — **True.**
9. *The working-set model prevents thrashing by ensuring each process has |WS| frames.* — **True.**
10. *Clock algorithm needs a reference bit per page.* — **True.**

11. *FIFO may replace a frequently used page.* — **True.**
12. *LRU always gives the minimum number of faults.* — **False** (OPT does).
13. *Hit ratio can fall when more frames are given under FIFO.* — **True** (Belady).
14. *A page fault is detected by the hardware but serviced by the OS.* — **True.**
15. *Demand paging increases the degree of multiprogramming.* — **True.**
16. *The valid bit is 0 for pages not in memory.* — **True.**

9.9 Quick-revision capsule

- Demand paging: load on first use; page fault → trap → find victim (write back if dirty) → read page → update PT → restart instruction.
- **EAT = (1−p)m + p·S**; S = disk time (+ write-back if dirty).
- FIFO (queue, Belady ✔), OPT (look forward), LRU (look backward; stack algorithm), Clock (RU bit + hand: 1→0 skip, 0 → evict).
- Min faults = #distinct pages; OPT ≤ all others; LRU/OPT never show Belady's anomaly.
- Thrashing: low utilisation → more processes → worse. Cure: working set ($D = \Sigma|WS| \leq$ frames), PFF, suspend processes.
- Pinned pages: ISR and I/O buffers never evicted.

9.10 ADDED Practice set (GATE style)

Q1. Reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Faults for FIFO, LRU, OPT with 3 and 4 frames? Does Belady's anomaly occur?

Q2. String 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2 with **3 frames**: faults under FIFO, LRU, OPT and Clock. LRU hit ratio?

Q3. String 1,2,3,4,1,2,3,4,1,2,3,4 with 3 frames: FIFO, LRU, OPT faults?

Q4. m = 200 ns, S = 8 ms, $p = 10^{-3}$. EAT? What p allows only a 10 % slowdown?

Q5. m = 100 ns, page-fault service = 8 ms if the victim is clean, 16 ms if dirty, 60 % of victims dirty, $p = 10^{-6}$. EAT?

Q6. 62 free frames, two processes of 10 and 127 pages: proportional allocation?

Q7. String 1, 2, 1, 3, 4, 1, 2, 3, 2, 1, $\Delta = 4$: working-set sizes at t = 4, 7, 10. If there are 3 processes with such working sets of 3, 4 and 5 and only 10 frames — what should the OS do?

Q8. True/False: (a) FIFO with more frames never has more faults. (b) LRU with more frames never has more faults. (c) The minimum number of page faults for a string with 7 distinct pages is 7. (d) A page fault is handled entirely in hardware.

Q9. Memory is byte-addressable, page = 100 bytes, references (decimal addresses): 100, 200, 430, 499, 510, 530, 560, 120, 220, 240, 260, 320, 370. With 2 frames and FIFO, how many faults?

Q10. A process has 8 pages, runs with 4 frames under LRU on the string 0,1,2,3,4,0,1,2,3,4,5,6,7,0,1,2,3. How many faults?

Solutions

Q1. FIFO: **9** (3 frames), **10** (4 frames) → **Belady's anomaly occurs**. LRU: 10 and 8. OPT: 7 and 6. (Tables in §9.5.4.)

Q2. FIFO **9**, LRU **7**, OPT **6**, Clock **8**. LRU hit ratio = $(12-7)/12 = \mathbf{5/12} \approx \mathbf{0.417}$.

Q3. FIFO **12**, LRU **12**, OPT **6** (cyclic strings are the worst case for FIFO/LRU).

Q4. EAT = $0.999 \times 200 + 0.001 \times 8\,000\,000 =$ **8199.8 ns**. For ≤ 10 % slowdown: $\mathbf{p \leq 2.5 \times 10^{-6}}$.

Q5. S_avg $= 0.4 \times 8 + 0.6 \times 16 = 12.8$ ms; EAT $= (1-10^{-6}) \times 100 + 10^{-6} \times 12.8 \times 10^{6}$ ns $\approx 100 + 12.8 \approx$ **112.8 ns**.

Q6. $a_1 = 10/137 \times 62 \approx 4.5 \rightarrow$ **4**; $a_2 = 127/137 \times 62 \approx 57.5 \rightarrow$ **57** (total $61 \leq 62$).

Q7. WS sizes **3, 4, 3**. For three processes with $|WS| = 3, 4, 5$: $D = 12 > 10$ frames $\Rightarrow$ thrashing risk $\rightarrow$ **suspend (swap out) one process** (e.g., the lowest priority, or the one with $|WS| = 5$ leaving $D = 7 \leq 10$).

Q8. (a) **False** (Belady). (b) **True.** (c) **True.** (d) **False** — the hardware only raises the trap; the OS does the work (and the faulting process blocks during the disk I/O).

Q9. Pages $= \lfloor \text{addr}/100 \rfloor$: 100→1, 200→2, 430→4, 499→4, 510→5, 530→5, 560→5, 120→1, 220→2, 240→2, 260→2, 320→3, 370→3. Merging consecutive repeats: **1, 2, 4, 5, 1, 2, 3**. FIFO with 2 frames: 1 F; 2 F; 4 F (evict 1); 5 F (evict 2); 1 F (evict 4); 2 F (evict 5); 3 F (evict 1) $\rightarrow$ **7 faults**.

Q10. LRU, 4 frames: 0 F,1 F,2 F,3 F (4); 4 F (evict 0) (5); 0 F (evict 1) (6); 1 F (evict 2) (7); 2 F (evict 3) (8); 3 F (evict 4) (9); 4 F (evict 0) (10); 5 F (evict 1) (11); 6 F (evict 2) (12); 7 F (evict 3) (13); 0 F (evict 4) (14); 1 F (evict 5) (15); 2 F (evict 6) (16); 3 F (evict 7) (17) $\rightarrow$ **17 faults** (every reference faults: with the pages being used cyclically over 5+ pages and only 4 frames, LRU is at its worst case).

PART V

Storage

CHAPTER 10

File Systems: Files, Directories, Inodes, Allocation, Free Space

GATE IMPORTANCE ★★★☆☆ *part of the "Disk / File system / I/O" cluster ≈ 1.4 marks/paper, Tier 2*

◎ **GATE pattern (from your analysis):** *Numerical* inode / FAT questions: "maximum file size", "disk size", "how many disk accesses to reach byte X", "size of the FAT", "number of blocks used by a file including index blocks" — plus 1-mark statements on allocation methods, directories and free-space management. The arithmetic is always **powers of 2**: learn the three templates in §10.6.

10.1 What is a file system?

Programs and data must survive after a program ends/power-off, so they live on **secondary storage** (disk). The **file system (FS)** is the OS module that lets us **create, read, write and manage files** on it.

- **File** NOTES = a **structured set of bits** with a name. *Operations:* create, read, write, update (reposition/seek, delete, truncate).
- **Key goals** NOTES: **organised storage** (names, directories), **efficient metadata**, and **efficient disk access**.
- **Disk units:** the smallest hardware division of a magnetic disk is a **sector**. The FS reads/writes in **blocks** (1–2 or more sectors, e.g., 512 B–4 KB). *The FS sees the disk as a **sequence of blocks**; its job is: given a **file name + offset**, find the right block.*
- **Directory** NOTES = a file with special contents — a table of ⟨**file name, inode number**⟩. In Unix **everything is a file** (including directories and devices).

Metadata NOTES

Information *about* the file, kept separately from its data: **name, location (block pointers), size, protection (permission) bits, time/date stamps, user (owner) info**, and an **identifier** — in Unix the **inode number**.

10.2 Inode NOTES

In Unix every file or directory has one **inode** (index node), identified by its **inode number**. **File = inode + data blocks.**

- The inode holds the **metadata** (owner, permissions, size, timestamps, link count ...) and the **data-block pointers** (where the file's blocks are). The **file name is *not* in the inode** — it lives in the directory entry (⟨name, inode no.⟩ pairs). (*The notes list "name?" with a question mark; in classic Unix the name is only in the directory.*)
- For a **directory**, the inode's pointers lead to the directory's data = the table ⟨file name → inode number⟩.

Path resolution (ADDED): to open `/usr/ast/mbox` the OS reads the **root inode**, then root's data block (find `usr`'s inode number), then `usr`'s inode, `usr`'s data block, `ast`'s inode, `ast`'s data block, and finally `mbox`'s inode: **2 × (number of components) + 1** reads (if each directory fits in one block and the root inode's location is known) = 2·3 + 1 = **7**.

Open-file tables ADDED: each `open()` returns a **file descriptor** (index into the process's table) → an entry in the **system-wide open-file table** (current file offset, access mode) → the **in-memory inode**. `fork` shares open-file-table entries (and hence the file offset) between parent and child.

Links ADDED: a **hard link** = another directory entry pointing to the **same inode** (the inode's *link count* rises; the file is deleted when it reaches 0; cannot cross file systems, not for directories). A **symbolic (soft) link** = a small file containing a **path name**; it can dangle and can cross file systems.

10.3 Directory structures NOTES

Structure	Idea	Problem
Single-level	one directory for all files	name clashes, no grouping
Two-level	one directory per user	no sub-grouping, sharing is hard
Tree (multilevel)	directories inside directories — used today	—
Acyclic graph (DAG)	tree + shared subdirectories/files via links	deletion and dangling-link issues; cycles are avoided

Implementation of a directory: linear list (simple, slow search) or hash table (fast, collisions).

10.4 Disk layout (old Unix) NOTES

Boot block → Superblock (FS size, number of files, free-list info) → Inode area → File-content (data) area → Swap area.

10.5 File allocation (implementation) methods NOTES

Metrics to compare: fragmentation, ease of appending (growth), access time (sequential and random), implementation complexity, storage overhead.

1 Contiguous allocation

The user declares the **length up front**; the FS gives **one contiguous run of blocks**. The inode stores ⟨**start block, size**⟩.

- ✔ Sequential and random access are **very fast** (address = start + i).
- ⊗ **Cannot grow** (the next blocks may be taken); large **external fragmentation**; some internal fragmentation (last block).

2 Extent-based allocation

A file is made of **several contiguous extents**; the inode keeps an **array of** ⟨**start block, size**⟩.

- ✔ Can **grow** (add an extent); **small internal fragmentation, reduced external fragmentation**; fast sequential, simple random access.

3 Linked allocation

The file is divided into chunks (slightly smaller than a block); **each block stores a pointer to the next block**; the inode keeps the pointer to the **first** block.

- ✔ Easy growth; **no external fragmentation**.
- ⊗ **Pointer overhead in every block**; **sequential access slow** (follow the chain on disk); **random access very difficult** — to reach block i you must read i blocks from disk.

4 FAT (File Allocation Table) based

Keep all the "next" pointers **in one table**, the **FAT**, with **one entry per disk block**: **FAT[i] = j** means *the block after block i is block j* (a special value marks end of file). The FAT is cached **in memory**, so to reach block 100 you read **the FAT 100 times (memory)** and then **one disk read** for the data — instead of 100 disk reads. Random access is much better than plain linked allocation.

NOTES **FAT size example:** FAT entry = 16 bits ⇒ at most 2^{16} blocks. Block = 512 B ⇒ **maximum file-system size = 2^{16} × 512 B = 2^{25} B = 32 MB**; **FAT size = 2^{16} entries × 16 bits = 2^{17} B = 128 KB** (*the notebook says 127 KB — a typo, see Appendix C*).

5 Indexed allocation

All the block pointers of a file are kept in one index table (the "index block") in/for the inode. Must fix a **maximum file size** up front; the index table needs contiguous space.

- ✔ Fast sequential and random access; **no external fragmentation**; the file can grow up to the maximum.
- ⊗ **Index overhead** (the index block is paid even for tiny files; bigger inode).

6 Multilevel indexed (the real Unix inode) NOTES

The inode contains **D direct pointers + 1 single-indirect + 1 double-indirect + 1 triple-indirect** pointer.

- Small files: only direct pointers (fast, no extra I/O).
- Large files: indirect blocks (a block full of pointers); the inode stays small yet files can be huge. Reaching a block deep in the file may need **extra disk reads** (one per indirection level).

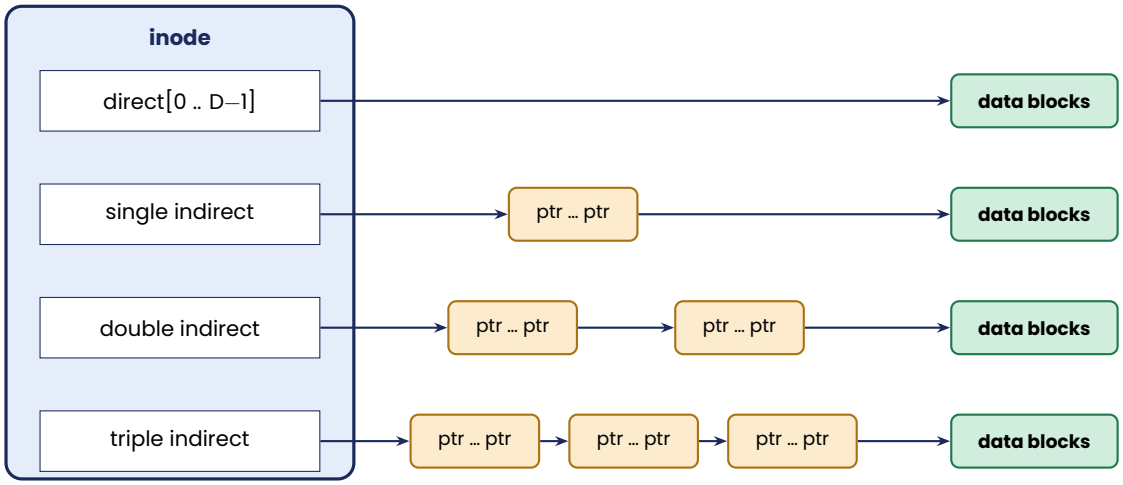


Figure 10.1. Unix-style inode: direct pointers plus single, double and triple indirect blocks.

Comparison (learn this table)

Method	External frag.	Growth	Sequential	Random access	Overhead
Contiguous	**High**	✘	fastest	fastest	none
Extent	low	✔	fast	simple	small
Linked	none	✔✔	slow	**very slow**	pointer in every block
FAT	none	✔	OK	OK (chain in memory)	FAT in memory (large for big disks)
Indexed	none	up to max	fast	fast	index block per file
Multilevel indexed	none	huge	fast	fast ($\leq$ 4 reads)	inode + indirect blocks

10.6 The three arithmetic templates

Template A — "Maximum file size" with a Unix-like inode

Let **B** = block size, **p** = pointer size $\Rightarrow$ **n = B / p pointers per block**. With **D direct** + single + double + triple indirect:

Max file size = (D + n + n^2 + n^3) × B (limited also by the **disk size** and by the file-size field)

NOTES **Worked example 10.1:** block = **2048 B**, **disk addresses are 32-bit** (4 B pointers) $\Rightarrow$ the disk has 2^{32} blocks; inode with **12 direct, 1 single, 1 double, 1 triple** indirect.

- **(a) Disk size** = 2^{32} blocks × 2^{11} B = 2^{43} **B = 8 TB.**
- **(b)** n = 2048/4 = **512**. Max file = (12 + 512 + 512^2 + 512^3) × 2048 B = 134 480 396 blocks × 2048 B $\approx$ **2.75 × 10^{11} B $\approx$ 256.5 GB** ($\approx 2^8$ GB × 1.002). (*Shortcut: the triple-indirect term dominates:* $512^3 \times 2\ KB = 2^{27} \times 2^{11} = 2^{38}\ B = 256\ GB$.)

Template B — "How many disk accesses to read block k?"

Assume the **inode is already in memory** and nothing is cached.

- Block in the **direct** area $\rightarrow$ **1** access (the data block).
- In **single-indirect** range $\rightarrow$ **2** (index block + data).
- In **double-indirect** range $\rightarrow$ **3**; **triple** $\rightarrow$ **4**. Ranges (block numbers counting from 0): direct 0...D−1; single D...D+n−1; double D+n...D+n+n^2−1; triple beyond. *Example:* D = 12, n = 512, block number **600** $\rightarrow$ 600 $\geq$ 12+512 = 524 $\rightarrow$ **double** indirect range $\rightarrow$ **3 accesses**.

Template C — "Total blocks used by a file, including index blocks"

Example: **file 20 MB, block 4 KB, 4 B pointers (n = 1024), 12 direct pointers.**

- Data blocks = 20 MB / 4 KB = **5120**.
- 12 in direct; next 1024 via **1 single-indirect block**; remaining 5120 − 12 − 1024 = **4084** via the double-indirect: the **top double-indirect block (1)** + $\lceil 4084/1024 \rceil$ = **4** second-level blocks.
- Index blocks = 1 + 1 + 4 = **6** $\Rightarrow$ total = 5120 + 6 = **5126 blocks**.

Other quick formulas

Quantity	Formula
Blocks in a file of L bytes	$\lceil L / B \rceil$; internal fragmentation = $\lceil L/B \rceil \cdot B - L$
Linked allocation: reach block i	i disk reads (+1 for the first pointer in the inode)
FAT: reach block i	i FAT (memory) look-ups + **1** disk read
FAT size	#blocks × entry size (entry width $\geq \log_2$ #blocks)
Indexed (single index block): max file	n × B
Contiguous: block address	start + $\lfloor$offset / B$\rfloor$

10.7 Free-space management NOTES

Needs: **find a free block quickly; small bookkeeping space.**

Method	How	Cost
Bit vector (bitmap)	1 bit per block (1 = free in the notes' convention; 0 = free in some books); N bits for N blocks	search O(N), may need several disk reads; compact. *e.g., 1 TB disk, 4 KB blocks* $\rightarrow 2^{28}$ *blocks* $\rightarrow 2^{28}$ ***bits = 32 MB***
Linked list	head pointer to the first free block; each free block stores a pointer to the next	**O(1)** to take a block; extra space $\approx \log_2$ N bits per pointer (stored in the free blocks themselves)
Grouping	one free block stores the addresses of **several** free blocks (the last points to another such block)	fewer reads
Counting	store ⟨**start block, count**⟩ of runs of free blocks	compact when free space is contiguous

10.8 MSQ / statement bank

1. *In Unix the file name is stored in the inode.* — **False** (in the directory entry).
2. *A directory in Unix is a special file containing ⟨name, inode number⟩ pairs.* — **True.**
3. *Contiguous allocation supports efficient random access but suffers from external fragmentation.* — **True.**
4. *Linked allocation supports efficient random access.* — **False.**
5. *FAT must be loaded in memory to get fast random access.* — **True.**
6. *In a multilevel inode, small files need no indirect blocks.* — **True.**
7. *Indexed allocation suffers from external fragmentation.* — **False.**
8. *The maximum size of a file in a Unix-like FS can be limited by the pointer size, not just by the inode structure.* — **True** (a pointer can't address more blocks than 2^(pointer bits)).
9. *Bit-vector free-space management needs space proportional to disk size.* — **True.**
10. *A hard link and its original name refer to the same inode.* — **True.**
11. *Extent-based allocation reduces external fragmentation compared to contiguous.* — **True.**
12. *A file-system block may consist of several disk sectors.* — **True** (hardware unit = sector; FS unit = block).

10.9 Quick-revision capsule

- FS: file = inode + data; directory = ⟨name, inode no.⟩ table; metadata in the inode (not the name).
- Allocation: contiguous (fast, no growth), extent, linked (slow random), FAT (chain in memory), indexed, multilevel indexed (Unix).
- **Max file =** $(\mathbf{D + n + n^2 + n^3}) \cdot \mathbf{B}$, n = B/p. Disk size = #blocks × B = 2^(pointer bits) × B.
- Accesses: direct 1, single 2, double 3, triple 4 (inode cached).
- FAT size = #blocks × entry width; FS size = 2^(entry bits) × B.
- Free space: bit vector (N bits), linked list (O(1)), grouping, counting.

10.10 ADDED Practice set (GATE style)

Q1. Block size 1 KB, 4-byte pointers, an inode with 10 direct + single + double + triple indirect pointers. Maximum file size?

Q2. For Q1's inode, how many disk accesses (inode in memory) to read the byte at offset 5 000 000? (offset → block number.)

Q3. File of 20 MB, block 4 KB, pointer 4 B, inode with 12 direct + 1 single + 1 double + 1 triple pointers: total blocks (data + index)?

Q4. A FAT-16 file system (16-bit entries) with 4 KB blocks: maximum FS size and FAT size?

Q5. A 1 GB disk with 4 KB blocks uses a FAT. What is the FAT size if an entry is (a) exactly as wide as needed, (b) rounded up to whole bytes, (c) 4 bytes?

Q6. A file of 10 500 bytes on a 1 KB-block FS using contiguous allocation: blocks used and internal fragmentation?

Q7. Free-space bitmap for a 1 TB disk with 4 KB blocks: size?

Q8. Linked allocation: how many disk reads to read the 50th block of a file (the inode holds the first-block pointer)? FAT version?

Q9. Disk with 2^{32} blocks of 2 KB and 4-byte pointers, 12 direct, one single, one double, one triple indirect: is the max file size limited by the inode scheme or by the disk?

Q10. True/False: (a) A file's name is stored in its inode. (b) A single-indirect pointer adds n blocks to the reachable range. (c) The inode of a file stays in memory while the file is open.

Solutions

Q1. $n = 1024/4 = 256$. Blocks $= 10 + 256 + 256^2 + 256^3 = 10 + 256 + 65\,536 + 16\,777\,216 = 16\,843\,018$ blocks $\Rightarrow \times 1$ KB $\approx$ **16.06 GB** ($\approx 1.72 \times 10^{10}$ B).

Q2. Offset 5 000 000 / 1024 = 4882.8 $\rightarrow$ **block 4882**. Direct: 0–9; single: 10–265 (256 blocks); double: 266–65 801 ($266 + 65\,536 - 1$). 4882 lies in the **double-indirect** range $\Rightarrow$ **3 disk accesses** (two index blocks + the data block).

Q3. **5126 blocks** (5120 data + 1 single-indirect + 1 double-indirect top + 4 second-level; Template C).

Q4. Max blocks $= 2^{16} \Rightarrow$ FS size $= 2^{16} \times 4$ KB = **256 MB**; FAT $= 2^{16} \times 2$ B = **128 KB**.

Q5. Blocks $= 2^{30}/2^{12} = 2^{18} = 262\,144 \Rightarrow$ entries need ≥ 18 bits. (a) exact 18-bit entries: $2^{18} \times 18$ bits = **576 KB**; (b) rounded up to whole bytes (3 B): 786 432 B = **768 KB**; (c) 4 B entries: **1 MB**.

Q6. $\lceil 10\,500/1024 \rceil$ = **11 blocks**; internal fragmentation $= 11 \times 1024 - 10\,500$ = **764 B**.

Q7. Blocks $= 2^{40}/2^{12} = 2^{28} \Rightarrow 2^{28}$ **bits** = 2^{25} **B = 32 MB**.

Q8. Linked: **50 disk reads** (follow the chain; counting the data block itself as one of them, i.e., the 50th block needs 50 reads). FAT: **49 memory look-ups + 1 disk read** (if the FAT is in memory).

Q9. The disk is $2^{32} \times 2$ KB = 8 TB; the inode scheme allows ≈ 256.5 GB — so the **inode scheme** is the limit (8 TB is far larger).

Q10. (a) **False.** (b) **True** (n = B/p more data blocks). (c) **True** (the in-memory copy is used for speed).

CHAPTER 11

Disk Structure, Disk Scheduling, and I/O

GATE IMPORTANCE ★★★☆☆ *same cluster as Ch 10;* ADDED *= added from the PYQ analysis — **the lecture notes do not cover this chapter***

◎ **GATE pattern (from your analysis):** (a) **Disk access time** — seek + rotational latency + transfer (a 2-mark numerical, e.g., "7200 RPM, … find the time to read …"); (b) **Disk-scheduling order / total head movement** for FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK; (c) **I/O methods** — polling vs interrupt vs DMA, "what fraction of CPU time is used", "what happens at the end of a DMA transfer"; (d) 1-mark statements. Everything is small arithmetic — you only need the **definitions and the order of events**.

11.1 ADDED Disk structure

A magnetic disk = several **platters** stacked on a spindle that rotates at a fixed **RPM**.

- Each platter has two **surfaces**; each surface has a **read/write head** (all heads move together on the **arm**).
- A surface is divided into concentric **tracks**; the same track number on all surfaces forms a **cylinder**.
- Each track is divided into **sectors** (the smallest unit the hardware reads/writes; typically 512 B – 4 KB). The FS uses **blocks** = 1 or more sectors (Ch 10).

Capacity = (#surfaces) × (tracks per surface) × (sectors per track) × (bytes per sector). (#surfaces = 2 × #platters if both sides are used.)

Block number → (cylinder, head, sector) (0-indexed, filling a track, then the next head in the same cylinder, then the next cylinder):

cylinder = $\lfloor b / (H \cdot S) \rfloor$ head = $\lfloor b / S \rfloor \bmod H$ sector = $b \bmod S$ (H = surfaces/heads, S = sectors per track)

Example: H = 8, S = 200: block 123 456 → cylinder $\lfloor 123456/1600 \rfloor$ = **77**, head = ($\lfloor 123456/200 \rfloor$ = 617) mod 8 = **1**, sector = 123456 mod 200 = **56**. (Check: 77·1600 + 1·200 + 56 = 123 456 ✔.)

11.2 ADDED Disk access time — the formula you must know

Access time = seek time + rotational latency + transfer time

Component	Meaning	Formula
Seek time	move the arm to the correct **cylinder**	given (average) or "x ms per cylinder × distance"
Rotational latency	wait for the required sector to rotate under the head	**average = ½ rotation time**; worst = 1 rotation
Transfer time	time for the sectors to pass under the head	(bytes to read / bytes per track) × rotation time

Rotation time = 60 / RPM seconds. ⚡ **Memorise:** 5400 RPM → 11.11 ms (avg latency 5.56 ms); **7200 RPM → 8.33 ms (avg latency 4.17 ms)**; 10 000 RPM → 6 ms (avg 3 ms); 15 000 RPM → 4 ms (avg 2 ms). **Data transfer rate** = (bytes per track) × (rotations per second) = bytes per track / rotation time.

Worked examples

Ex 11.1. 7200 RPM, 100 sectors per track of 512 B, average seek 5 ms. **Read a 4 KB block (8 contiguous sectors on one track).**

- rotation time = 8.33 ms → avg latency = **4.17 ms**
- transfer = 8 × (8.33 / 100) = **0.67 ms**
- **Total = 5 + 4.17 + 0.67 ≈ 9.83 ms.**
- *One sector only:* 5 + 4.17 + 0.083 = **9.25 ms**.
- *Data rate:* track = 51 200 B per 8.33 ms ≈ **6.1 MB/s**.

Ex 11.2 — sequential vs random. Reading **100 sectors**:

- **scattered (random) sectors**, each needs its own seek + latency: 100 × 9.25 ms = **925 ms**;
- **contiguous (one whole track)**: 5 + 4.17 + 8.33 = **17.5 ms**. → *This is why file systems try to keep files contiguous and why disk scheduling matters.*

> ⚠ **Traps:** (1) *If the question says "the head is already on the correct track" → seek = 0.* (2) *Rotational latency* uses the **average** (half rotation) unless the "worst case" is asked. (3) *If successive blocks need a seek each time, count a seek per block.* (4) Convert RPM carefully (60/RPM s → ms ×1000).

11.3 ADDED Disk scheduling

Many requests may be waiting for different cylinders; the **order** in which they are served changes the **total head movement (seek)**. Seek time dominates (rotation and transfer are fixed), so schedulers try to **minimise head movement** and keep things fair.

Request queue (cylinders): **98, 183, 37, 122, 14, 124, 65, 67**, head at **53**, disk with cylinders **0–199**.

Algorithms

Algorithm	Rule
FCFS	serve in arrival order
SSTF (Shortest Seek Time First)	serve the request **closest to the current head position** (greedy; like SJF)
SCAN ("elevator")	head moves in **one direction to the very end of the disk**, serving requests on the way, then **reverses** and serves on the way back
C-SCAN (Circular SCAN)	go in one direction to the end serving requests; then **jump back to the other end** without serving, and sweep again in the **same** direction → **more uniform** waiting time
LOOK	like SCAN but **reverse at the last request** in the direction (don't go to the disk end)
C-LOOK	like C-SCAN but jump back from the **last request** to the **first (lowest) request**

Worked answers (head at 53, moving up initially, cylinders 0–199)

Algorithm	Order of service	Total head movement
FCFS	98, 183, 37, 122, 14, 124, 65, 67	45+85+146+85+108+110+59+2 = **640**
SSTF	65, 67, 37, 14, 98, 122, 124, 183	12+2+30+23+84+24+2+59 = **236**
SCAN (up first)	65, 67, 98, 122, 124, 183, **199**, 37, 14	(199−53) + (199−14) = 146 + 185 = **331**
SCAN (down first)	37, 14, **0**, 65, 67, 98, 122, 124, 183	53 + 183 = **236**
LOOK (up first)	65, 67, 98, 122, 124, 183, 37, 14	(183−53) + (183−14) = 130 + 169 = **299**
LOOK (down first)	37, 14, 65, 67, 98, 122, 124, 183	(53−14) + (183−14) = 39 + 169 = **208**
C-SCAN (up)	65, 67, 98, 122, 124, 183, **199**, (jump to **0**), 14, 37	146 + 199 + 37 = **382** *(counting the return jump)*
C-LOOK (up)	65, 67, 98, 122, 124, 183, (jump to 14), 14, 37	130 + 169 + 23 = **322** *(counting the jump)*

⚠ **Convention:** for C-SCAN/C-LOOK, whether the **return jump is counted** as head movement depends on the question — GATE says explicitly ("the return seek is/isn't counted"). The numbers above count it. **Also note:** whether SCAN goes to the physical end (199 / 0) or only to the last request (that is LOOK) — check the wording; the simple "SCAN" goes to the end.

⚡ **Quick method for SCAN/LOOK:** sort the requests; split at the head position; the total movement = (distance to the far end of the first direction) + (distance back to the farthest request on the other side). Example SCAN-up: (end − head) + (end − lowest request).

Properties

	Pros	Cons
FCFS	fair, simple	long head movement
SSTF	low total seek	**starvation** of far requests (like SJF)
SCAN	no starvation, bounded wait	middle cylinders served more often than the edges; wasted travel to the end
C-SCAN	**uniform** waiting time	return trip does no useful work
LOOK / C-LOOK	avoid useless travel to the ends	—

SCAN/C-SCAN/LOOK/C-LOOK are best when the **load is heavy**; with very few requests FCFS is fine. Scheduling matters only for **magnetic disks** (an SSD has no seek).

Second example (practice)

Queue **82, 170, 43, 140, 24, 16, 190**; head at **50**; cylinders 0–199; moving up. **FCFS 642; SSTF 208 (order 43, 24, 16, 82, 140, 170, 190); SCAN-up 332; LOOK-up 314; C-SCAN-up 391; C-LOOK-up 341.** (*SCAN-down = 240, LOOK-down = 208.*)

11.4 ADDED Input/Output (I/O) methods

An I/O device is operated through its **controller**, which has **registers**: a **status** register (busy/ready/error), a **command** register, and **data** registers/buffer. The CPU can address them either with **special I/O instructions** (port-mapped; privileged) or as ordinary memory addresses (**memory-mapped I/O**).

The three ways to do I/O

1. Polling (programmed I/O, busy-wait). The CPU **repeatedly reads the status register** until the device is ready, then moves the data itself.

- ✔ Simple, no interrupt overhead; good if the device is **very fast** or the CPU has nothing else to do.
- ⊗ **Wastes CPU time** while waiting.

2. Interrupt-driven I/O. The CPU issues the command and **goes on with other work**; when the device is ready it raises an **interrupt**; the **ISR** transfers one byte/word and returns.

- ✔ No busy waiting.
- ⊗ **One interrupt per byte/word** — expensive for large/fast transfers (context overhead each time).

3. DMA (Direct Memory Access). A **DMA controller** moves a **whole block directly between device and memory without the CPU touching each word**. Sequence of events (exam favourite):

1. CPU (driver) **programs the DMA controller**: **device, memory address, byte count, direction (read-/write)**.
2. The DMA controller **requests the bus**, transfers the data word by word (**cycle stealing** — it takes bus cycles from the CPU — or burst mode), **incrementing the address and decrementing the count**.
3. When the count reaches 0, the DMA controller **raises ONE interrupt** to tell the CPU the whole transfer is done.
4. The CPU (ISR) handles completion, e.g., marks the waiting process ready.

- ✔ CPU is free during the transfer; **one interrupt per block** instead of per word. ⊗ Needs extra hardware; the DMA competes with the CPU for the memory bus.

	Polling	Interrupt	DMA
CPU involved in each word?	yes	yes (ISR per word)	**no**
CPU waiting?	busy-wait	free	free
Interrupts per transfer	0	one per word/byte	**one per block**
Best for	fast/simple, short waits	slow/char devices (keyboard)	**block devices** (disk, network)

ADDED CPU-time calculation templates

- **Polling:** fraction of CPU = (polls per second × cycles per poll) / (CPU clock). *E.g.*, 100 polls/s × 400 cycles on a 1 GHz CPU → 40 000 / 10^9 = **0.004 %**.
- **Interrupt-driven:** if the device delivers a 4-byte word every **4 µs** (1 MB/s) and each interrupt costs **2 µs** of CPU, the CPU spends **2 / 4 = 50 %** of its time on I/O.
- **DMA:** one block of 4096 B = 1024 words × 4 µs = **4096 µs** of device time; the CPU spends only the **setup + completion-interrupt** cost, say 20 µs → 20 / 4096 ≈ **0.49 %** (plus a little bus contention).
- **Time for a DMA transfer** = setup + (block size / data-transfer rate) + interrupt handling.

ADDED Related ideas

- **Buffering** (hold data in memory to smooth speed differences), **caching** (keep copies of frequently used disk blocks — the *buffer cache*), **spooling** (queue output for a slow device such as a printer).
- **Device drivers** are the device-specific part of the OS (run in kernel mode); the OS offers device-independent system calls (`read`, `write`) on top.
- **Blocking I/O** — the process sleeps until the I/O completes (Ch 2: running → waiting → ready); **non-blocking / asynchronous I/O** — the call returns at once.
- **Pinned pages** (Ch 9): buffers used for DMA must not be paged out during the transfer.

11.5 MSQ / statement bank

1. *Average rotational latency is half the time of one revolution.* — **True.**
2. *SSTF can cause starvation.* — **True.**
3. *SCAN can cause starvation.* — **False** (every request is served within a sweep).
4. *LOOK always has a smaller (or equal) head movement than SCAN for the same queue.* — **True.**
5. *C-SCAN provides more uniform waiting times than SCAN.* — **True.**
6. *In DMA, the CPU is interrupted after every byte.* — **False** (once, at the end of the block).
7. *DMA needs the CPU to supply the memory address, count and direction before the transfer starts.* — **True.**
8. *Polling is more efficient than interrupts when the device is very fast and always ready.* — **True.**
9. *Seek time depends on the rotation speed of the disk.* — **False.**
10. *Reading contiguous sectors of a track avoids repeated seek and rotational latency.* — **True.**
11. *Disk-scheduling algorithms reduce the rotational latency.* — **False** (they reduce seek).
12. *A system call `read()` on a disk file from a user program must run in user mode only.* — **False** (the actual I/O in the kernel; I/O instructions are privileged).
13. *Memory-mapped I/O lets ordinary load/store instructions access device registers.* — **True.**
14. *FCFS disk scheduling is fair but can give a high average seek time.* — **True.**

11.6 Quick-revision capsule

- Access time = **seek + ½ rotation + transfer**; rotation = 60/RPM (7200 RPM → 8.33 ms, avg latency 4.17 ms).
- Scheduling: FCFS, SSTF (starves), SCAN (to the end), C-SCAN (jump, uniform), LOOK/C-LOOK (to last request). Count head movement in cylinders.
- I/O: polling (CPU spins), interrupt (per word), DMA (per block: program controller → transfer → one interrupt).

11.7 ADDED Practice set

Q1. 7200 RPM, average seek 8 ms, 200 sectors per track × 512 B. Time to read 10 contiguous sectors from a random track? Time to read one sector?

Q2. Disk with 4 platters (8 surfaces), 1000 tracks per surface, 200 sectors per track, 512 B sectors. (a) Capacity? (b) cylinder/head/sector of block 123 456?

Q3. Queue 82, 170, 43, 140, 24, 16, 190; head at 50 (cylinders 0–199, moving up). Compute FCFS and SSTF total movement.

Q4. In Q3, if the head moves 0.1 ms per cylinder, how long is the total seek time under FCFS and SSTF?

Q5. Compare CPU overhead: a device sends 4-byte words at 1 MB/s. (a) interrupt-driven with 2 µs per interrupt; (b) DMA with 4 KB blocks and 20 µs overhead per block.

Q6. A DMA transfer reads 8 KB at 2 MB/s (assume 1 MB = 10^6 B... use 8192 B / 2×10^6 B/s). The setup costs 100 µs and the completion interrupt 50 µs. Total time?

Q7. Rotation times for 5400 and 10 000 RPM; and the average latencies.

Q8. *True/False:* (a) SCAN may serve a request that arrives behind the head only after the head reverses. (b) In C-SCAN the head serves requests on the return trip. (c) LOOK never goes to cylinder 0 unless there is a request there.

Solutions

Q1. rotation = 8.33 ms → latency 4.17 ms; one sector transfer = 8.33/200 = 0.0417 ms. **10 sectors:** 8 + 4.17 + 0.417 ≈ **12.58 ms**. **One sector:** 8 + 4.17 + 0.04 ≈ **12.21 ms**.

Q2. (a) 8 × 1000 × 200 × 512 B = 819 200 000 B ≈ **781 MiB (≈ 819 MB)**. (b) cylinder **77**, head **1**, sector **56** (§11.1).

Q3. **FCFS = 642**; **SSTF = 208** (order 43, 24, 16, 82, 140, 170, 190: 7+19+8+66+58+30+20).

Q4. FCFS: 642 × 0.1 = **64.2 ms**; SSTF: 208 × 0.1 = **20.8 ms**.

Q5. (a) 2 µs per 4 µs → **50 %** of the CPU. (b) 4096 B block = 1024 words = 4096 µs of transfer; overhead 20 µs → ≈ **0.49 %**.

Q6. Transfer time = 8192 / (2×10^6) = 4096 µs ≈ 4.1 ms. Total ≈ 100 + 4096 + 50 = **4246 µs ≈ 4.25 ms** (the CPU is busy for only the 150 µs of setup + completion).

Q7. 5400 RPM → 60/5400 = **11.11 ms**, avg latency **5.56 ms**. 10 000 RPM → **6 ms**, avg **3 ms**.

Q8. (a) **True.** (b) **False** (it jumps back without serving). (c) **True.**

PART VI

Exam Toolkit

CHAPTER 12

Formula Sheet, Trap List, Mixed Practice

GATE IMPORTANCE ★★★☆☆

Use this chapter in the **last two days**: skim §12.1 and §12.2 every morning, then do §12.3 under a timer. Everything here is distilled from Chapters 1–11.

12.1 Master formula sheet

Processes, threads, fork (Ch 1–3)

- **fork count:** n sequential unconditional `fork()` calls ⇒ 2^n **processes** ($2^n - 1$ new). `&&`: the right operand runs only in the parent (non-zero return); `||`: the right operand runs only in the child (zero return).
- A process that finishes while its parent has not called `wait` is a **zombie**; if the parent dies first the child is an **orphan** (adopted by `init`).
- Threads share **code, data, heap, files**; private: **PC, registers, stack**. ULT: one block blocks all; KLT: true parallelism.
- Race: two threads × n increments, no lock ⇒ final x ∈ **[2, 2n]** (n ≥ 2).
- Mode switch ≠ context switch; user → kernel is automatic, kernel → user is a privileged instruction.

CPU scheduling (Ch 4)

- **TAT = CT − AT; WT = TAT − BT; RT = first run − AT.** Non-preemptive: RT = WT.
- Σ WT = Σ CT − Σ AT − Σ BT.
- **SJF/SRTF:** min average WT. **Exponential average: S(n+1) = α·t_n + (1−α)·S_n.**
- **RR efficiency = q / (q + s)**; q ≥ max BT ⇒ FCFS; new arrival at the instant of quantum expiry goes **before** the preempted process.
- CPU utilisation (I/O problems) = Σ CPU bursts / total time.
- **HRRN ratio = (W + BT) / BT.**

Synchronization (Ch 5)

- Three CS requirements: **ME, Progress, Bounded waiting**. Strict alternation: ME ✔ P ✘ BW=1. Peterson: all ✔, BW = 1. TSL spinlock: ME ✔ P ✔ BW ✘; BW-TSL algorithm: BW = n−1.
- Blocking semaphore: **value = −k ⇒ k blocked**; wait = P = down; signal = V = up.
- Producer–consumer: `empty = N, full = 0, mu = 1` — counting wait **before** `mu`.
- Reader–writer: `mu`, `write`, `reader_count` (first reader locks, last unlocks).
- Semaphore final value = initial − (#wait) + (#signal).

Deadlock (Ch 6)

- Coffman: **ME, hold & wait, no preemption, circular wait**.
- **Min resources = Σ(m_i−1) + 1**; max processes = ⌊**(K−1)/(x−1)**⌋.
- **Need = Max − Allocation**; safety: repeatedly pick Need ≤ Work, then Work += Allocation.
- RAG: single instances ⇒ cycle = deadlock; multi-instance ⇒ reduce.

Paging & virtual memory (Ch 8–9)

- #pages = VAS/P; #frames = PAS/P; PT = #pages × e; PTEs/page = P/e.
- **Levels = ⌈(VA bits − offset) / $\log_2$(P/e)⌉.** PTE pointer bits = $\log_2$(PAS / pointed-object size).
- **EAT (TLB, 1-level) = t + (2 − h)·m; (k-level) = t + m + (1−h)·k·m.** No TLB: (k+1)·m.
- **EAT with faults = (1−p)·m + p·S.** TLB reach = entries × P. Optimal page size = √(2se).
- Faults ≥ #distinct pages; OPT ≤ all; LRU/OPT immune to Belady; FIFO is not.
- Working-set: D = Σ|WS_i| > frames ⇒ thrashing.

Files, disk, I/O (Ch 10–11)

- **Max file = (D + n + n^2 + n^3)·B**, n = B/p. Accesses: direct 1, single 2, double 3, triple 4.
- FAT size = #blocks × entry size; FS size = 2^(entry bits) × B. Bitmap = #blocks bits.
- **Access time = seek + ½·(60/RPM) + (bytes / track bytes)·(60/RPM).**
- Disk scheduling: FCFS, SSTF (starves), SCAN (to the end), C-SCAN, LOOK, C-LOOK (to the last request).
- DMA: program controller → block transfer → **one** interrupt.

12.2 The trap list (every item has cost someone marks)

1. **Byte- vs word-addressable** memory (Ch 8) — changes address bits.
2. **Page-size and PTE-size units** — KB vs bits; "PTE of 4 bytes" ⇒ 32 bits.
3. Using **ready-queue arrival order** wrongly in RR (new arrival before the preempted process).
4. Forgetting **idle time** at the start of a Gantt chart (AT of first process > 0).
5. Confusing **response time** with **waiting time** in preemptive algorithms.
6. SRTF/LRTF **ties**: keep the running process.
7. Counting **context switches**: merge same-process segments; do not count the first dispatch or the end.
8. In I/O problems, forgetting that **I/O of different processes overlaps**.
9. **Peterson**: set the flag first, then `turn` — swapping them breaks mutual exclusion; `||` instead of `&&` deadlocks.
10. **Producer–consumer**: `wait(mu)` before `wait(empty)` ⇒ deadlock.
11. **TSL spinlock** satisfies ME and progress but **not** bounded waiting.
12. Binary semaphore: extra `signal` does **not** accumulate.
13. **Safe ≠ no request ever denied**; unsafe ≠ deadlock.
14. Deadlock minimum-resource formula: it is Σ(m_i−1)+1 — **not** Σm_i.
15. **RAG with multi-instance** resources: a cycle is not enough.
16. Belady's anomaly: only **FIFO** (not LRU/OPT).
17. Page-fault service: **dirty victim doubles** the disk work.
18. **TLB hit ratio** formulas: check whether TLB time is given or negligible.
19. **Multilevel paging**: top-level table may use fewer bits; chunks that exist are stored in full.
20. **Inode arithmetic**: pointers per block = B / p (block bytes ÷ pointer bytes). And check the disk-size limit.
21. **Disk**: rotational latency is **half a rotation** on average; seek time is not affected by RPM.

22. **SCAN goes to the physical end**, LOOK to the last request; check whether the return jump of C-SCAN is counted.
23. **DMA** interrupts once per block, not per byte.
24. **Library function ≠ system call**; `fork` after `exec` replaces the image.

12.3 Mixed practice — 12 problems (30 minutes)

M1. Two CPUs, non-preemptive SJF: P1(0,8), P2(1,4), P3(2,9), P4(3,5), P5(4,2). Average WT and TAT?

M2. VA = 46 bits, page = 8 KB, PTE = 8 B. (a) page-table levels needed? (b) size of the flat page table? (c) bits at the top level?

M3. m = 50 ns, TLB lookup = 5 ns, hit ratio 0.9, 2-level page table, page-fault rate p = 10^{-4}, fault service 5 ms. EAT?

M4. Bounded buffer N = 5 with the standard semaphores. 8 items have been produced and 3 consumed (no one blocked). Values of `empty` and `full`?

M5. A system has 5 identical printers; each process needs at most 3 at the same time. Largest number of processes for which deadlock is impossible?

M6. String 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4. FIFO faults with 3 and 4 frames? What does it show?

M7. Inode with 10 direct + single + double + triple indirect pointers, block = 4 KB, pointer = 8 B. Maximum file size?

M8. How many times is "x" printed? `if (fork() && fork() || fork()) { } printf("x\n");`

M9. Disk scheduling: queue 82, 170, 43, 140, 24, 16, 190, head 50, cylinders 0–199. Total movement for SCAN and LOOK if the head first moves **down**.

M10. Four threads W, X (add +1) and Y, Z (add −2) update shared x (initially 0) as `wait(S); r=x; r=r+k; x=r; signal(S);` with S initially **2**. Max and min final x?

M11. Process with 5 frames under LRU; a program loops over 6 pages cyclically (1,2,3,4,5,6,1,2,3,4,5,6,...). Fault rate in the long run?

M12. True/False: (a) Peterson's algorithm uses atomic test-and-set. (b) In C-SCAN the head serves requests while returning. (c) A user-level thread blocking on I/O blocks the whole process. (d) With TLB hit ratio 100 %, EAT = TLB time + m.

Solutions

M1. CPU1: P1 0–8. CPU2: P2 1–5. At t = 5 (CPU2 free): ready P3(9), P4(5), P5(2) → P5 5–7; at t = 7 CPU2: P4 7–12; at t = 8 CPU1: P3 8–17. CT = 8, 5, 17, 12, 7; TAT = 8, 4, 15, 9, 3 → avg **7.8**; WT = 0, 0, 6, 4, 1 → avg **2.2**.

M2. Offset 13 bits ⇒ page-number bits = 33. PTEs per page = $2^{13}/2^3 = 2^{10}$ ⇒ 10 bits per level ⇒ $\lceil 33/10 \rceil$ = **4 levels**. Flat PT = $2^{33} \times 8$ B = 2^{36} B = **64 GB**. Top level gets 33 − 30 = **3 bits** (split 3 | 10 | 10 | 10 | 13).

M3. Memory EAT = t + m + (1−h)·k·m = 5 + 50 + 0.1 × 2 × 50 = **65 ns**. With faults: 0.9999 × 65 + 10^{-4} × 5 × 10^6 ns = 64.99 + 500 = ≈ **565 ns**.

M4. The buffer holds 8 − 3 = 5 items = N ⇒ **full = 5, empty = 0** (the producer would now block on the next item).

M5. P·(3−1) + 1 ≤ 5 ⇒ P ≤ 2 ⇒ **2 processes**.

M6. FIFO: **9** faults with 3 frames, **10** with 4 frames ⇒ **Belady's anomaly** (more frames, more faults).

M7. n = 4096/8 = 512; blocks = 10 + 512 + 262 144 + 134 217 728 = 134 480 394; × 4 KB = 550 831 693 824 B ≈ **513 GB** (≈ **0.5 TB**). (Shortcut: $512^3 \times 4$ KB = $2^{27} \times 2^{12} = 2^{39}$ B = 512 GB dominates.)

M8. Parent: `fork()` $\neq 0 \rightarrow$ `fork()` again: parent $\neq 0$ (true) $\Rightarrow$ `||` skipped; the child of the second fork gets $0 \Rightarrow$ false $\Rightarrow$ evaluates `|| fork()` $\Rightarrow$ 2 processes. The child of the first fork gets $0 \Rightarrow$ `&&` short-circuits $\Rightarrow$ false $\Rightarrow$ `|| fork()` $\Rightarrow$ 2 processes. Count: parent side = parent + 2nd child + that child's forked process = 3; first child side = 2. Total **5** processes $\Rightarrow$ "x" printed **5 times** (verified by running it).

M9. Sorted: lower side 43, 24, 16 (descending when going down), upper side 82, 140, 170, 190. **SCAN-down:** $50 \rightarrow 0$ (50) then $0 \rightarrow 190$ (190) = **240**. **LOOK-down:** $50 \rightarrow 16$ (34) then $16 \rightarrow 190$ (174) = **208**.

M10. Two processes can be inside together, so a stale load can be overwritten. **Max = 2** (X loads 1 and stalls; Y and Z run to completion $\rightarrow$ x = -3; X stores 2). **Min = -4** (verified by exhaustive search).

M11. Cyclic access over 6 pages with 5 frames under LRU: every reference evicts exactly the page needed next $\Rightarrow$ **every reference faults (fault rate = 100 %)**; OPT would fault only about once per 5 references.

M12. (a) **False** — Peterson uses only ordinary reads/writes (assumes atomic loads/stores). (b) **False.** (c) **True.** (d) **True** (hit $\Rightarrow$ t + m).

12.4 Final-week checklist

- ☐ Can I do a 4-process RR/SRTF table in 3 minutes? (Ch 4)
- ☐ Can I solve "possible outputs" and "min/max x" semaphore questions? (Ch 5)
- ☐ Do I know the three CS requirements and how to break a solution with one schedule?
- ☐ Banker's: Need, safe sequence, request test in under 4 minutes? (Ch 6)
- ☐ Paging geometry: levels, PTE bits, flat PT size, EAT — formulas memorised? (Ch 8)
- ☐ FIFO / LRU / OPT / Clock tables — error-free? (Ch 9)
- ☐ inode max file size and access-count templates; disk access time; SCAN/LOOK. (Ch 10–11)
- ☐ Fork counting: doubling rule and `&&`/`||`. (Ch 2)

APPENDIX A

PYQ-style playbook (what to do, in what order, with time budgets)

Question style	Time	Steps
Page-table geometry (7 questions in recent papers)	2 min	(1) write VA/PA bits and page offset in powers of 2; (2) pages = 2^(VA−offset); frames likewise; (3) PTE bits (frame bits + control bits) → bytes; (4) levels = ⌈page-no bits / $\log_2$(P/e)⌉; (5) check top-level size fits a frame
Page-replacement trace	2–3 min	Draw the frame rows; mark F; FIFO queue order, LRU look back, OPT look forward; compare with min = #distinct pages
EAT with TLB / page faults	1 min	Write the formula first; check "t given or negligible"; for faults add (1−p)m + pS
Scheduling Gantt	3 min	Table AT/BT; clock; ready set; segments; CT → TAT → WT; check ΣBT; for multi-CPU, assign to the earliest-free CPU
Scheduling concepts (MSQ)	1 min	Starvation (SJF/SRTF/priority/LJF), convoy (FCFS), preemptive vs non-preemptive, response time vs WT
Semaphore output / min–max	3 min	Find the only runnable process; trace the signals; search for a stale load for max/min; deadlock = cycle of waits
CS-solution check	2 min	Hunt for an ME-violating schedule (check-then-set), a progress failure (other in remainder), then BW
Banker's / safe state	3 min	Need = Max − Alloc; scan top-down repeating; request: ≤ Need, ≤ Available, simulate, test safety
Min resources / max processes	30 s	Σ(m_i − 1) + 1
RAG / deadlock detection	1 min	Single instance: cycle; else reduce
inode / FAT arithmetic	2 min	n = B/p; (D+n+n^2+n^3)·B; accesses by range
Disk access time / scheduling	2 min	60/RPM; ½ rotation; track fraction; draw head path
Disk scheduling order	2 min	Sort requests around the head; apply algorithm; add distances
I/O event sequence	1 min	Program controller → transfer → one interrupt → ISR/process ready
fork counting	1–2 min	2^n rule, short-circuit, then draw the tree for prints
System call / mode / thread MSQ	1 min	Library ≠ syscall; trap = user→kernel automatic; threads share heap/data, own stack

Time budget for the OS section: roughly 9 marks ⇒ about 12–14 minutes. Do the **numerical 2-markers last** if time is short; do the **1-mark statements first**.

APPENDIX B

Conventions used in this book (and what to do when the question disagrees)

1. **Lower priority number = higher priority** unless the question says otherwise.
2. **Ties** in SJF/SRTF/priority: smaller AT, then smaller process id; SRTF/LRTF keep the running process on a tie.
3. **RR queue order:** a process that arrives at the same instant a quantum expires is queued **before** the preempted process.
4. **Context switches** = boundaries between *different* processes in the merged Gantt chart (the first dispatch and the final termination are not counted).
5. **I/O bursts** of different processes proceed in parallel; the CPU is the only serial resource.
6. **Page replacement:** loading into a free frame is a fault; frames start empty; local replacement with a fixed allocation.
7. **Clock algorithm:** a newly loaded page has RU = 1; the hand moves past it.
8. **Memory sizes** are powers of two (KB = 2^{10} …); **disk-scheduling distances** are in cylinders.
9. **C-SCAN / C-LOOK** return jump counted unless the question says "not counted".
10. **SCAN** goes to the disk end; **LOOK** only to the last request.
11. **TLB time negligible** only if stated.
12. A question's own definition **always overrides** these conventions.

APPENDIX C

Corrections to the lecture notes (small slips, fixed in this book)

Notes page	What the notebook says	Correct / used here
p25–26 (RR, q = 4)	RT of P1 written as 1	P1 arrives at 1 and runs at 1 $\Rightarrow$ **RT = 0** (the other RTs 3, 5, 6 are right)
p26 (RR, 200,2,2,2,2)	one burst appears as "9" in the table	The consistent value is **2** (the stated AWT 84 and ART 82.4 only work with 2)
p29 (I/O FIFO example)	RT/WT labelled with "first CPU time − I/O"	These are the **ready-queue waits** (0, 6, 20) — pure waiting time = TAT − (CPU + I/O)
p51 (bounded-waiting TSL)	listing (as transcribed) has no line after the spin loop	Added the textbook line `waiting[i] = false;` before the CS
p66 (6 instances, N processes needing 2)	option (D) "N = 4 could deadlock"	N $\leq$ 5 is **always safe**; deadlock first becomes possible at **N = 6**
p92 (multilevel paging variant)	"PT2 entries = 2^{23}"	2^{18} ($2^{28}/2^{10}$); the final split 6 | 12 | 10 | 16 | 20 is right
p94 (chunk sizes, example C)	rounds 2^20.5 to $2^{20} \Rightarrow$ 20-bit L1 PTE, 40 B chunk	Strictly **21 bits** and **42 B**; method unchanged
p116 (FAT)	FAT size "127 KB"	**128 KB** ($2^{16} \times 16$ bits = 2^{17} B)
p113 (inode)	"name?" listed among the metadata	In classic Unix the **name is in the directory entry**, not in the inode
p23 (SRTF context switches)	counts 6	Correct under the "$P_i \rightarrow P_j$ changes" convention (kept)

Topics that the notes do not contain, added here (marked ADDED): disk structure & access time, disk scheduling (FCFS/SSTF/SCAN/C-SCAN/LOOK/C-LOOK), I/O (polling, interrupts, DMA), EAT formulas (TLB, multilevel, page faults), TLB reach, optimal page size, HRRN and MLFQ, multi-CPU scheduling, semaphore-output puzzles, Belady's anomaly tables, working-set and PFF, frame allocation, copy-on-write, hard/soft links, open-file tables.

End of the book. Re-do the practice sets, then solve the previous-year papers cluster by cluster — Chapters 8 $\rightarrow$ 9 $\rightarrow$ 4 $\rightarrow$ 5 first.